

MaruBatsu 開発ガイド

Zankuu フレームワークから作る

2D〇×〇-ライト — 全システム解説と構築手順

対象: ゲームプログラミング学習者・本プロジェクト保守担当
内容: 全システム解説 + 全ソースコード + ゼロからの構築手順
制作: Natsume Shidara (N72) / 2026年5月

目次

第0章 はじめに — 本書の読み方とプロジェクト概要

- 0.1 本書の目的と対象読者
- 0.2 ゲームの全体像
- 0.3 「Zankuu基盤」とは何か
- 0.4 ディレクトリ構成と本書の用語

第1章 全体アーキテクチャ

- 1.1 レイヤ構成 — 基盤層とゲーム層
- 1.2 メインループ (main.cpp)
- 1.3 シーン管理 (scene)

第2章 基礎データ構造

- 2.1 駒と座標 (piece.h)
- 2.2 方向 (direction.h)
- 2.3 試合状態 (game_state.h)

第3章 盤面システム

- 3.1 盤面データ (board)
- 3.2 盤面操作 — スライド (board_ops)
- 3.3 勝利条件の走査 (win_check)
- 3.4 衝突判定 (collision)

第4章 フック設計 — 本プロジェクトの心臓部

- 4.1 なぜフックなのか
- 4.2 フックインターフェイス群 (hooks.h)
- 4.3 レジストリ (ability_registry)
- 4.4 デフォルト実装 (default_hooks)
- 4.5 チェインの仕組み

第5章 アビリティシステム

- 5.1 アビリティ基底クラス (ability.h)
- 5.2 抽選プール (ability_pool)
- 5.3 各アビリティの実装

第6章 ボスシステム

- 6.1 ボス基底クラス (boss.h)
- 6.2 氷盤の支配者 (ice_board_boss)
- 6.3 ボスギミック「氷盤」 (boss_ice_slide)

第7章 AI — 思考ルーチン

第8章 描画とテキスト

- 8.1 図形描画 (render_primitives)
- 8.2 テキスト描画 (text_draw)
- 8.3 動的グリフアトラス (font_atlas)
- 8.4 フェード (fade)

第9章 ラン進行とシーン

- 9.1 ラン状態 (run_state)
- 9.2 タイトルシーン (title)
- 9.3 報酬シーン (reward)

第10章 ゲームシーンの詳解

- 10.1 状態と定数
- 10.2 試合の初期化フロー
- 10.3 着手と評価のフロー
- 10.4 効果アニメーション
- 10.5 更新ループと入力
- 10.6 HUDとアビリティ履歴UI

第11章 ゼロからの構築手順

付録 ビルド設定・用語集・拡張のヒント

第0章 はじめに — 本書の読み方とプロジェクト概要

本書は、2D○×（まるばつ）ローグライト「**MaruBatsu**」を、既存の3DアクションゲームのフレームワークであるZankuuを土台として、**ゼロから再構築できる**ことを目的とした教育用ドキュメントです。各システムの設計思想・データの流れ・関数の役割を解説し、続けて全ソースコードを掲載します。最終章では「どの順番で作るか」を示します。

0.1 本書の目的と対象読者

このプロジェクトの最大の特徴は、ゲームのルールそのもの（勝利条件・着手数・思考時間・駒の挙動など）を「**フック**」として抽象化し、アビリティで自由に差し替えられる点にあります。単純な○×ゲームではなく、「ルールが書き換わっていくローグライト」を成立させるための設計が随所に盛り込まれています。

本書は次のような読者を想定しています。

- C++とDirectXの基礎を学び終え、**実際に動くゲームの設計**を学びたい学習者
- 本プロジェクトを引き継ぐ・保守する担当者
- 「拡張しやすいゲーム設計」の具体例を知りたい開発者

読み進めるには、C++20の基本文法（クラス、継承、仮想関数、`std::shared_ptr`、`std::vector`、ラムダ式）と、2Dスプライト描画の基本概念があると理解が早まります。DirectXの初期化やウィンドウ管理といった「基盤」部分の内部実装は前提知識として扱い、本書では深入りしません（0.3節参照）。

0.2 ゲームの全体像

MaruBatsuは、3×3の○×ゲームを下敷きにした**ローグライト**です。プレイヤーは○、ボス（AI）は×を担当します。基本的な流れは次の通りです。

1. **ボス戦**：制限時間（既定3分）の思考時間内に、○×の対戦を行う。縦・横・斜めのいずれかに自分の駒が3つ並べば勝利。
2. **報酬選択**：ボスを倒すと、3枚のカードから**アビリティ**を1つ選んで獲得する。
3. **次のボスへ**：獲得したアビリティはランを通じて累積し、ゲームのルールを少しずつ書き換えていく。これを繰り返してラスボスを目指す。

ボスもまた固有の「ギミック」を持ちます。たとえば最初のボス「氷盤の支配者」は、駒を置くたびに盤上の全ての駒を一方向へ滑らせます。プレイヤーが獲得するアビリティと、ボスのギミックは**同じ「フック」という仕組みの上に乗っています**。これが本設計の核心です。

本書が解説するコードは、コンテスト出展に向けた**プロトタイプ版**です。ボスは1体（氷盤の支配者）、アビリティは6種＋ボスギミック1種を実装しています。盤面サイズ拡張やレジェンドリー級アビリティなどはα版以降の予定として、フックの拡張点だけを先に用意してあります。

0.3 「Zankuu基盤」とは何か

このプロジェクトは、3DアクションゲームZankuuのソースをコピーして出発しました。3D特有のモジュール（モデル、3D当たり判定、カメラ等）は削除しましたが、2Dゲームでもそのまま使える**汎用基盤**を流用しています。本書ではこれを「**Zankuu基盤**」または「**基盤層**」と呼びます。基盤層が提供するものは次の通りです。

基盤層の機能	主な提供物	本書での扱い
ウィンドウ管理	<code>GameWindow_Create</code> 、ウィンドウプロシージャ	前提（解説対象外）
Direct3D 11	<code>Direct3D_Initialize</code> 、バックバッファ、ブレンドステート	前提（呼び出し方のみ）
2Dスプライト描画	<code>Sprite_Draw</code> 、 <code>Texture_Load</code>	前提（描画章で利用）
シェーダ	2D用頂点／ピクセルシェーダ	前提
入力	<code>Mouse</code> 、 <code>Keyboard</code> 、 <code>InputManager</code>	前提（利用方法は解説）
サウンド	<code>SoundManager</code>	前提
タイマー	<code>SystemTimer</code>	前提

つまり本書が解説する「ゲーム層」のコードは、これらの基盤APIを**呼び出す側**です。「ゼロから作る」とは、Zankuu基盤という土台がある前提で、その上にゲーム層を一から組み上げる、という意味になります。基盤層のAPIは、それぞれ `oo_Initialize` / `oo_Update` / `oo_Draw` / `oo_Finalize` という素直な 名前付けに従っているため、利用するだけなら難しくありません。

0.4 ディレクトリ構成と本書の用語

本書が解説する「ゲーム層」のファイルは、すべて `game/` ディレクトリ配下と、ルートの `main.cpp` です。構成は次の通りです。

<code>main.cpp</code>	アプリのエントリーポイント（基盤層とゲーム層を接続）
<code>game/</code>	
└ <code>piece.h</code>	駒・座標・陣営反転
└ <code>direction.h</code>	4方向の定義
└ <code>game_state.h</code>	1試合の状態スナップショット
└ <code>board.h</code> / <code>.cpp</code>	盤面データ（マス目と駒）
└ <code>board_ops.h</code> / <code>.cpp</code>	盤面操作（スライド処理）
└ <code>win_check.h</code> / <code>.cpp</code>	勝利条件の走査ロジック
└ <code>collision.h</code> / <code>.cpp</code>	2D衝突判定（汎用ユーティリティ）
└ <code>ai.h</code> / <code>.cpp</code>	AI（ボスの思考）
└ <code>run_state.h</code> / <code>.cpp</code>	1ラン（複数戦）の進行状態
└ <code>render_primitives.h</code> / <code>.cpp</code>	図形描画（線・矩形・円・×）
└ <code>text_draw.h</code> / <code>.cpp</code>	テキスト描画（UTF-8対応）
└ <code>font_atlas.h</code> / <code>.cpp</code>	動的グリフアトラス（日本語対応）
└ <code>fade.h</code> / <code>.cpp</code>	画面フェード

└ ability/	
└ hooks.h	フックインターフェイス群 ★最重要
└ ability.h	アビリティ基底クラス
└ ability_registry.*	フック実装の集約レジストリ
└ default_hooks.*	標準ルール（フックのデフォルト実装）
└ ability_pool.*	報酬抽選用プール
└ abilities/	個々のアビリティ実装（7種）
└ boss/	
└ boss.h	ボス基底クラス
└ ice_board_boss.*	ボス1「氷盤の支配者」
└ scene/	
└ scene.h / .cpp	シーン切り替え管理
└ title.h / .cpp	タイトルシーン
└ game.h / .cpp	ゲーム本編シーン ★中核
└ reward.h / .cpp	報酬選択シーン

本書で繰り返し使う用語を先に整理しておきます。

用語	意味
基盤層 / Zankuu基盤	Zankuuから流用した汎用部分（DirectX・入力・描画など）。
ゲーム層	本書が解説する <code>game/</code> 配下 + <code>main.cpp</code> 。
フック (hook)	ゲームのルールの差し替え点。インターフェイスとして抽象化されている。
アビリティ (ability)	フックを実装し、ゲームのルールを書き換える効果。報酬で獲得する。
レジストリ (registry)	現在有効なフック実装を集約・呼び出す窓口（ <code>AbilityRegistry</code> ）。
着手 (placement)	1個の駒を盤面に置くこと。
ターン (turn)	一方の陣営の手番。1ターンに複数着手することもある（二手打ち等）。
ラン (run)	タイトルから開始し、複数のボス戦を続ける1回の挑戦全体。
フェーズ (phase)	効果アニメーションの再生単位。1フェーズ＝同時に起きる移動の集合。

本書の構成

第1～10章で各システムを「解説 → 全ソース掲載」の順に扱います。第11章では、これらをどの順番で組み上げるかを構築手順としてまとめます。まず通読して全体像を掴み、実際に作るときは第11章を地図として使う、という読み方を推奨します。

第1章 全体アーキテクチャ

本章では、アプリケーション全体の骨格を概観します。基盤層とゲーム層がどう接続され、1フレームがどう処理され、3つのシーン（タイトル・ゲーム・報酬）がどう切り替わるかを押さえます。個々のゲームロジックは第2章以降で詳述します。

1.1 レイヤ構成 — 基盤層とゲーム層

このアプリは大きく2層に分かれます。下が**基盤層**（Zankuu由来の汎用機能）、上が**ゲーム層**（本書が解説する○×ログライト固有のコード）です。

ゲーム層 (game/ + main.cpp)	
シーン (タイトル / ゲーム / 報酬)	
盤面・フック・アビリティ・ボス・AI・UI	
基盤層 (Zankuu由来)	
ウィンドウ / Direct3D11 / スプライト描画 / 入力	
サウンド / タイマー / フェード	

ゲーム層は基盤層のAPIを**呼び出すだけ**で、基盤層がゲーム層を直接知ることはありません（依存は一方向）。この分離のおかげで、ゲーム層を一から書き直しても基盤層はそのまま使えます。本書の「ゼロから作る」とは、この上半分を組み上げることを指します。

ほとんどの基盤モジュールは `oo_Initialize()` / `oo_Update()` / `oo_Draw()` / `oo_Finalize()` という統一された関数群を公開します。ゲーム層のシーンやサブシステムも、この命名規則に ならって作られています。「初期化・更新・描画・終了」の4点セットで考える、という方針をプロジェクト全体で一貫させているわけです。

1.2 メインループ (main.cpp)

`main.cpp` は唯一のエントリーポイントであり、基盤層とゲーム層を「接続する配線」です。役割は3つに集約されます。

1. **初期化**：基盤層の各サブシステムを正しい順序で立ち上げ、最後にシーンを初期化する。
2. **メインループ**：毎フレーム「更新 → 描画 → 表示 → シーン遷移確定」を繰り返す。
3. **終了処理**：初期化と**逆順**でサブシステムを片付ける。

初期化の順序には依存関係があります。たとえばスプライト描画やフォントアトラスは Direct3Dデバイスを必要とするため、`Direct3D_Initialize` の後でなければ 初期化できません。逆に終了処理は依存される側を後に片付けるため、初期化と逆順にします。

メインループ（`WM_QUIT` まで回る `do-while`）の1フレームは次の流れです。

PeekMessage でOSメッセージあり? —Yes→ Translate/Dispatch (ウィンドウ処理)

|No



[更新フェーズ]

SystemTimer から経過時間を取得

KeyLogger_Update / PadLogger_Update ← 入力スナップショット更新

Scene_Update(elapsed_time) ← 現在シーンの更新

SpriteAnim_Update / Fade_Update

[描画フェーズ]

Scene_Draw() ← 現在シーンの描画

Fade_Draw() ← フェードを最前面に重ねる

(DEBUGビルドのみ) FPS表示

Direct3D_Present() ← バックバッファを画面へ (VSync待ち)

Scene_Refresh() ← 予約されたシーン遷移をここで確定

ポイントは、**シーン遷移の確定 (Scene_Refresh)** を**フレーム末尾で行う**ことです。更新や描画の途中でシーンを切り替えてしまうと、「更新は旧シーン・描画は新シーン」のような 不整合が起きます。そこで遷移は「予約」だけにとどめ、1フレームの処理を完全に終わってから 切り替える設計になっています（詳細は次節）。

フレームレートは Direct3D_Present() の内部でVSync（垂直同期）に同期するため、モニタのリフレッシュレートに追従します。経過時間 elapsed_time は SystemTimer から取得した実測値で、これを各 Update に渡すことで、フレームレートに依存しない時間進行（タイマー消費・アニメーション）を実現します。

main.cpp

```
/*
*****
* @file    main.cpp
* @brief   アプリケーションエントリーポイント (2D版)
* @author  Natsume Shidara
* @date    2025/06/06
* @update  2026/05/15 - O×ローグライト用2D版へスリム化
*
* WinMainを含むメインループの管理。
* 各サブシステムの初期化・終了を制御し、
* フレーム更新と描画のループを実行する。
*
* フレームレート制御:
*   VSync (垂直同期) により、モニターのリフレッシュレートに同期。
*   Direct3D_Present() 内の Present(1, 0) で制御。
*****
*/

#include <SDKDDKVer.h>
#define WIN32_LEAN_AND_MEAN
#include <Windows.h>
#include <sstream>

//-----
// ウィンドウ・入力
//-----
#include "game_window.h"
#include "key_logger.h"
#include "pad_logger.h"
#include "mouse.h"

//-----
// タイマー
//-----
#include "system_timer.h"
```



```

//-----
// サウンド・設定
//-----
#include "sound_manager.h"
#include "game_settings.h"

//-----
// Direct3D・シェーダー
//-----
#include "direct3d.h"
#include "sampler.h"
#include "shader.h"

//-----
// 描画リソース
//-----
#include "sprite.h"
#include "texture.h"
#include "sprite_anim.h"

//-----
// ゲームロジック・UI
//-----
#include "scene.h"
#include "fade.h"
#include "debug_text.h"
#include "text_draw.h"

//=====
// アプリケーションエントリーポイント
//=====
/**
 * @brief Windowsアプリケーションのエントリーポイント
 * @param hInstance アプリケーションのインスタンスハンドル
 * @param nCmdShow ウィンドウの初期表示状態
 * @return 終了コード (WM_QUIT の wParam)
 * @detail サブシステムを初期化 → メインループ (更新+描画) →
 * 初期化と逆順で終了処理、という流れを管理する。
 */
int APIENTRY WinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE, _In_ LPSTR,
_In_ int nCmdShow)
{
    //-----
    // 各サブシステムの初期化
    //-----
    // COM・DPI初期化
    (void)CoInitializeEx(nullptr, COINIT_MULTITHREADED);
    SetProcessDpiAwarenessContext(DPI_AWARENESS_CONTEXT_PER_MONITOR_AWARE_V2);

    // ウィンドウ作成
    HWND hWnd = GameWindow_Create(hInstance);

    // 入力・タイマー初期化
    SystemTimer_Initialize();
    KeyLogger_Initialize();
    Mouse_Initialize(hWnd);
    PadLogger_Initialize();

    // サウンド・ゲーム設定初期化
    SoundManager_Initialize();
    GameSettings_Initialize();

    // Direct3D・2Dシェーダー初期化

```

```

Direct3D_Initialize(hWnd);
Sampler_Initialize(Direct3D_GetDevice(), Direct3D_GetContext());
Shader_Initialize(Direct3D_GetDevice(), Direct3D_GetContext());

// 描画リソース初期化
Sprite_Initialize(Direct3D_GetDevice(), Direct3D_GetContext());
Texture_Initialize(Direct3D_GetDevice(), Direct3D_GetContext());
SpriteAnim_Initialize();
Fade_Initialize();

// デバッグテキスト初期化
hal::DebugText debugText(Direct3D_GetDevice(), Direct3D_GetContext(),
    L"assets/consolab_ascii_512.png",
    Direct3D_GetBackBufferWidth(), Direct3D_GetBackBufferHeight(),
    0.0f, 0.0f, 0, 0, 0.0f, 16.0f);

// マウス・シーン初期化
Mouse_SetVisible(true);
Mouse_SetMode(MOUSE_POSITION_MODE_ABSOLUTE);
Scene_Initialize();

// ウィンドウ表示
ShowWindow(hWnd, nCmdShow);
UpdateWindow(hWnd);

//-----
// フレーム計測用変数
//-----
SystemTimer_Reset();
double fps_last_time = SystemTimer_GetTime(); // 直近FPS計測時刻
double current_time = 0.0; // 現在時刻
ULONG frame_count = 0; // 計測区間のフレーム数
double fps = 0.0; // 算出されたFPS
double elapsed_time = 0.0; // 当該フレームの経過時間

//=====
// メインループ
//=====
MSG msg = {};

do
{
    if (PeekMessage(&msg, nullptr, 0, 0, PM_REMOVE))
    {
        // ウィンドウメッセージ処理
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    else
    {
        /*--- FPS計測 (1秒ごとに更新) ---*/
        current_time = SystemTimer_GetTime();
        double fps_elapsed_time = current_time - fps_last_time;

        if (fps_elapsed_time >= 1.0)
        {
            fps = frame_count / fps_elapsed_time;
            fps_last_time = current_time;
            frame_count = 0;
        }

        elapsed_time += SystemTimer_GetElapsedTime();

        //-----

```

```

        // 更新処理
        //-----
        KeyLogger_Update();
        PadLogger_Update();
        Scene_Update(elapsed_time);
        SpriteAnim_Update(elapsed_time);
        Fade_Update(elapsed_time);

        //-----
        // 描画処理
        //-----
        Scene_Draw();

        // フェード・デバッグ表示（最前面）
        Direct3D_SetBlendState(BlendMode::Alpha);
        Fade_Draw();

#ifdef DEBUG || defined(_DEBUG)
        // デバッグビルドのみ：FPSを画面に表示
        std::stringstream ss;
        ss << "fps:" << fps << std::endl;
        debugText.SetText(ss.str().c_str());
        debugText.Draw();
        debugText.Clear();
#endif

        // バックバッファ表示（VSync待ち）
        Direct3D_Present();
        Scene_Refresh();

        frame_count++;
        elapsed_time = 0.0;
    }
} while (msg.message != WM_QUIT);

//-----
// 終了処理（初期化の逆順）
//-----
// シーン
Scene_Finalize();

// ゲーム設定・サウンド
GameSettings_Finalize();
SoundManager_Finalize();

// 描画リソース
Text::Finalize();
Fade_Finalize();
SpriteAnim_Finalize();
Texture_Finalize();
Sprite_Finalize();

// シェーダー
Shader_Finalize();

// Direct3D・サンプラー
Sampler_Finalize();
Direct3D_Finalize();

// 入力
Mouse_Finalize();

return static_cast<int>(msg.wParam);
}

```

1.3 シーン管理 (scene)

本作には3つのシーンがあります。enum class Scene で表され、同時に有効なのは常に1つだけです。

シーン	役割
Scene::TITLE	タイトル画面。入力でゲーム本編へ。
Scene::GAME	ボス戦本編。盤面プレイ。
Scene::REWARD	ボス撃破後の報酬（アビリティ3択）画面。

scene.cpp は、現在シーン g_Scene と、次フレームの遷移先 g_SceneNext の2つの変数を持ちます。Scene_Update / Scene_Draw は g_Scene を見て、対応するシーンの関数（Title_Update、Game_Draw など）へ switch で振り分けます。これは「ポリモーフィズムを使わない素朴なディスパッチ」で、シーン数が少ないうちはこれで十分シンプルに保てます。

遷移は「予約 → 確定」の2段階

遷移の流れは次の通りです。

```
どこかで Scene_Change(Scene::GAME) を呼ぶ
→ g_SceneNext = GAME（予約するだけ。まだ切り替わらない）

フレーム末尾で Scene_Refresh()
→ g_Scene != g_SceneNext なら：
    SoundManager_StopAll()   旧シーンの音を止める
    Scene_Finalize()         旧シーンの終了処理
    g_Scene = g_SceneNext    切り替え
    Scene_Initialize()       新シーンの初期化
```

この2段階により、シーンの Finalize と次シーンの Initialize が 必ずフレームの境界で実行されることが保証されます。Game_Update の中で Scene_Change(Scene::REWARD) を呼んでも、その瞬間にゲームシーンの後片付けが 走ることはなく、安全です。

描画前のレンダーステート設定

Scene_Draw は各シーンの描画に先立ち、バックバッファをレンダーターゲットに 設定し、深度テストを無効化（2Dなので不要）、アルファブレンドを有効化、Sprite_Begin() を呼びます。各シーンはこの「2D描画の準備が済んだ状態」を前提に描画コードを書けます。

```
game/scene/scene.h

/*****
 * @file   scene.h
 * @brief シーン管理（2D版）
 * @author Natsume Shidara
 * @date   2025/07/10
 * @update 2026/05/15 - O×ローグライト用2D版へ書き換え
 *
 * タイトル/ゲーム/報酬の3シーンを切り替える。遷移は予約制で、
 * Scene_Refresh() のタイミングでのみ実際の切り替えが確定する。
 *****/
```

```

*****/

#ifndef SCENE_H
#define SCENE_H

//=====
// シーン種別
//=====
/**
 * @enum Scene
 * @brief 画面の種類を表す列挙型
 */
enum class Scene
{
    TITLE,    // タイトル画面
    GAME,     // ゲーム本編（盤面プレイ）
    REWARD,  // 撃破報酬の選択画面
};

//=====
// シーン管理 API
//=====
/** @brief 現在シーンの初期化 */
void Scene_Initialize();

/** @brief 現在シーンの終了処理 */
void Scene_Finalize();

/**
 * @brief シーン更新
 * @param elapsed_time 前フレームからの経過秒数
 */
void Scene_Update(double elapsed_time);

/** @brief 現在シーンの描画（2D） */
void Scene_Draw();

/** @brief 予約済みシーン遷移の確定処理（毎フレーム呼ぶ） */
void Scene_Refresh();

/**
 * @brief 次フレーム以降に切り替えるシーンを予約する
 * @param scene 遷移先シーン
 */
void Scene_Change(Scene scene);

#endif // SCENE_H

```

game/scene/scene.cpp

```

/*****
 * @file scene.cpp
 * @brief シーン管理（2D版）
 * @author Natsume Shidara
 * @date 2025/07/10
 * @update 2026/05/15 - O×ローグライト用2D版へ書き換え
 *****/

#include "scene.h"
#include "game.h"
#include "title.h"
#include "reward.h"

```

```

#include "input_manager.h"
#include "direct3d.h"
#include "sprite.h"
#include "sound_manager.h"

//-----
// 内部状態
//-----
static auto g_Scene = Scene::TITLE; // 現在のシーン
static Scene g_SceneNext = g_Scene; // 次フレームで切り替える予約先シーン

//=====
// シーン初期化／終了
//=====
void Scene_Initialize()
{
    // 現在シーンに応じた初期化関数へ振り分ける
    switch (g_Scene)
    {
        case Scene::TITLE: Title_Initialize(); break;
        case Scene::GAME: Game_Initialize(); break;
        case Scene::REWARD: Reward_Initialize(); break;
        default: break;
    }
}

void Scene_Finalize()
{
    // 現在シーンに応じた終了関数へ振り分ける
    switch (g_Scene)
    {
        case Scene::TITLE: Title_Finalize(); break;
        case Scene::GAME: Game_Finalize(); break;
        case Scene::REWARD: Reward_Finalize(); break;
        default: break;
    }
}

//=====
// シーン更新
//=====
void Scene_Update(double elapsed_time)
{
    // 入力状態を先に更新してから各シーンへ渡す
    InputManager_Update();

    switch (g_Scene)
    {
        case Scene::TITLE: Title_Update(elapsed_time); break;
        case Scene::GAME: Game_Update(elapsed_time); break;
        case Scene::REWARD: Reward_Update(elapsed_time); break;
        default: break;
    }
}

//=====
// シーン描画
//=====
void Scene_Draw()
{
    /*--- 2D描画用のレンダーステートを設定 ---*/
    Direct3D_SetBackBufferRenderTarget();
    Direct3D_Clear();
    Direct3D_DepthStencilStateDepthIsEnable(false); // 2Dなので深度テスト無効

```

```

Direct3D_SetBlendState(BlendMode::Alpha);
Sprite_Begin();

/*--- 現在シーンの描画関数へ振り分け ---*/
switch (g_Scene)
{
case Scene::TITLE: Title_Draw(); break;
case Scene::GAME: Game_Draw(); break;
case Scene::REWARD: Reward_Draw(); break;
default: break;
}
}

//=====
// シーン遷移
//=====
void Scene_Refresh()
{
    // 予約先が現在シーンと異なる場合のみ、終了→切替→初期化を行う
    if (g_Scene != g_SceneNext)
    {
        SoundManager_StopAll(); // 旧シーンの再生音を停止
        Scene_Finalize();
        g_Scene = g_SceneNext;
        Scene_Initialize();
    }
}

void Scene_Change(Scene scene)
{
    // 実際の切り替えは Scene_Refresh() まで遅延させる
    g_SceneNext = scene;
}

```

第2章 基礎データ構造

ゲームのほぼ全モジュールが参照する、最小単位の変数型を定義します。「駒」「座標」「方向」「試合状態」の4つです。ここを最初に固めることで、以降のシステムは共通の語彙で書けるようになります。

2.1 駒と座標 (piece.h)

piece.h は3つの要素を定義します。いずれもヘッダのみ（実装ファイル不要）です。

- `enum class Piece` — 1マスの状態。Empty（空き）／Player（プレイヤーの○）／Enemy（ボスの×）の3値。盤面はこの値の集合として表現されます。
- `struct Vec2` — `int x, y` の2D整数座標。盤面のマス座標に使います。x は列（横）、y は行（縦）。負値（例 {-1, -1}）を「無効座標」の意味で使う慣習があります。
- `Opponent(Piece)` — 相手陣営を返すヘルパ。Player⇔Enemy を反転します。ターン交代で多用します。

ここで重要なのは、Piece が「駒の所有者」と「マスの状態」を兼ねている点です。空きマスも1つの Piece 値（Empty）として扱うことで、盤面を「Piece の配列」というシンプルな形で表現でき、特別扱いが減ります。

game/piece.h

```
/*
 * @file piece.h
 * @brief 駒の種類・2D整数座標・陣営反転ヘルパ
 * @author Natsume Shidara
 * @date 2026/05/15
 *
 * ○×ゲームの最小データ型をまとめたヘッダ。
 * 盤面(Board)・各種フック・AI など、ほぼ全モジュールが参照する。
 */
#ifndef PIECE_H
#define PIECE_H

//=====
// 駒の種類
//=====
/**
 * @enum Piece
 * @brief 1マスに置かれている駒の状態
 * @detail Empty=空きマス / Player=プレイヤーの○ / Enemy=AI(ボス)の×。
 * 盤面はこの値の集合で表現される。
 */
enum class Piece
{
    Empty,    // 空きマス
    Player,   // プレイヤーの駒 (○)
    Enemy,    // 敵(ボス)の駒 (×)
};

//=====
// 2D整数座標
```



```
//=====
/**
 * @struct Vec2
 * @brief 盤面のマス座標などに使う2D整数ベクトル
 * @detail x は列(横)、y は行(縦)。負値は「無効座標」を表す用途にも使う。
 */
struct Vec2
{
    int x; // 列(横方向)のインデックス
    int y; // 行(縦方向)のインデックス
};

//=====
// ヘルパ関数
//=====
/**
 * @brief 相手陣営の駒種を返す
 * @detail Player⇔Enemy を反転する。ターン交代の判定などで使用。
 * @param p 反転元の駒種
 * @return 相手の駒種。p が Empty の場合は Empty を返す。
 */
inline Piece Opponent(Piece p)
{
    if (p == Piece::Player) return Piece::Enemy;
    if (p == Piece::Enemy) return Piece::Player;
    return Piece::Empty;
}

#endif // PIECE_H
```

2.2 方向 (direction.h)

direction.h は上下左右の4方向 `enum class Direction` と、方向→数値ベクトルへの変換ヘルパを提供します。氷盤（全駒スライド）、氷駒（1マス滑走）、方向インジケータの矢印描画など、「方向」を扱うすべての箇所で共通利用します。

- `DirX(d) / DirY(d)` — 方向のX成分・Y成分（-1 / 0 / +1）。座標 `(x,y)` に足せば1マス先が得られます。
- `DirectionName(d)` — 「上／下／左／右」の文字列。UI表示用。

画面座標系に注意

`DirY` は **Up が -1、Down が +1** を返します。これは「画面の上ほどY座標が 小さい」という2Dスクリーン座標系に合わせた定義です。盤面のマス座標もこの向きに統一しておく、描画とロジックの座標がそのまま一致し、変換ミスが減ります。

game/direction.h

```
/*=====
 * @file direction.h
 * @brief 4方向の列挙と、方向→ベクトル/名称への変換ヘルパ
 * @author Natsume Shidara
 * @date 2026/05/15
 *
 * 氷盤の支配者(全駒スライド)・押し出し・ワープ駒など、
 * 「方向」を扱うアビリティ全般で共通利用する。
 *=====*/
#ifndef DIRECTION_H
```

```

#define DIRECTION_H

//=====
// 方向の種別
//=====
/**
 * @enum    Direction
 * @brief   上下左右の4方向
 * @detail  スライド方向・押し出し方向などに用いる。
 */
enum class Direction
{
    Up,      // 上
    Down,    // 下
    Left,    // 左
    Right,   // 右
};

//=====
// 変換ヘルパ
//=====
/**
 * @brief   方向の X 成分(-1/0/+1)を返す
 * @param   d 対象の方向
 * @return  Left=-1 / Right=+1 / その他=0
 */
inline int DirX(Direction d)
{
    switch (d)
    {
        case Direction::Left: return -1;
        case Direction::Right: return 1;
        default: return 0;
    }
}

/**
 * @brief   方向の Y 成分(-1/0/+1)を返す
 * @detail  画面座標系に合わせ、Up が -1・Down が +1。
 * @param   d 対象の方向
 * @return  Up=-1 / Down=+1 / その他=0
 */
inline int DirY(Direction d)
{
    switch (d)
    {
        case Direction::Up: return -1;
        case Direction::Down: return 1;
        default: return 0;
    }
}

/**
 * @brief   方向の日本語名(1文字)を返す
 * @detail  UI(方向インジケータ等)の表示に使用する。
 * @param   d 対象の方向
 * @return  "上"/"下"/"左"/"右"。不正値は "?"。
 */
inline const char* DirectionName(Direction d)
{
    switch (d)
    {
        case Direction::Up: return "上";
        case Direction::Down: return "下";
    }
}

```

```

    case Direction::Left: return "左";
    case Direction::Right: return "右";
    }
    return "?";
}

#endif // DIRECTION_H

```

2.3 試合状態 (game_state.h)

`GameState` は、1試合の状態を丸ごと1つにまとめた構造体です。盤面・手番・残り思考時間・着手数・決着情報・効果アニメーション情報を持ちます。

この構造体が重要なのは、**フック（第4章）に渡される共通の引数**だからです。勝利条件・設置イベント・ターン処理といった各フックは、この `GameState` を受け取り、必要に応じて参照・書き換えを行います。「試合に関する情報は全部ここにある」という1か所に集約することで、フックの引数がシンプルになり、新しいフックを足すときも 迷いません。

メンバ	意味
<code>board</code>	盤面データ（第3章）。
<code>currentPlayer</code>	現在の手番（ <code>Player</code> か <code>Enemy</code> ）。
<code>remainingTime</code>	現ターンの残り思考時間（秒）。0以下で時間切れ。
<code>turnCount</code>	経過ターン数。
<code>placementsRemaining</code>	現ターンに「あと何個置けるか」。通常1、二手打ちで2。
<code>result</code>	試合結果（ <code>MatchResult</code> ）。進行中／勝利／敗北／引分／時間切れ。
<code>winner</code>	決着時の勝者。
<code>animPhases</code>	効果アニメーションのフェーズ列（第10章で詳述）。

`MatchResult` 列挙型は、試合の進行状態と決着種別を1つにまとめています。`Playing` 以外の値になった瞬間、試合は決着しています。`animPhases` は「駒を動かす効果」がアニメ用の移動記録を積むための領域で、基礎データのうちでは唯一やや特殊です。これは第10章「効果アニメーション」で扱います。

game/game_state.h

```

/*****
 * @file   game_state.h
 * @brief  1試合の状態(フック設計の引き渡し対象)
 * @author Natsume Shidara
 * @date   2026/05/15
 * @update 2026/05/22 - 効果アニメーション用フェーズ列を追加
 *
 * 盤面・現プレイヤー・ターン残り時間などの、1試合のスナップショットを
 * まとめた構造体。各フック(IWinCondition等)はこの GameState を
 * 受け取り、参照および書き換えを行う。
 *****/

#ifndef GAME_STATE_H
#define GAME_STATE_H

```

```

//-----
// インクルード
//-----
#include "board.h"      // 盤面データ Board
#include "board_ops.h"  // BoardOps::MoveList (アニメ用の移動記録)
#include "piece.h"      // Piece / MatchResult が参照

#include <vector>

//=====
// 試合結果
//=====
/**
 * @enum   MatchResult
 * @brief  1試合の進行状態 / 決着種別
 */
enum class MatchResult
{
    Playing, // 進行中
    Win,      // プレイヤー勝利
    Lose,     // AI(ボス)勝利
    Draw,     // 引き分け (盤面が満杯で決着なし)
    Timeout,  // 時間切れ (プレイヤーの敗北)
};

//=====
// 試合状態
//=====
/**
 * @struct GameState
 * @brief  1試合の全状態をまとめたスナップショット
 * @detail フック群はこの構造体を介して盤面やターン情報を読み書きする。
 *          ゲームシーンが唯一の実体を保持する。
 */
struct GameState
{
    //-----
    // 盤面・ターン情報
    //-----
    Board      board;                                // 盤面データ
    Piece       currentPlayer = Piece::Player;       // 現在の手番
    double      remainingTime  = 0.0;                // 現ターンの残り思考時間(秒)
    int         turnCount      = 0;                  // 経過ターン数
    int         placementsRemaining = 0;              // 現ターンの残り着手数

    //-----
    // 決着情報
    //-----
    MatchResult result = MatchResult::Playing; // 試合結果
    Piece        winner = Piece::Empty;        // 勝者(決着時のみ有効)

    //-----
    // 効果アニメーション
    //-----
    // 1回の設置で発生した効果を「フェーズ」単位で記録する。
    // 各フェーズ内の移動は同時、フェーズ間は順次再生される。
    // 駒を動かす効果フック(氷盤/氷駒/連鎖/爆弾など)が自分の移動を
    // 1フェーズとして push し、ゲームシーンが順番にアニメ再生する。
    std::vector<BoardOps::MoveList> animPhases;
};

#endif // GAME_STATE_H

```

第3章 盤面システム

○×ゲームの「土俵」である盤面を扱うシステムです。盤面データそのもの、駒を動かす スライド操作、勝利条件の走査、そして汎用の衝突判定を解説します。ここはアビリティやボスギミックから繰り返し呼ばれる土台になります。

3.1 盤面データ (board)

`Board` クラスは、`width × height` のマス目を保持します。内部表現は1次元の `std::vector<Piece>` で、座標 `(x,y)` は `index = y * width + x` で1次元インデックスに対応づけます (`Index()`)。2次元配列ではなく1次元 `vector` にしているのは、サイズ可変 (`Resize`) を 素直に扱うためです。

境界外アクセスの安全化

`Board` の設計でとくに大事なのは、`Get` と `Set` が 盤外座標を安全に扱う点です。

- `Get(x,y)` — 盤外なら `Piece::Empty` を返す。
- `Set(x,y,p)` — 盤外なら何もしない（無視）。

この「盤外 = 空マス扱い」のおかげで、勝利条件の走査やスライド処理が、わざわざ境界チェックを書かなくても `Board::Get` を呼ぶだけで安全に書けます。たとえば後述の `win_check` は「1マス手前が同色か」を `Get(x-dx,y-dy)` で 調べますが、手前が盤外でも `Empty` が返るので破綻しません。境界処理を `Board` の中に1か所だけ閉じ込める、という考え方です。

サイズ変更 (Resize)

`Resize(w,h)` は盤面サイズを変えて全マスを空に戻します。これは「盤面拡張」アビリティ（第5章）が、試合開始時に3×3を4×4へ広げるために使います。盤面サイズを固定値でハードコードせず、最初から可変にしてあるのがポイントです。

game/board.h

```
/*
 * @file board.h
 * @brief 盤面データ (マス目と駒の管理)
 * @author Natsume Shidara
 * @date 2026/05/15
 *
 * width × height のマス目で駒を保持する。内部は1次元 vector で、
 * index = y * width + x の対応で2D座標と紐付ける。
 * 盤面拡張アビリティ (α版予定) に備え Resize も用意している。
 */
#ifndef BOARD_H
#define BOARD_H

//-----
// インクルード
//-----
#include "piece.h"
#include <vector>
```

```

//=====
// 盤面クラス
//=====
/**
 * @class Board
 * @brief O×ゲームの盤面
 * @detail マス目(セル)の集合を保持し、駒の取得/設置/状態判定を提供する。
 *         デフォルトサイズは 3×3。
 */
class Board
{
public:
    //-----
    // メンバ変数 (盤面データ)
    //-----
    int width  = 3;          // 横のマス数
    int height = 3;          // 縦のマス数
    std::vector<Piece> cells; // 全マスの駒 (1次元配列, index = y*width + x)

    //=====
    // コンストラクタ
    //=====
    /** @brief デフォルト構築 (3×3で初期化) */
    Board();
    /**
     * @brief サイズ指定で構築
     * @param w 横マス数
     * @param h 縦マス数
     */
    explicit Board(int w, int h);

    //=====
    // 盤面操作
    //=====
    /** @brief 全マスを空に戻す */
    void Reset();
    /**
     * @brief 盤面サイズを変更し、全マスを空に戻す
     * @param w 新しい横マス数
     * @param h 新しい縦マス数
     */
    void Resize(int w, int h);

    /**
     * @brief 指定マスの駒を取得
     * @param x,y マス座標
     * @return マスの駒。盤外の場合は Piece::Empty。
     */
    Piece Get(int x, int y) const;
    /**
     * @brief 指定マスに駒を設置
     * @detail 盤外座標を渡した場合は何もしない。
     * @param x,y マス座標
     * @param p   設置する駒
     */
    void Set(int x, int y, Piece p);

    //=====
    // 状態判定
    //=====
    /** @brief 座標が盤内かを判定 @return 盤内なら true */
    bool IsValid(int x, int y) const;
    /** @brief 指定マスが盤内かつ空かを判定 @return 空マスなら true */

```

```

bool IsEmpty(int x, int y) const;
/** @brief 盤面が満杯(空きマス無し)かを判定 @return 満杯なら true */
bool IsFull() const;

/** @brief 空きマスの数を返す */
int EmptyCount() const;
/**
 * @brief 2D座標を cells の1次元インデックスへ変換
 * @param x,y マス座標
 * @return cells 内のインデックス
 */
int Index(int x, int y) const { return y * width + x; }
};

#endif // BOARD_H

```

game/board.cpp

```

/*****
 * @file board.cpp
 * @brief 盤面データの実装
 * @author Natsume Shidara
 * @date 2026/05/15
 *****/
#include "board.h"

//=====
// コンストラクタ
//=====
/** @brief デフォルト構築 - 既定サイズ(3x3)で全マスを空に初期化 */
Board::Board()
{
    Reset();
}

/** @brief サイズ指定で構築 - w×h で全マスを空に初期化 */
Board::Board(int w, int h)
    : width(w), height(h)
{
    Reset();
}

//=====
// 盤面操作
//=====
void Board::Reset()
{
    // width × height 個のマスを全て Empty で埋め直す
    cells.assign(static_cast<size_t>(width) * height, Piece::Empty);
}

void Board::Resize(int w, int h)
{
    width = w;
    height = h;
    Reset(); // サイズ変更に伴い全マスを再確保
}

Piece Board::Get(int x, int y) const
{
    if (!IsValid(x, y)) return Piece::Empty; // 盤外は空マス扱い
    return cells[Index(x, y)];
}

```

```

void Board::Set(int x, int y, Piece p)
{
    if (!IsValid(x, y)) return;           // 盤外への設置は無視
    cells[Index(x, y)] = p;
}

//=====
// 状態判定
//=====
bool Board::IsValid(int x, int y) const
{
    return x >= 0 && x < width && y >= 0 && y < height;
}

bool Board::IsEmpty(int x, int y) const
{
    return IsValid(x, y) && cells[Index(x, y)] == Piece::Empty;
}

bool Board::IsFull() const
{
    // 1つでも空マスがあれば満杯ではない
    for (Piece p : cells)
    {
        if (p == Piece::Empty) return false;
    }
    return true;
}

int Board::EmptyCount() const
{
    int count = 0;
    for (Piece p : cells)
    {
        if (p == Piece::Empty) ++count;
    }
    return count;
}

```

3.2 盤面操作 — スライド (board_ops)

`BoardOps` 名前空間は、駒を移動させる「純粋な盤面操作」をまとめます。これらは氷盤（ボスギミック）や氷駒（アビリティ）から共通で呼ばれます。

移動の記録 — Move / MoveList

スライド関数は、盤面を書き換えるだけでなく「**どの駒がどこからどこへ動いたか**」を記録できます。これが `BoardOps::Move`（from座標・to座標・駒種）であり、その集まりが `MoveList` です。**1つの `MoveList` = 同時に起きる移動の集合 = アニメーションの1フェーズ**、という対応になっています。盤面を一瞬で書き換えても、この記録があれば「駒が滑っていく様子」を後からアニメーション再生できます（第10章）。

SlideAll — 全駒スライド（2048タイル風）

`SlideAll(board, dir, outMoves)` は、盤上の全ての駒を指定方向の端まで 詰めます。アルゴリズムは行（または列）ごとに次の3段階です。

1. そのライン上の非空マスの駒を、**出現順を保ったまま**収集する。
2. ラインを一旦すべて空にする。
3. 端から詰め直して再配置し、実際に位置が変わった駒を `outMoves` に記録する。

「左右スライドなら行ごと・上下スライドなら列ごと」「右下方方向なら末尾寄せ・左上方向なら先頭寄せ」を、`horizontal` と `toEnd` の2つの真偽値と `cellAt` ラムダで切り替えています。4方向それぞれに別ループを書かず、1つのロジックでまとめているのが読みどころです。

SlideOne — 1マスだけ滑らせる

`SlideOne` は指定マスの駒を1マスだけ動かします。滑り先が **盤内かつ空マス**のときのみ移動し、移動できたかを `bool` で返します。氷駒アビリティ（駒1個を1マス滑らせる）が、これを所持数ぶん繰り返して使います。

game/board_ops.h

```
/*
 * @file    board_ops.h
 * @brief   盤面操作ユーティリティ (スライド処理)
 * @author   Natsume Shidara
 * @date    2026/05/15
 * @update  2026/05/22 - スライド移動の記録に対応 (アニメーション用)
 *
 * 氷盤 (全駒スライド)・氷駒 (1マススライド) など、駒を移動させる
 * 効果フックから共通で呼ばれる、純粋な盤面操作をまとめる。
 */

#ifndef BOARD_OPS_H
#define BOARD_OPS_H

// -----
// インクルード
// -----
#include "direction.h"
#include "piece.h"

#include <vector>

class Board; // 前方宣言

// =====
// 盤面操作 名前空間
// =====
namespace BoardOps
{
    // -----
    // 移動記録の型
    // -----
    /**
     * @struct Move
     * @brief 1駒の移動記録 (from → to)
     * @detail アニメーション再生で「どの駒がどこからどこへ動いたか」を
     *          再現するために使う。
     */
    struct Move
```

```

{
    int    fromX, fromY; // 移動元のマス座標
    int    toX,  toY;   // 移動先のマス座標
    Piece piece;        // 移動した駒の種別
};

/** @brief 同時に起こる移動の集まり (1効果 = 1フェーズ) */
using MoveList = std::vector<Move>;

//-----
// スライド操作
//-----
/**
 * @brief 全ての駒を指定方向の端まで滑らせる (2048タイル風)
 * @detail 各ライン(行または列)ごとに、駒の順序を保ったまま端へ詰める。
 * @param board 対象の盤面 (直接書き換える)
 * @param dir   滑らせる方向
 * @param outMoves 非nullなら、実際に移動した駒の記録を追記する
 */
void SlideAll(Board& board, Direction dir, std::vector<Move>* outMoves = nullptr);

/**
 * @brief 指定マスの駒を1マスだけ滑らせる
 * @detail 滑り先が盤内かつ空マスのときのみ移動する。
 * @param board 対象の盤面 (直接書き換える)
 * @param x,y   滑らせる駒のマス座標
 * @param dir   滑らせる方向
 * @param outMoves 非nullなら、移動が起きた場合に記録を追記する
 * @return 実際に移動したら true、移動しなかったら false
 */
bool SlideOne(Board& board, int x, int y, Direction dir,
               std::vector<Move>* outMoves = nullptr);
}

#endif // BOARD_OPS_H

```

game/board_ops.cpp

```

/*****
 * @file   board_ops.cpp
 * @brief  盤面操作ユーティリティの実装
 * @author Natsume Shidara
 * @date   2026/05/15
 * @update 2026/05/22 - スライド移動の記録に対応
 *****/
#include "board_ops.h"
#include "board.h"

#include <vector>

namespace BoardOps
{
    //=====
    // 全駒スライド
    //=====
    void SlideAll(Board& board, Direction dir, std::vector<Move>* outMoves)
    {
        /*--- 走査方向の決定 ---*/
        // horizontal: 左右スライドなら行ごと、上下なら列ごとに処理する
        // toEnd      : 右・下方向は「末尾寄せ」、左・上方向は「先頭寄せ」
        const bool horizontal = (dir == Direction::Left || dir == Direction::Right);
        const bool toEnd      = (dir == Direction::Right || dir == Direction::Down);
        const int  lineLen    = horizontal ? board.width  : board.height;

```

```

const int lineCount = horizontal ? board.height : board.width;

// ラインインデックス i から盤面座標へ変換するヘルパ
auto cellAt = [&](int line, int i) -> Vec2 {
    return horizontal ? Vec2{ i, line } : Vec2{ line, i };
};

/*--- ライン単位で駒を端へ詰める ---*/
for (int line = 0; line < lineCount; ++line)
{
    // 1) 非空の駒を出現順に収集 (順序を保ったまま詰めるため)
    struct Item { Piece piece; Vec2 from; };
    std::vector<Item> kept;
    kept.reserve(static_cast<size_t>(lineLen));
    for (int i = 0; i < lineLen; ++i)
    {
        const Vec2 c = cellAt(line, i);
        const Piece p = board.Get(c.x, c.y);
        if (p != Piece::Empty) kept.push_back({ p, c });
    }

    // 2) ラインを一旦全消去
    for (int i = 0; i < lineLen; ++i)
    {
        const Vec2 c = cellAt(line, i);
        board.Set(c.x, c.y, Piece::Empty);
    }

    // 3) 端へ詰めて再配置し、移動した駒を記録する
    const int offset = toEnd
        ? (lineLen - static_cast<int>(kept.size())) // 末尾寄せの開始位置
        : 0; // 先頭寄せ
    for (int k = 0; k < static_cast<int>(kept.size()); ++k)
    {
        const Vec2 dst = cellAt(line, offset + k);
        board.Set(dst.x, dst.y, kept[k].piece);

        // 実際に位置が変わった駒だけ移動記録へ追加
        if (outMoves
            && (dst.x != kept[k].from.x || dst.y != kept[k].from.y))
        {
            outMoves->push_back({ kept[k].from.x, kept[k].from.y,
                                   dst.x, dst.y, kept[k].piece });
        }
    }
}

}

//=====
// 単駒スライド
//=====
bool SlideOne(Board& board, int x, int y, Direction dir,
              std::vector<Move>* outMoves)
{
    /*--- 移動可否の判定 ---*/
    if (!board.IsValid(x, y)) return false; // 元座標が盤外
    const Piece p = board.Get(x, y);
    if (p == Piece::Empty) return false; // 動かす駒が無い

    const int nx = x + DirX(dir);
    const int ny = y + DirY(dir);
    if (!board.IsValid(nx, ny)) return false; // 滑り先が盤外
    if (board.Get(nx, ny) != Piece::Empty) return false; // 滑り先が埋まっている
}

```

```

    /*--- 移動の実行 ---*/
    board.Set(x, y, Piece::Empty);
    board.Set(nx, ny, p);
    if (outMoves) outMoves->push_back({ x, y, nx, ny, p });
    return true;
}
}

```

3.3 勝利条件の走査 (win_check)

`WinCheck` 名前空間は、「縦・横・斜めに駒が並んでいるか」という**純粋な走査 ロジック**を提供します。勝利条件フックのデフォルト実装やAIの先読みが、これを利用します。

HasLine — N並びの判定

`HasLine(board, player, required)` は、`player` の駒が `required` 個（既定3）連続している箇所があるかを返します。実装の要点は2つです。

- **走査方向は4つだけ**：右・下・右下・右上。左・上・左下・左上は 逆走査になるだけなので省略できます。
- **始点からのみ数える**：「1マス手前が同色なら、そのラインは別の始点で 数えるのでスキップ」という枝刈りを入れ、同じラインを重複して数えません。

盤面サイズに依存しない汎用実装なので、3×3でも4×4でも、また将来「4並びで勝利」に ルールが変わっても、`required` を変えるだけで対応できます。

CountMaxLine — 最長連続数

`CountMaxLine` は、ある陣営の「最も長く繋がっている駒数」を返します。AIが盤面を評価する材料などに使えます。

game/win_check.h

```

/*****
 * @file   win_check.h
 * @brief  勝利条件判定（連続並びの走査ロジック）
 * @author Natsume Shidara
 * @date   2026/05/15
 *
 * 縦・横・斜めの連続並びを走査する純粋な判定処理。
 * デフォルト勝利条件（DefaultWinCondition）や AI の先読みから利用される。
 *****/
#ifndef WIN_CHECK_H
#define WIN_CHECK_H

#include "piece.h"

class Board; // 前方宣言

//=====
// 勝利条件判定 名前空間
//=====
namespace WinCheck
{
    /**
     * @brief player が指定本数の連続並びを達成しているか判定
    */

```

```

    * @detail 縦・横・斜めの全方向を走査する。
    * @param board 対象の盤面
    * @param player 判定する陣営
    * @param required 勝利に必要な連続数 (デフォルト3)
    * @return 達成していれば true
    */
bool HasLine(const Board& board, Piece player, int required = 3);

/**
 * @brief player の最大連続長を取得
 * @detail 任意方向で最も長く繋がっている駒数を返す。AI評価などで使用。
 * @param board 対象の盤面
 * @param player 判定する陣営
 * @return 最大連続長 (0以上)
 */
int CountMaxLine(const Board& board, Piece player);
}

#endif // WIN_CHECK_H

```

game/win_check.cpp

```

/*****
 * @file win_check.cpp
 * @brief 勝利条件判定の実装
 * @author Natsume Shidara
 * @date 2026/05/15
 *****/
#include "win_check.h"
#include "board.h"

#include <algorithm>

//-----
// 内部ヘルパ
//-----
namespace
{
    // 走査する4方向の単位ベクトル: 右 / 下 / 右下 / 右上。
    // 残り4方向(左・上・左上・左下)は逆走査になるため省略できる。
    constexpr int DIR_COUNT = 4;
    constexpr int DIRS[DIR_COUNT][2] = {
        { 1, 0 }, // 右 (横ライン)
        { 0, 1 }, // 下 (縦ライン)
        { 1, 1 }, // 右下 (斜めライン \)
        { 1, -1 }, // 右上 (斜めライン /)
    };
};

/**
 * @brief (sx,sy)から方向(dx,dy)へ、駒 p が連続する長さを数える
 * @return 連続して並んでいる p の数
 */
int LineLengthFrom(const Board& board, int sx, int sy, int dx, int dy, Piece p)
{
    int count = 0;
    int x = sx, y = sy;
    while (board.IsValid(x, y) && board.Get(x, y) == p)
    {
        ++count;
        x += dx;
        y += dy;
    }
    return count;
}

```

```

    }
}

namespace WinCheck
{
    //=====
    // 連続並びの達成判定
    //=====
    bool HasLine(const Board& board, Piece player, int required)
    {
        if (player == Piece::Empty) return false; // 空マスは勝利対象外
        if (required <= 0) return true;           // 0本要求は常に達成扱い

        // 全マスを始点候補として、各方向のライン長を調べる
        for (int y = 0; y < board.height; ++y)
        {
            for (int x = 0; x < board.width; ++x)
            {
                if (board.Get(x, y) != player) continue;
                for (int d = 0; d < DIR_COUNT; ++d)
                {
                    const int dx = DIRS[d][0];
                    const int dy = DIRS[d][1];
                    // 1つ手前が同色なら、そのラインは別始点で数えるためスキップ
                    if (board.Get(x - dx, y - dy) == player) continue;
                    if (LineLengthFrom(board, x, y, dx, dy, player) >= required)
                    {
                        return true;
                    }
                }
            }
        }
        return false;
    }

    //=====
    // 最大連続長の取得
    //=====
    int CountMaxLine(const Board& board, Piece player)
    {
        if (player == Piece::Empty) return 0;

        int best = 0;
        for (int y = 0; y < board.height; ++y)
        {
            for (int x = 0; x < board.width; ++x)
            {
                if (board.Get(x, y) != player) continue;
                for (int d = 0; d < DIR_COUNT; ++d)
                {
                    const int dx = DIRS[d][0];
                    const int dy = DIRS[d][1];
                    // ラインの始点からのみ計測（重複計測を避ける）
                    if (board.Get(x - dx, y - dy) == player) continue;
                    best = (std::max)(best,
                                      LineLengthFrom(board, x, y, dx, dy, player));
                }
            }
        }
        return best;
    }
}

```

3.4 衝突判定 (collision)

`collision` は、円・矩形（AABB）同士の重なり判定と、点の内外判定を提供する **汎用2Dユーティリティ**です。Zankuuから流用し、2D向けにスリム化したモジュールです。

本作のUIヒット判定（ボタンやカードのクリック）は、実際にはこのモジュールを介さず 各シーン側で直接矩形比較していますが、汎用部品として用意されています。円判定が「距離の二乗で比較して `sqrt` を避ける」など、2D衝突判定の 基本テクニックを学ぶ題材にもなります。

game/collision.h

```

/*****
 * @file    collision.h
 * @brief   2D衝突判定モジュール
 * @author  Natsume Shidara
 * @date    2025/01/02
 * @update  2026/05/15 - O×ローグライト用2D版へスリム化
 *
 * 円・矩形(AABB)同士の重なり判定と、点の内外判定を提供する。
 * マウス座標からマス目を求める処理などで使用する。
 *****/
#ifdef COLLISION_H
#define COLLISION_H

#include <DirectXMath.h>

//=====
// 形状定義
//=====
/**
 * @struct Circle
 * @brief 2D 円形
 */
struct Circle {
    DirectX::XMFLOAT2 center; // 中心座標
    float radius; // 半径
};

/**
 * @struct Box
 * @brief 2D 軸平行矩形 (AABB)
 * @detail 中心座標とハーフサイズ(中心から辺までの距離)で表現する。
 */
struct Box {
    DirectX::XMFLOAT2 center; // 中心座標
    float half_width; // 横方向のハーフサイズ
    float half_height; // 縦方向のハーフサイズ
};

//=====
// 判定関数
//=====
/**
 * @brief 円同士の重なり判定
 * @param a,b 判定する2つの円
 * @return 重なっていれば true
 */
bool Collision_IsOverlapCircle(const Circle& a, const Circle& b);

/**
```

```

* @brief  矩形(AABB)同士の重なり判定
* @param  a,b 判定する2つの矩形
* @return 重なっていれば true
*/
bool Collision_IsOverlapBox(const Box& a, const Box& b);

/**
* @brief  点が矩形の内側にあるか判定
* @detail マウス座標 → マス判定などに使用する。
* @param  px,py 判定する点の座標
* @param  box   判定対象の矩形
* @return 点が矩形内(辺上を含む)なら true
*/
bool Collision_IsPointInBox(float px, float py, const Box& box);

#endif // COLLISION_H

```

game/collision.cpp

```

/*****
* @file    collision.cpp
* @brief    2D衝突判定の実装
* @author   Natsume Shidara
* @date     2025/01/02
* @update   2026/05/15 - O×ローグライト用2D版へスリム化
*****/
#include "collision.h"

//=====
// 判定関数
//=====
bool Collision_IsOverlapCircle(const Circle& a, const Circle& b)
{
    // 中心間距離の二乗 ≤ 半径和の二乗 なら重なっている (sqrtを避けて高速化)
    float dx  = b.center.x - a.center.x;
    float dy  = b.center.y - a.center.y;
    float rSum = a.radius + b.radius;
    return (dx * dx + dy * dy) <= (rSum * rSum);
}

bool Collision_IsOverlapBox(const Box& a, const Box& b)
{
    // 各矩形の上下左右の端を求める
    float aLeft  = a.center.x - a.half_width;
    float aRight  = a.center.x + a.half_width;
    float aTop    = a.center.y - a.half_height;
    float aBottom = a.center.y + a.half_height;

    float bLeft  = b.center.x - b.half_width;
    float bRight  = b.center.x + b.half_width;
    float bTop    = b.center.y - b.half_height;
    float bBottom = b.center.y + b.half_height;

    // 1軸でも完全に離れていれば非重なり。その否定が重なり判定となる。
    return !(aRight < bLeft || aLeft > bRight || aBottom < bTop || aTop > bBottom);
}

bool Collision_IsPointInBox(float px, float py, const Box& box)
{
    // 点が矩形の X 範囲・Y 範囲の両方に収まっているか
    return (px >= box.center.x - box.half_width)
        && (px <= box.center.x + box.half_width)
        && (py >= box.center.y - box.half_height)

```



```
    && (py <= box.center.y + box.half_height);  
}
```

第4章 フック設計 — 本プロジェクトの心臓部

本章は本書で最も重要です。「ゲームのルールそのものを差し替え可能にする」フック設計を解説します。この仕組みがあるからこそ、アビリティでルールが書き換わるローグライトが無理なく成立します。逆に、ここを後付けしようとするとな全面的な作り直しになります。

4.1 なぜフックなのか

普通に○×ゲームを作るなら、勝利判定は「3並びがあるか」と直接書き、思考時間は「180秒固定」と直接書くでしょう。しかし本作はローグライトです。アビリティによって次のようなことが起こります。

- 思考時間が +30秒される（熟考）／ −90秒される（呪い系）
- 1ターンに2回置けるようになる（二手打ち）
- 盤面が3×3から4×4へ広がる（盤面拡張）
- 駒を置くと盤上の駒が滑る（氷盤・氷駒）
- 時間切れの敗北が打ち消される（不死／予定）

これらをすべて「if（氷盤を持っている）{ ... }」のような分岐でゲーム本体に書き込むと、アビリティが増えるたびにゲームループが膨れ上がり、組み合わせも破綻します。

フック設計の核心

そこで本作は発想を逆転させます。「勝利条件」「設置イベント」「ターン処理」「思考時間」「敗北処理」「盤面サイズ」という**ルールの構成要素を、それぞれインターフェイス（フック）として抽象化**します。ゲーム本体はルールを直接書かず、「現在登録されているフック」に判断を委ねます。アビリティはフックの実装を差し替える／追加することでルールを書き換えます。

この設計は**後付けが効きません**。プロト版の最初期、まだアビリティが1つも無い段階からフックを通してゲームを動かしておく必要があります。デフォルト実装（4.4節）が「アビリティゼロの標準ルール」を担うため、フックを通して素の○×ゲームとして問題なく動きます。

4.2 フックインターフェイス群 (hooks.h)

`hooks.h` は6つのインターフェイス（純粋仮想クラス）を定義します。これがゲームルールの「差し替え点」の一覧です。

フック	役割	呼ばれるタイミング
<code>IWinCondition</code>	勝利条件の判定	着手後の評価時
<code>IPlacementHandler</code>	駒設置時のイベント	駒を1個置いた直後

ITurnHandler	1ターンの着手数・ターン境界処理	ターン開始/終了時
ITurnTimeProvider	思考時間の決定	ターン開始時
IDepeatHandler	敗北の打ち消し	敗北が発生したとき
IBoardModifier	盤面サイズの決定	試合開始時

各フックの考え方を補足します。

- `IWinCondition::Check` — 盤面と陣営を受け取り、勝っているかを返す。「3並び」も「隅の支配」も、この実装の違いとして表現できます。
- `IPlacementHandler::OnPlace` — 駒が置かれた直後に呼ばれ、`GameState` を書き換えられる。氷駒・氷盤がここで駒を滑らせます。盤面を動かす効果はここに集まります。
- `ITurnHandler` — 1ターンの着手数（`GetPlacementCount`）と、ターン開始/終了の処理を持つ。二手打ちは着手数を2にします。
- `ITurnTimeProvider::GetTurnTimeSeconds` — その陣営の制限時間を返す。熟考・余裕が加算します。
- `IDepeatHandler::OnDefeat` — 敗北発生時に呼ばれ、`true` を返すと敗北を打ち消す。不死などの「敗北回避」系の受け皿です。
- `IBoardModifier::GetBoardSize` — 盤面の縦横マス数を決める。盤面拡張が3×3を4×4にします。

```
game/ability/hooks.h

/*****
 * @file    hooks.h
 * @brief   フックインターフェイス群（ゲームルールの差し替え点）
 * @author  Natsume Shidara
 * @date    2026/05/15
 *
 * 本プロジェクトの根幹設計。ゲームルール自体を「フック」として
 * 抽象化し、アビリティで差し替え可能にする。各アビリティは
 * これらのインターフェイスを実装してルールを書き換える。
 *
 * 一覧：
 *   IWinCondition      勝利条件の判定
 *   IPlacementHandler  駒設置時/設置後イベント
 *   ITurnHandler       ターン処理（1ターンの着手数、ターン開始/終了）
 *   ITurnTimeProvider  思考時間の決定
 *   IDefeatHandler     敗北処理（return true で敗北回避）
 *   IBoardModifier     盤面サイズの決定（盤面拡張等）
 *****/
#ifndef ABILITY_HOOKS_H
#define ABILITY_HOOKS_H

#include "piece.h"

class Board;      // 前方宣言
struct GameState; // 前方宣言

//=====
// 勝利条件フック
//=====
/**
 * @class IWinCondition
 * @brief 勝利条件の判定インターフェイス
 */
```

```

* @detail 着手後に評価され、Check が true を返した陣営が勝利する。
*/
class IWinCondition
{
public:
    virtual ~IWinCondition() = default;
    /**
     * @brief player が勝利条件を満たしているか判定
     * @param board 対象の盤面
     * @param player 判定する陣営
     * @return 勝利していれば true
     */
    virtual bool Check(const Board& board, Piece player) = 0;
};

//=====
// 駒設置イベントフック
//=====
/**
 * @class IPlacementHandler
 * @brief 駒設置時のイベントインターフェイス
 * @detail 設置直後に呼ばれる。盤面を書き換えたり追加駒を置いたりできる
 *         (例: 氷駒で1マス滑らせる、連鎖駒で隣接マスに自動配置)。
 */
class IPlacementHandler
{
public:
    virtual ~IPlacementHandler() = default;
    /**
     * @brief 駒が設置された直後に呼ばれる
     * @param state 試合状態
     * @param pos 設置されたマス座標
     * @param placedBy 設置した陣営
     */
    virtual void OnPlace(GameState& state, Vec2 pos, Piece placedBy) = 0;
};

//=====
// ターン処理フック
//=====
/**
 * @class ITurnHandler
 * @brief ターン処理のインターフェイス
 * @detail 1ターンの着手数や、ターン境界での状態更新を担う。
 */
class ITurnHandler
{
public:
    virtual ~ITurnHandler() = default;
    /** @brief 1ターンに置ける駒数を返す (例: 二手打ちで2) */
    virtual int GetPlacementCount(Piece player) = 0;
    /** @brief ターン開始時に呼ばれる */
    virtual void OnTurnStart(GameState& state, Piece player) = 0;
    /** @brief ターン終了時に呼ばれる */
    virtual void OnTurnEnd (GameState& state, Piece player) = 0;
};

//=====
// 思考時間プロバイダフック
//=====
/**
 * @class ITurnTimeProvider
 * @brief 思考時間を決定するインターフェイス
 * @detail 各陣営のターン開始時に呼ばれる。
 */

```

```

*           例:「熟考」は +30s、「焦燥」は -90s などを反映する。
*/
class ITurnTimeProvider
{
public:
    virtual ~ITurnTimeProvider() = default;
    /**
     * @brief 指定陣営のターン制限時間を返す
     * @param player 対象の陣営
     * @return 制限時間(秒)
     */
    virtual double GetTurnTimeSeconds(Piece player) = 0;
};

//=====
// 敗北処理フック
//=====
/**
 * @class IDefeatHandler
 * @brief 敗北処理のインターフェイス
 * @detail 敗北発生時(時間切れ・勝利条件達成等)に呼ばれる。
 *          true を返すと敗北を打ち消す (例:「不死」「復活の意志」)。
 */
class IDefeatHandler
{
public:
    virtual ~IDefeatHandler() = default;
    /**
     * @brief 敗北発生時に呼ばれる
     * @param state 試合状態
     * @param loser 敗北しようとしている陣営
     * @return true を返すと敗北を回避する
     */
    virtual bool OnDefeat(GameState& state, Piece loser) = 0;
};

//=====
// 盤面サイズフック
//=====
/**
 * @class IBoardModifier
 * @brief 盤面サイズを決定するインターフェイス
 * @detail 試合開始時に呼ばれ、盤面の縦横マス数を決める。
 *          例:「盤面拡張」は 3×3 を 4×4 に広げる。
 */
class IBoardModifier
{
public:
    virtual ~IBoardModifier() = default;
    /**
     * @brief 盤面サイズを問い合わせる
     * @param width [in/out] 盤面の横マス数
     * @param height [in/out] 盤面の縦マス数
     */
    virtual void GetBoardSize(int& width, int& height) = 0;
};

#endif // ABILITY_HOOKS_H

```

4.3 レジストリ (ability_registry)

AbilityRegistry は、「現在有効なフック実装」を集約し、ゲーム本体へ提供する 窓口です。ゲームシーンはフックを直接触らず、すべて AbilityRegistry 経由で 呼び出します。

単一フックと複数フック

フックは保持の仕方で2種類に分かれます。

種別	該当フック	保持方法
単一（差し替え式）	IWinCondition / ITurnHandler / ITurnTimeProvider / BoardModifier	shared_ptr を1つ。Setoo で上書き。
複数（追加式）	IPlacementHandler / IDefeathandler	vector。Addoo で追加し、全部呼ぶ。

「勝利条件」や「思考時間」は**本質的に1つの答え**を出すものなので単一にします。一方「設置イベント」は、氷駒と氷盤が同時に効くなど**複数が共存しうる**ので、登録順に全部呼びます。「敗北処理」も複数の打ち消し手段が共存しうるため追加式です。

呼び出しヘルパ

CheckWin / OnPlace / GetPlacementCount / GetTurnTime / HandleDefeat / GetBoardSize などの ヘルパが、保持しているフックへ処理を委譲します。ゲーム本体から見ると、g_Registry.CheckWin(board, player) のように1行でルールを問い合わせられます。

```
game/ability/ability_registry.h

/*****
 * @file    ability_registry.h
 * @brief   各フック実装を保持し、ゲームロジックへ提供するレジストリ
 * @author  Natsume Shidara
 * @date    2026/05/15
 *
 * Game シーンは AbilityRegistry を介してフックを呼び出す。
 * 生成時にデフォルト実装がセットされ、アビリティ獲得時に
 * Set/Add で上書き・追加される。
 *****/

#ifndef ABILITY_REGISTRY_H
#define ABILITY_REGISTRY_H

#include "hooks.h"
#include <memory>
#include <vector>

//=====
// アビリティレジストリ
//=====
/**
 * @class   AbilityRegistry
 * @brief   フック実装の集約・呼び出し窓口
 * @detail  単一フック(勝利条件・ターン処理・思考時間)は差し替え式、
 *          複数フック(設置イベント・敗北処理)は追加式で保持する。
 */
```

```

*/
class AbilityRegistry
{
public:
    //=====
    // 構築・リセット
    //=====
    /** @brief 構築（デフォルトフックを初期セット）*/
    AbilityRegistry();

    /** @brief 全フックをデフォルト実装へ戻す（試合の初期化/リセット時）*/
    void ResetToDefaults();

    //=====
    // 単一フックの差し替え
    //=====
    // いずれも旧実装を返す。アビリティ側が chained に保存してチェーン呼び出しできる。
    /** @brief 勝利条件を差し替え @return 旧実装 */
    std::shared_ptr<IWinCondition> SetWinCondition (std::shared_ptr<IWinCondition> w);
    /** @brief ターン処理を差し替え @return 旧実装 */
    std::shared_ptr<ITurnHandler> SetTurnHandler (std::shared_ptr<ITurnHandler> t);
    /** @brief 思考時間プロバイダを差し替え @return 旧実装 */
    std::shared_ptr<ITurnTimeProvider> SetTurnTimeProvider(std::shared_ptr<ITurnTimeProvider> p);
    /** @brief 盤面サイズフックを差し替え @return 旧実装 */
    std::shared_ptr<IBoardModifier> SetBoardModifier (std::shared_ptr<IBoardModifier> m);

    //=====
    // 複数登録可能なフック
    //=====
    /** @brief 設置イベントハンドラを追加 */
    void AddPlacementHandler(std::shared_ptr<IPlacementHandler> h);
    /** @brief 敗北ハンドラを追加 */
    void AddDefeatHandler (std::shared_ptr<IDefeatHandler> h);

    //=====
    // フック呼び出しヘルパ
    //=====
    /** @brief 勝利条件を評価 @return player が勝利していれば true */
    bool CheckWin (const Board& board, Piece player);
    /** @brief 全設置ハンドラの OnPlace を順に呼ぶ */
    void OnPlace (GameState& state, Vec2 pos, Piece placedBy);
    /** @brief 1ターンの着手数を取得 */
    int GetPlacementCount(Piece player);
    /** @brief ターン開始フックを呼ぶ */
    void OnTurnStart (GameState& state, Piece player);
    /** @brief ターン終了フックを呼ぶ */
    void OnTurnEnd (GameState& state, Piece player);
    /** @brief 指定陣営のターン制限時間を取得 */
    double GetTurnTime (Piece player);
    /** @brief 敗北ハンドラを順に呼ぶ @return いずれかが敗北回避したら true */
    bool HandleDefeat (GameState& state, Piece loser);
    /**
     * @brief 盤面サイズを取得する（盤面拡張アビリティ等を反映）
     * @param width [in/out] 盤面の横マス数
     * @param height [in/out] 盤面の縦マス数
     */
    void GetBoardSize (int& width, int& height);

    //=====
    // アクセサ
    //=====
    /** @brief 現在の勝利条件を取得（AIが直接参照する用途）*/
    IWinCondition* GetWinCondition() { return m_Win.get(); }

```

```
private:
    //-----
    // 保持するフック実装
    //-----
    std::shared_ptr<IWinCondition>    m_Win;           // 勝利条件(単一)
    std::shared_ptr<ITurnHandler>     m_Turn;          // ターン処理(単一)
    std::shared_ptr<ITurnTimeProvider> m_TimeProvider; // 思考時間(単一)
    std::shared_ptr<IBoardModifier>   m_BoardMod;     // 盤面サイズ(単一)
    std::vector<std::shared_ptr<IPlacementHandler>>> m_OnPlace; // 設置イベント(複数)
    std::vector<std::shared_ptr<IDefeatHandler>>>     m_OnDefeat; // 敗北処理(複数)
};

#endif // ABILITY_REGISTRY_H
```

game/ability/ability_registry.cpp

```
/*
 * @file    ability_registry.cpp
 * @brief   AbilityRegistry の実装
 * @author  Natsume Shidara
 * @date    2026/05/15
 */
#include "ability_registry.h"
#include "default_hooks.h"

//=====
// 構築・リセット
//=====
AbilityRegistry::AbilityRegistry()
{
    ResetToDefaults();
}

void AbilityRegistry::ResetToDefaults()
{
    // 単一フックはデフォルト実装で初期化し、複数フックは空にする
    m_Win      = std::make_shared<DefaultWinCondition>();
    m_Turn      = std::make_shared<DefaultTurnHandler>();
    m_TimeProvider = std::make_shared<DefaultTurnTimeProvider>();
    m_BoardMod   = std::make_shared<DefaultBoardModifier>();
    m_OnPlace.clear();
    m_OnDefeat.clear();
}

//=====
// 単一フックの差し替え
//=====
std::shared_ptr<IWinCondition> AbilityRegistry::SetWinCondition(std::shared_ptr<IWinCondition> w)
{
    auto prev = m_Win;           // 旧実装を控える(チェーン用に返却)
    if (w) m_Win = std::move(w);
    return prev;
}

std::shared_ptr<ITurnHandler> AbilityRegistry::SetTurnHandler(std::shared_ptr<ITurnHandler> t)
{
    auto prev = m_Turn;
    if (t) m_Turn = std::move(t);
    return prev;
}

std::shared_ptr<ITurnTimeProvider> AbilityRegistry::SetTurnTimeProvider(std::shared_ptr<ITurnTimeProvider> p)
{
    auto prev = m_TimeProvider;
    if (p) m_TimeProvider = std::move(p);
    return prev;
}
```



```

    auto prev = m_TimeProvider;
    if (p) m_TimeProvider = std::move(p);
    return prev;
}

std::shared_ptr<IBoardModifier> AbilityRegistry::SetBoardModifier(std::shared_ptr<IBoardModifier> m)
{
    auto prev = m_BoardMod;
    if (m) m_BoardMod = std::move(m);
    return prev;
}

//=====
// 複数登録可能なフック
//=====
void AbilityRegistry::AddPlacementHandler(std::shared_ptr<IPlacementHandler> h)
{
    if (h) m_OnPlace.push_back(std::move(h));
}

void AbilityRegistry::AddDefeatHandler(std::shared_ptr<IDefeatHandler> h)
{
    if (h) m_OnDefeat.push_back(std::move(h));
}

//=====
// フック呼び出しヘルパ
//=====
bool AbilityRegistry::CheckWin(const Board& board, Piece player)
{
    return m_Win ? m_Win->Check(board, player) : false;
}

void AbilityRegistry::OnPlace(GameState& state, Vec2 pos, Piece placedBy)
{
    // 登録順に、全ハンドラの OnPlace を呼ぶ
    for (auto& h : m_OnPlace) h->OnPlace(state, pos, placedBy);
}

int AbilityRegistry::GetPlacementCount(Piece player)
{
    return m_Turn ? m_Turn->GetPlacementCount(player) : 1;
}

void AbilityRegistry::OnTurnStart(GameState& state, Piece player)
{
    if (m_Turn) m_Turn->OnTurnStart(state, player);
}

void AbilityRegistry::OnTurnEnd(GameState& state, Piece player)
{
    if (m_Turn) m_Turn->OnTurnEnd(state, player);
}

double AbilityRegistry::GetTurnTime(Piece player)
{
    return m_TimeProvider ? m_TimeProvider->GetTurnTimeSeconds(player) : 180.0;
}

bool AbilityRegistry::HandleDefeat(GameState& state, Piece loser)
{
    // いずれかのハンドラが true を返したら敗北を回避する
    for (auto& h : m_OnDefeat)
    {

```

```

        if (h->OnDefeat(state, loser)) return true;
    }
    return false;
}

void AbilityRegistry::GetBoardSize(int& width, int& height)
{
    if (m_BoardMod) m_BoardMod->GetBoardSize(width, height);
}

```

4.4 デフォルト実装 (default_hooks)

`default_hooks` は、アビリティを1つも持っていないときの「標準ルール」です。`AbilityRegistry` は構築時とリセット時に、これらをセットします。

デフォルト実装	標準ルール
<code>DefaultWinCondition</code>	縦横斜めの3並びで勝利。
<code>DefaultTurnHandler</code>	1ターン1着手。ターン境界処理なし。
<code>DefaultTurnTimeProvider</code>	制限時間180秒。
<code>DefaultBoardModifier</code>	盤面サイズ3×3。

デフォルト実装があるおかげで、「フックを通す」こと自体は素の○×ゲームの動作を 何も変えません。フック設計の足場を最初に組んでも、ゲームは普通に遊べます。

game/ability/default_hooks.h

```

/*****
 * @file   default_hooks.h
 * @brief  標準フック実装 (3並び勝利 / 1ターン1着手 / 3分タイマー / 3×3盤面)
 * @author Natsume Shidara
 * @date   2026/05/15
 *
 * アビリティ未取得時の既定ルール。AbilityRegistry の初期状態を成す。
 *****/
#ifndef DEFAULT_HOOKS_H
#define DEFAULT_HOOKS_H

#include "hooks.h"

//=====
// 標準勝利条件
//=====
/**
 * @class DefaultWinCondition
 * @brief 標準の勝利条件 (縦/横/斜めの required 並び)
 */
class DefaultWinCondition : public IWinCondition
{
public:
    int required = 3; // 勝利に必要な連続数
    bool Check(const Board& board, Piece player) override;
};

```

```
//=====
// 標準ターン処理
//=====
/**
 * @class DefaultTurnHandler
 * @brief 標準のターン処理 (1ターン1着手、開始/終了処理なし)
 */
class DefaultTurnHandler : public ITurnHandler
{
public:
    int placementCount = 1; // 1ターンに置ける駒数
    int GetPlacementCount(Piece /*player*/) override { return placementCount; }
    void OnTurnStart(GameState& /*state*/, Piece /*player*/) override {}
    void OnTurnEnd (GameState& /*state*/, Piece /*player*/) override {}
};

//=====
// 標準ターン時間
//=====
/**
 * @class DefaultTurnTimeProvider
 * @brief 標準のターン制限時間 (3分)
 */
class DefaultTurnTimeProvider : public ITurnTimeProvider
{
public:
    double seconds = 180.0; // ターン制限時間(秒)
    double GetTurnTimeSeconds(Piece /*player*/) override { return seconds; }
};

//=====
// 標準盤面サイズ
//=====
/**
 * @class DefaultBoardModifier
 * @brief 標準の盤面サイズ (3×3)
 */
class DefaultBoardModifier : public IBoardModifier
{
public:
    int width = 3; // 既定の横マス数
    int height = 3; // 既定の縦マス数
    void GetBoardSize(int& w, int& h) override { w = width; h = height; }
};

#endif // DEFAULT_HOOKS_H
```

game/ability/default_hooks.cpp

```
/*
 * @file default_hooks.cpp
 * @brief 標準フック実装
 * @author Natsume Shidara
 * @date 2026/05/15
 */
#include "default_hooks.h"
#include "board.h"
#include "win_check.h"

//=====
// 標準勝利条件
//=====
bool DefaultWinCondition::Check(const Board& board, Piece player)
```

```
{
  // 縦・横・斜めのいずれかに required 並びがあれば勝利
  return WinCheck::HasLine(board, player, required);
}
```

4.5 チェインの仕組み

単一フックは1つしか保持できません。では、熟考（+30秒）と余裕（+60秒）を両方持ったらどうなるのでしょうか。答えが**チェイン**です。

`AbilityRegistry::SetTurnTimeProvider` は、新しい実装をセットすると同時に **古い実装を戻り値で返します**。アビリティはこの「旧実装」を自分の中に `chained` として保持します。そして自分が呼ばれたら、まず `chained` に問い合わせ、その結果に自分の効果を足して返します。

登録の流れ（熟考 → 余裕 の順に登録した場合）

registry 初期状態: [Default(180秒)]

熟考.RegisterTo:

```
chained ← registry.SetTurnTimeProvider(熟考) // 旧 Default を受け取る
registry: [熟考] ← chained → [Default]
```

余裕.RegisterTo:

```
chained ← registry.SetTurnTimeProvider(余裕) // 旧 熟考 を受け取る
registry: [余裕] → [熟考] → [Default]
```

呼び出し: `registry.GetTurnTime(Player)`

余裕.Get(): 熟考.Get() の結果 + 60

熟考.Get(): Default.Get() の結果 + 30

Default.Get(): 180

→ 210

→ 270

このように、単一フックでも**連結リスト状に積み重なって全効果が合算**されます。複数フック（設置イベント等）は `vector` に並べて全部呼ぶだけなので、チェインは不要です。第5章以降の各アビリティで、この `chained` パターンが 繰り返し登場します。

登録順が意味を持つ

チェインは登録順に積まれます。たとえば設置イベントでは、プレイヤーのアビリティを先に登録し、ボスのギミックを後に登録します。こうすると `OnPlace` は「プレイヤー効果 → ボス効果」の順で発火します（氷駒で置いた駒を滑らせてから、氷盤で全体を滑らせる、という意図した順序）。登録順は第10章の `LoadBoss` で確定します。

第5章 アビリティシステム

第4章のフックを実際に「実装する側」がアビリティです。本章では、全アビリティの共通基底、報酬抽選プール、そしてプレイヤーが獲得する6種のアビリティを1つずつ解説します。各アビリティは「どのフックを、どう実装するか」の具体例になっています。

5.1 アビリティ基底クラス (ability.h)

`Ability` は全アビリティの共通基底です。表示情報（名前・説明・レアリティ）と、2系統の振る舞いを定義します。

パッシブ系とアクティブ系

- **パッシブ系** — フックを実装してルールを書き換える。 `RegisterTo(registry)` で自身をレジストリに登録します。熟考・氷駒・二手打ち・盤面拡張がこれです。
- **アクティブ系（任意発動）** — `activatable = true` とし、ゲーム中にボタンとして表示され、クリックで `Activate` が呼ばれる。 `CanActivate`（発動可否）と `ChargesLeft`（残り回数）も持ちます。集中がこれです。

1つのアビリティがパッシブとアクティブを兼ねることも可能ですが、プロト版の6種はどちらか一方です。アクティブ専用のアビリティは、フック登録が不要なので `RegisterTo` を空実装にします。

`shared_from_this` とフック登録

`Ability` は `std::enable_shared_from_this` を継承しています。アビリティは「`Ability` としての顔」と「フック（例 `ITurnTimeProvider`）としての顔」を多重継承で併せ持つため、登録時には自分自身をフック型の `shared_ptr` としてレジストリへ渡す必要があります。 `shared_from_this()` で自分の `shared_ptr` を取得し、 `dynamic_pointer_cast` で目的のフック型に変換して登録する、という手順を 各アビリティの `RegisterTo` が踏みます。

レアリティ

`enum class Rarity` は コモン／レア／エピック／レジェンダリー／呪いの5段階。報酬カードの枠色や、将来の抽選重み付けに使います。

game/ability/ability.h

```
/*
 * @file    ability.h
 * @brief   アビリティ基底クラス + レアリティ定義
 * @author  Natsume Shidara
 * @date    2026/05/15
 *
 * 全アビリティの共通基底。各アビリティはフック (IWinCondition 等) を
 * 継承し、RegisterTo で自身を AbilityRegistry に登録する。
 */
#ifndef ABILITY_H
```

```

#define ABILITY_H

//-----
// インクルード・前方宣言
//-----
#include <memory>
#include <string>

class AbilityRegistry; // 前方宣言
struct GameState;      // 前方宣言

//=====
// レアリティ
//=====
/**
 * @enum    Rarity
 * @brief   アビリティのレアリティ階層
 * @detail  報酬抽選の重み付けや、カード枠の色分けに使う。
 */
enum class Rarity
{
    Common,      // コモン（基礎調整）
    Rare,        // レア（ビルドの方向性を決める）
    Epic,        // エピック（シナジーの核）
    Legendary,   // レジェンダリー（ビルドの完成形）
    Curse,       // 呪い（ハイリスク・ハイリターン）
};

//=====
// アビリティ基底クラス
//=====
/**
 * @class   Ability
 * @brief   全アビリティの共通基底
 * @detail  各派生クラスは RegisterTo で1つ以上のフックを registry に登録する。
 *          例: IceTileAbility は IPlacementHandler を継承し、
 *          registry.AddPlacementHandler(self) で自身を登録する。
 *          enable_shared_from_this 継承により、RegisterTo 内で
 *          shared_from_this() から各フック型へ cast して登録できる。
 */
class Ability : public std::enable_shared_from_this<Ability>
{
public:
    virtual ~Ability() = default;

    //-----
    // メンバ変数（アビリティ情報）
    //-----
    std::string name;                // アビリティ名
    std::string description;         // 説明文（報酬カード等に表示）
    Rarity rarity = Rarity::Common; // レアリティ

    /// 任意発動(クリック発動)できるアビリティか。
    /// true のものはゲーム中にボタンとして表示され、クリックで Activate される。
    bool activatable = false;

    //=====
    // フック登録
    //=====
    /**
     * @brief   自身を1つ以上のフックとして registry に登録する
     * @detail  派生クラスは IPlacementHandler / ITurnHandler 等を継承し、
     *          shared_from_this() を dynamic_pointer_cast で取り出して登録する。
     *          任意発動のみのアビリティはフック登録不要（空実装でよい）。

```

```

    * @param registry 登録先のレジストリ
    */
virtual void RegisterTo(AbilityRegistry& registry) = 0;

//=====
// 任意発動アビリティ用 (activatable == true のとき使用)
//=====
/**
 * @brief クリック発動時の効果
 * @detail state を直接書き換えてよい。既定実装は何もしない。
 * @param state 試合状態
 */
virtual void Activate(GameState& /*state*/) {}

/**
 * @brief 現在発動可能かを返す
 * @detail チャージ切れ等で false を返すと、ボタンが無効表示になる。
 * @return 発動可能なら true
 */
virtual bool CanActivate(const GameState& /*state*/) const { return true; }

/**
 * @brief 残り使用回数を返す
 * @return 残り回数。-1 は回数制限なし(UIに回数を出さない)。
 */
virtual int ChargesLeft() const { return -1; }
};

#endif // ABILITY_H

```

5.2 抽選プール (ability_pool)

AbilityPool は、報酬画面の3択を生成するための候補リストです。

- CreateAll() — 報酬候補となる全アビリティを、**毎回新規インスタンス**で生成して返す。インスタンスを使い回さないのは、アビリティが chained やチャージなどの状態を持つためです。
- PickRandom(count) — 全候補をシャッフルし、先頭 count 個を返す（重複なし抽選）。報酬画面はこれで3つ取り出します。

プロト版は全候補を等確率で抽選しますが、α版以降ではレアリティ重み付き抽選へ拡張する想定です。その差し替えも、この PickRandom 1か所で済みます。

game/ability/ability_pool.h

```

/*****
 * @file ability_pool.h
 * @brief 報酬抽選用アビリティプール
 * @author Natsume Shidara
 * @date 2026/05/15
 *
 * 報酬画面の3択生成に使う候補リストを提供する。
 * α/β でレアリティ重み付き抽選への拡張を予定。
 *****/

#ifndef ABILITY_POOL_H
#define ABILITY_POOL_H

//-----

```

```
// インクルード
//-----
#include "ability/ability.h"
#include <memory>
#include <vector>

//=====
// アビリティプール 名前空間
//=====
namespace AbilityPool
{
    /** @brief 全候補アビリティを、新規インスタンスで生成して返す */
    std::vector<std::shared_ptr<Ability>> CreateAll();

    /**
     * @brief プールから指定個数をランダム抽選する（重複なし）
     * @param count 抽選する個数
     * @return 抽選されたアビティ一覧
     */
    std::vector<std::shared_ptr<Ability>> PickRandom(int count);
}

#endif // ABILITY_POOL_H
```

game/ability/ability_pool.cpp

```
/*
*****
* @file   ability_pool.cpp
* @brief  アビリティプールの実装
* @author Natsume Shidara
* @date   2026/05/15
*****
*/
#include "ability_pool.h"

#include "ability/abilities/board_expand.h"
#include "ability/abilities/contemplation.h"
#include "ability/abilities/focus.h"
#include "ability/abilities/ice_tile.h"
#include "ability/abilities/margin.h"
#include "ability/abilities/two_hands.h"

#include <algorithm>
#include <random>

namespace AbilityPool
{
    //=====
    // 候補生成・抽選
    //=====
    std::vector<std::shared_ptr<Ability>> CreateAll()
    {
        // 報酬候補となる全アビリティを、毎回新規インスタンスで生成する
        return {
            std::make_shared<ContemplationAbility>(),
            std::make_shared<MarginAbility>(),
            std::make_shared<FocusAbility>(),
            std::make_shared<IceTileAbility>(),
            std::make_shared<BoardExpandAbility>(),
            std::make_shared<TwoHandsAbility>(),
        };
    }

    std::vector<std::shared_ptr<Ability>> PickRandom(int count)
```



```

{
    // 全候補をシャッフルし、先頭 count 個を取り出す
    auto all = CreateAll();
    static std::mt19937 rng(std::random_device{}());
    std::shuffle(all.begin(), all.end(), rng);
    if (static_cast<int>(all.size()) > count) all.resize(count);
    return all;
}
}

```

5.3 各アビリティの実装

熟考 (ContemplationAbility) — 思考時間 +30秒

`ITurnTimeProvider` を実装するパッシブアビリティです。`RegisterTo` で `SetTurnTimeProvider` を呼び、戻り値（旧実装）を `chained` に保持します。`GetTurnTimeSeconds` は、対象が `targetSide` なら「`chained` の結果 + `bonusSeconds`」を返し、それ以外は `chained` の結果をそのまま返します。第4章で図解したチェーンの典型例です。

このクラスは「余裕」の基底も兼ねており、加算量を変えるだけで派生が作れるよう、`bonusSeconds` をメンバ変数にしてあります。

game/ability/abilities/contemplation.h

```

/*****
 * @file    contemplation.h
 * @brief   熟考 - 思考時間 +30秒（恒久）のアビリティ
 * @author  Natsume Shidara
 * @date    2026/05/15
 *
 * ITurnTimeProvider をチェーンし、プレイヤーのターン時間に
 * bonusSeconds を恒久的に加算する。
 *****/
#ifndef ABILITY_CONTEMPLATION_H
#define ABILITY_CONTEMPLATION_H

#include "ability/ability.h"
#include "ability/hooks.h"

#include <memory>

//=====
// 熟考アビリティ
//=====
/**
 * @class   ContemplationAbility
 * @brief   プレイヤーのターン時間を恒久加算するアビリティ
 * @detail  余裕(MarginAbility)など、加算量違いの派生クラスの基底も兼ねる。
 */
class ContemplationAbility : public Ability, public ITurnTimeProvider
{
public:
    //-----
    // メンバ変数
    //-----
    double bonusSeconds = 30.0;           // 加算する秒数
    double baseSeconds  = 180.0;         // チェイン先が無い場合の基準秒数
    Piece targetSide    = Piece::Player; // 効果を適用する陣営

```

```

// 旧 ITurnTimeProvider をチェーンで保持し、加算前の値を委譲取得する
std::shared_ptr<ITurnTimeProvider> chained;

ContemplationAbility();

/** @brief 指定陣営のターン時間を返す (targetSide には bonus を加算) */
double GetTurnTimeSeconds(Piece player) override;
/** @brief 思考時間プロバイダとして registry に登録 */
void RegisterTo(AbilityRegistry& registry) override;
};

#endif // ABILITY_CONTEMPLATION_H

```

game/ability/abilities/contemplation.cpp

```

/*****
 * @file    contemplation.cpp
 * @brief   熟考の実装
 * @author  Natsume Shidara
 * @date    2026/05/15
 *****/
#include "contemplation.h"
#include "ability/ability_registry.h"

//=====
// 構築
//=====
ContemplationAbility::ContemplationAbility()
{
    name          = "熟考";
    description    = "毎ターン思考時間\n+30秒 (恒久)";
    rarity         = Rarity::Common;
}

//=====
// ITurnTimeProvider
//=====
double ContemplationAbility::GetTurnTimeSeconds(Piece player)
{
    // チェイン先があればその値を、無ければ baseSeconds を基準にする
    const double base = chained ? chained->GetTurnTimeSeconds(player) : baseSeconds;
    if (player == targetSide) return base + bonusSeconds;
    return base;
}

//=====
// Ability
//=====
void ContemplationAbility::RegisterTo(AbilityRegistry& registry)
{
    auto self = std::dynamic_pointer_cast<ITurnTimeProvider>(shared_from_this());
    chained = registry.SetTurnTimeProvider(self); // 旧実装をチェーンに保存
}

```

余裕 (MarginAbility) — 思考時間 +60秒

`ContemplationAbility` を継承し、コンストラクタで名前・説明・レアリティ・`bonusSeconds` を上書きするだけの派生クラスです。ロジックは一切書きません。「基底クラスのパラメータ違い」で新アビリティを増やせる、という設計の好例です。

game/ability/abilities/margin.h

```
/******  
 * @file   margin.h  
 * @brief  余裕 - 思考時間 +60秒 (恒久・強コモン)  
 * @author Natsume Shidara  
 * @date   2026/05/15  
 *  
 * 熟考 (ContemplationAbility) の加算量を強化した派生クラス。  
 *****/  
#ifndef ABILITY_MARGIN_H  
#define ABILITY_MARGIN_H  
  
#include "ability/abilities/contemplation.h"  
  
//=====  
// 余裕アビリティ  
//=====  
/**  
 * @class MarginAbility  
 * @brief 思考時間を +60秒する強コモンアビリティ  
 * @detail ContemplationAbility を継承し、名前・説明・加算量のみ変更する。  
 */  
class MarginAbility : public ContemplationAbility  
{  
public:  
    MarginAbility();  
};  
  
#endif // ABILITY_MARGIN_H
```

game/ability/abilities/margin.cpp

```
/******  
 * @file   margin.cpp  
 * @brief  余裕の実装 (熟考の強化版)  
 * @author Natsume Shidara  
 * @date   2026/05/15  
 *****/  
#include "margin.h"  
  
//=====  
// 構築  
//=====  
MarginAbility::MarginAbility()  
{  
    // 基底(熟考)から、名前・説明・加算量を上書きする  
    name      = "余裕";  
    description = "毎ターン思考時間\n+60秒 (恒久)";  
    rarity     = Rarity::Common;  
    bonusSeconds = 60.0;  
}
```

集中 (FocusAbility) — 任意発動：このターン +2分

唯一の**アクティブ系**アビリティです。 `activatable = true` とし、フックは実装しません（`RegisterTo` は空）。代わりに、

- `Activate(state)` — `state.remainingTime` に `bonusSeconds`（120秒）を加算し、`charges` を1減らす。
- `CanActivate(state)` — `charges > 0` なら発動可。

- `ChargesLeft()` — 残りチャージ数を返す（UIの回数表示に使う）。

パッシブが「常に効くルール変更」なのに対し、アクティブは「プレイヤーが任意のタイミングで切る、回数制限つきの効果」です。ゲームシーンのHUDがこのボタンを描画し、クリックで `Activate` を呼びます（第10章）。

game/ability/abilities/focus.h

```

/*****
 * @file   focus.h
 * @brief  集中 - 任意発動アビリティ（このターン思考時間 +2分）
 * @author Natsume Shidara
 * @date   2026/05/22
 *
 * 企画書コモン「集中」。クリックで発動する初の任意発動アビリティ。
 * パッシブなフックは持たず、クリック時の Activate でのみ作用する。
 *****/

#ifndef ABILITY_FOCUS_H
#define ABILITY_FOCUS_H

#include "ability/ability.h"

//=====
// 集中アビリティ
//=====
/**
 * @class   FocusAbility
 * @brief   クリック発動でこのターンの思考時間を加算するアビリティ
 * @detail  任意発動(activatable)型。チャージを消費して発動する。
 */
class FocusAbility : public Ability
{
public:
    //-----
    // メンバ変数
    //-----
    int    charges      = 3;        // 残りチャージ数
    double bonusSeconds = 120.0;    // 1回あたりの加算秒数

    FocusAbility();

    //=====
    // 任意発動アビリティのインターフェイス
    //=====
    /** @brief 発動: チャージを1消費し、残り時間を加算する */
    void Activate(GameState& state) override;
    /** @brief 発動可能か（チャージが残っているか） */
    bool CanActivate(const GameState& state) const override;
    /** @brief 残りチャージ数を返す */
    int ChargesLeft() const override { return charges; }

    //=====
    // Ability
    //=====
    /** @brief パッシブフックは持たないため、登録処理は空 */
    void RegisterTo(AbilityRegistry& /*registry*/) override {}
};

#endif // ABILITY_FOCUS_H

```

game/ability/abilities/focus.cpp

```

/*****
 * @file   focus.cpp
 * @brief  集中の実装
 * @author Natsume Shidara
 * @date   2026/05/22
 *****/
#include "focus.h"
#include "game_state.h"

//=====
// 構築
//=====
FocusAbility::FocusAbility()
{
    name          = "集中";
    description    = "クリックで\nこのターン +2分";
    rarity         = Rarity::Common;
    activatable   = true;    // 任意発動アビリティとして登録
}

//=====
// 任意発動アビリティのインターフェイス
//=====
void FocusAbility::Activate(GameState& state)
{
    if (charges <= 0) return;          // チャージ切れなら何もしない
    state.remainingTime += bonusSeconds; // このターンの残り時間を加算
    --charges;
}

bool FocusAbility::CanActivate(const GameState& /*state*/) const
{
    return charges > 0;
}

```

氷駒 (IceTileAbility) — 置いた駒が1マス滑る

`IPlacementHandler` を実装するパッシブアビリティです。`RegisterTo` では `AddPlacementHandler` を使います（設置イベントは複数登録できる追加式フック）。

`OnPlace` は、置かれた駒が自陣営なら、その駒を滑走方向へ1マス滑らせ、移動を `animPhases` に1フェーズとして記録します。注目すべきは **重ね掛け対応** です。氷駒を複数所持すると `OnPlace` が所持数ぶん 呼ばれます。各回で「設置マスから滑走方向へ走査して駒の現在位置を特定 → そこから 1マス滑らせる」ことで、先に滑った結果を引き継ぎ、合計で所持数ぶんのマスを滑ります。複数インスタンスを特別にマージする仕組みを持たず、各インスタンスが素直に協調動作するのがうまい点です。

game/ability/abilities/ice_tile.h

```

/*****
 * @file   ice_tile.h
 * @brief  氷駒 — ボス1撃破報酬のプレイヤー側アビリティ
 * @author Natsume Shidara
 * @date   2026/05/15
 *
 * 仕様：プレイヤーが置いた駒のみ、設置時に1マス分滑る。
 * 本アビリティを重ね掛け(複数所持)すると、所持数だけ滑走距離が
 * +1マスずつ伸びる。
 * プロト段階では方向選択UIは無く、Direction::Down に固定する。
 *****/

```

```

* ボス1「氷盤の支配者」の弱体化報酬にあたる。
*****/

#ifndef ABILITY_ICE_TILE_H
#define ABILITY_ICE_TILE_H

#include "ability/ability.h"
#include "ability/hooks.h"
#include "direction.h"

//=====
// 氷駒アビリティ
//=====
/**
 * @class IceTileAbility
 * @brief 設置した自分の駒を滑らせるアビリティ（重ね掛けで滑走距離+1）
 */
class IceTileAbility : public Ability, public IPlacementHandler
{
public:
    //-----
    // メンバ変数
    //-----
    Direction slideDir = Direction::Down; // 滑る方向
    Piece ownerSide = Piece::Player; // 効果対象の陣営

    IceTileAbility();

    /** @brief 設置時、自分の駒を1マス滑らせる（重ね掛け分だけ累積する） */
    void OnPlace(GameState& state, Vec2 pos, Piece placedBy) override;
    /** @brief 設置ハンドラとして registry に登録 */
    void RegisterTo(AbilityRegistry& registry) override;
};

#endif // ABILITY_ICE_TILE_H

```

game/ability/abilities/ice_tile.cpp

```

/*****
 * @file ice_tile.cpp
 * @brief 氷駒の実装
 * @author Natsume Shidara
 * @date 2026/05/15
 *****/
#include "ice_tile.h"

#include "ability/ability_registry.h"
#include "board_ops.h"
#include "game_state.h"

//=====
// 構築
//=====
IceTileAbility::IceTileAbility()
{
    name = "氷駒";
    description = "置いた自分の駒が\n1マス滑る";
    rarity = Rarity::Rare;
}

//=====
// IPlacementHandler
//=====
void IceTileAbility::OnPlace(GameState& state, Vec2 pos, Piece placedBy)

```

```

{
    if (placedBy != ownerSide) return;    // 自分の駒以外は対象外

    /*--- 重ね掛け対応：駒の現在位置を特定する ---*/
    // 氷駒を複数所持していると、本ハンドラは所持数だけ OnPlace される。
    // 先行コピーで既に滑った後は設置マス pos が空になっているため、
    // pos から滑走方向へ走査して「設置した駒の現在位置」を求める。
    // これにより各回で1マスずつ滑り、滑走距離が所持数に応じて伸びる。
    int cx = pos.x;
    int cy = pos.y;
    while (state.board.IsValid(cx, cy) && state.board.Get(cx, cy) == Piece::Empty)
    {
        cx += DirX(slideDir);
        cy += DirY(slideDir);
    }
    if (state.board.Get(cx, cy) != placedBy) return;    // 駒が見つからなければ何もしない

    /*--- 現在位置から1マス滑らせ、効果フェーズとして記録する(アニメ用) ---*/
    BoardOps::MoveList moves;
    if (BoardOps::SlideOne(state.board, cx, cy, slideDir, &moves))
    {
        state.animPhases.push_back(std::move(moves));
    }
}

//=====
// Ability
//=====
void IceTileAbility::RegisterTo(AbilityRegistry& registry)
{
    auto self = shared_from_this();
    registry.AddPlacementHandler(std::dynamic_pointer_cast<IPlacementHandler>(self));
}

```

■ 二手打ち (TwoHandsAbility) — 1ターンに2回置ける

`ITurnHandler` を実装するパッシブアビリティです。`ITurnHandler` は単一フックなので、`SetTurnHandler` でチェーンします。`GetPlacementCount` は、対象陣営なら2、それ以外は `chained` へ委譲。`OnTurnStart` / `OnTurnEnd` もチェーン先へ委譲するだけです。

「自分の関心事（着手数）だけ変え、それ以外はチェーン先にそのまま流す」という、チェーン型フックの定石どおりの実装です。

game/ability/abilities/two_hands.h

```

/*****
 * @file    two_hands.h
 * @brief   二手打ち - 1ターンに2個連続で配置
 * @author  Natsume Shidara
 * @date    2026/05/15
 *
 * ITurnHandler をチェーンし、対象陣営の1ターン着手数を2に増やす。
 *****/
#ifndef ABILITY_TWO_HANDS_H
#define ABILITY_TWO_HANDS_H

#include "ability/ability.h"
#include "ability/hooks.h"

#include <memory>

```

```

//=====
// 二手打ちアビリティ
//=====
/**
 * @class TwoHandsAbility
 * @brief 1ターンの着手数を2に増やすアビリティ
 */
class TwoHandsAbility : public Ability, public ITurnHandler
{
public:
    //-----
    // メンバ変数
    //-----
    Piece targetSide      = Piece::Player; // 効果を適用する陣営
    int  doublePlacementCount = 2;         // 適用時の着手数

    // 旧 ITurnHandler をチェーンで保持し、対象外陣営は委譲する
    std::shared_ptr<ITurnHandler> chained;

    TwoHandsAbility();

    //=====
    // ITurnHandler
    //=====
    /** @brief 1ターンの着手数 (targetSide は2、他はチェーン先へ委譲) */
    int  GetPlacementCount(Piece player) override;
    /** @brief ターン開始時 (チェーン先へ委譲) */
    void OnTurnStart(GameState& state, Piece player) override;
    /** @brief ターン終了時 (チェーン先へ委譲) */
    void OnTurnEnd (GameState& state, Piece player) override;

    //=====
    // Ability
    //=====
    /** @brief ターン処理として registry に登録 */
    void RegisterTo(AbilityRegistry& registry) override;
};

#endif // ABILITY_TWO_HANDS_H

```

game/ability/abilities/two_hands.cpp

```

/*****
 * @file two_hands.cpp
 * @brief 二手打ちの実装
 * @author Natsume Shidara
 * @date 2026/05/15
 *****/
#include "two_hands.h"
#include "ability/ability_registry.h"

//=====
// 構築
//=====
TwoHandsAbility::TwoHandsAbility()
{
    name      = "二手打ち";
    description = "1ターンの2回連続で配置できる";
    rarity     = Rarity::Rare;
}

//=====

```



```
// ITurnHandler
//=====
int TwoHandsAbility::GetPlacementCount(Piece player)
{
    // 対象陣営は2着手、それ以外はチェーン先(デフォルト等)へ委譲
    if (player == targetSide) return doublePlacementCount;
    return chained ? chained->GetPlacementCount(player) : 1;
}

void TwoHandsAbility::OnTurnStart(GameState& state, Piece player)
{
    if (chained) chained->OnTurnStart(state, player);
}

void TwoHandsAbility::OnTurnEnd(GameState& state, Piece player)
{
    if (chained) chained->OnTurnEnd(state, player);
}

//=====
// Ability
//=====
void TwoHandsAbility::RegisterTo(AbilityRegistry& registry)
{
    auto self = std::dynamic_pointer_cast<ITurnHandler>(shared_from_this());
    chained = registry.SetTurnHandler(self); // 旧ハンドラ (BossIce等) をチェーン
}

```

盤面拡張 (BoardExpandAbility) — 盤面を3×3から4×4へ

`IBoardModifier` を実装するパッシブアビリティです。`GetBoardSize` は `chained` が決めたサイズを基準に、自分の目標サイズ (4×4) との**大きい方**を採用します (縮小はしない)。試合開始時にゲームシーンがこのフックを呼び、得られたサイズで `Board::Resize` します。

盤面が広がっても勝利条件は3並びのままなので、勝ち筋 (フォーク) が大幅に増える 強力なエピックアビリティになります。盤面サイズという「構造」を変える効果も、他と同じフックの枠組みで表現できることを示す好例です。

game/ability/abilities/board_expand.h

```

/*****
 * @file   board_expand.h
 * @brief  盤面拡張 - 盤面を 3x3 から 4x4 へ広げる
 * @author Natsume Shidara
 * @date   2026/05/15
 * @update 2026/05/22 - 隅支配勝利(旧ライン本数追加)から盤面拡張へ仕様変更
 *
 * IBoardModifier をチェーンし、試合開始時の盤面サイズを拡大する。
 * 勝利条件(3並び)は据え置きのため、勝利機会が大幅に増える。
 *****/

#ifndef ABILITY_BOARD_EXPAND_H
#define ABILITY_BOARD_EXPAND_H

#include "ability/ability.h"
#include "ability/hooks.h"

#include <memory>

//=====
// 盤面拡張アビリティ
//=====

```

```

/**
 * @class BoardExpandAbility
 * @brief 盤面サイズを拡大するアビリティ
 * @detail IBoardModifier をチェーンし、盤面を 4x4 (既定) へ広げる。
 *         盤面構造カテゴリのエピックアビリティ。
 */
class BoardExpandAbility : public Ability, public IBoardModifier
{
public:
    //-----
    // メンバ変数
    //-----
    int targetWidth = 4; // 拡張後の横マス数
    int targetHeight = 4; // 拡張後の縦マス数

    // 既存の盤面サイズフックをチェーンで保持する
    std::shared_ptr<IBoardModifier> chained;

    BoardExpandAbility();

    //=====
    // IBoardModifier
    //=====
    /** @brief 盤面サイズを問い合わせ、拡張後サイズへ広げる */
    void GetBoardSize(int& width, int& height) override;

    //=====
    // Ability
    //=====
    /** @brief 盤面サイズフックとして registry に登録 */
    void RegisterTo(AbilityRegistry& registry) override;
};

#endif // ABILITY_BOARD_EXPAND_H

```

game/ability/abilities/board_expand.cpp

```

/*****
 * @file board_expand.cpp
 * @brief 盤面拡張の実装
 * @author Natsume Shidara
 * @date 2026/05/15
 * @update 2026/05/22 - 隅支配勝利(旧ライン本数追加)から盤面拡張へ仕様変更
 *****/
#include "board_expand.h"
#include "ability/ability_registry.h"

#include <algorithm>

//=====
// 構築
//=====
BoardExpandAbility::BoardExpandAbility()
{
    name = "盤面拡張";
    description = "盤面が3x3から\n4x4に広がる";
    rarity = Rarity::Epic;
}

//=====
// IBoardModifier
//=====
void BoardExpandAbility::GetBoardSize(int& width, int& height)

```

```
{
    // チェイン先（デフォルト等）が決めたサイズを基準として取得する
    if (chained) chained->GetBoardSize(width, height);

    // 基準サイズより大きい場合のみ拡張する（縮小はしない）
    width  = (std::max)(width,  targetWidth);
    height = (std::max)(height, targetHeight);
}

//=====
// Ability
//=====
void BoardExpandAbility::RegisterTo(AbilityRegistry& registry)
{
    auto self = std::dynamic_pointer_cast<IBoardModifier>(shared_from_this());
    chained = registry.SetBoardModifier(self); // 旧サイズフックをチェインに保存
}
```

第6章 ボスシステム

本作の敵は単なるAIではなく、固有の「ギミック」を持つボスです。本章では、ボスの抽象化、プロト版の唯一のボス「氷盤の支配者」、そしてそのギミック「氷盤」を解説します。ギミックがアビリティと同じフックの上に乗っている点に注目してください。

6.1 ボス基底クラス (boss.h)

`Boss` は、ステージごとの敵を表す抽象基底クラスです。表示情報（名前・説明・難易度★）に加えて、2つの抽象メソッドを持ちます。

- `GetBossAbilities()` — ボス戦中に登録するアビリティ群を返す。これがボスのギミック本体（ボス側強化版）です。
- `GetRewardAbility()` — 撃破時にプレイヤーが獲得するアビリティ（弱体化版）を返す。

ボスギミック = アビリティ

本設計の妙は、ボスのギミックが第5章のアビリティと**まったく同じ仕組み**でできていることです。ギミックも `Ability` を継承しフックを実装します。ボス戦が始まると、プレイヤーのアビリティとボスのギミックが、同じ `AbilityRegistry` に一緒に登録されます。ゲーム本体から見れば、両者の区別はありません。「敵の特殊ルール」と「自分の特殊ルール」を別系統で作らずに済む、という大きな利点があります。

さらに、撃破報酬がギミックの弱体化版である点も特徴です。ボス1「氷盤の支配者」のギミックは「全駒が滑る」ですが、撃破報酬は「自分の駒だけ1マス滑る」氷駒です。プレイヤーは倒したボスの力を一部受け継ぐ、というローグライトらしい体験になります。

game/boss/boss.h

```
/*
 * @file boss.h
 * @brief ボス基底クラス
 * @author Natsume Shidara
 * @date 2026/05/15
 *
 * 各ボスは GetBossAbilities() で自身のギミック（ボス側強化版）を、
 * GetRewardAbility() で撃破時にプレイヤーが得る弱体化アビリティを返す。
 */

#ifndef BOSS_H
#define BOSS_H

//-----
// インクルード
//-----
#include "ability/ability.h"

#include <memory>
#include <string>
#include <vector>
```

```
//=====
// ボス基底クラス
//=====
/**
 * @class Boss
 * @brief ステージごとの敵（ボス）を表す抽象基底クラス
 * @detail ボス固有のギミックはアビリティとして表現される。
 *          ボス戦中はボス側強化版を、撃破後はプレイヤーへ弱化版を渡す。
 */
class Boss
{
public:
    //=====
    // コンストラクタ／デストラクタ
    //=====
    /** @brief 仮想デストラクタ（派生クラスを基底ポインタで破棄するため）*/
    virtual ~Boss() = default;

    //-----
    // メンバ変数（ボス情報）
    //-----
    std::string name;           // ボス名
    std::string description;    // ギミックの説明文
    int          difficulty = 1; // 難易度（★の数）

    //=====
    // 派生クラスが実装するインターフェース
    //=====
    /**
     * @brief ボス側として動作するアビリティ一覧を取得
     * @return ボス戦中に登録するアビリティ群（ボス側強化版）
     */
    virtual std::vector<std::shared_ptr<Ability>> GetBossAbilities() = 0;

    /**
     * @brief 撃破時にプレイヤーが獲得するアビリティを取得
     * @return 報酬として渡す弱化版アビリティ
     */
    virtual std::shared_ptr<Ability> GetRewardAbility() = 0;
};

#endif // BOSS_H
```

6.2 氷盤の支配者 (ice_board_boss)

`IceBoardBoss` は `Boss` を継承する、プロト版唯一の具体ボスです。難易度★1。実装はごく短く、コンストラクタで名前・説明・難易度を設定し、2つの抽象メソッドを次のように実装するだけです。

- `GetBossAbilities()` → `BossIceSlideAbility`（氷盤）を返す。
- `GetRewardAbility()` → `IceTileAbility`（氷駒）を返す。

新しいボスを追加するときは、この `IceBoardBoss` と同じ形で `Boss` の派生クラスを書き、固有のギミックアビリティを用意するだけです。プロト版ではゲームシーンが固定でこのボスを生成しますが、α版ではボス番号に応じて生成を切り替える「ボスマネージャ」へ拡張する想定です。

game/boss/ice_board_boss.h

```

/*****
 * @file   ice_board_boss.h
 * @brief  ボス1: 氷盤の支配者
 * @author Natsume Shidara
 * @date   2026/05/15
 *
 * 設置時/設置後フックを使う最初のボス。難易度★1。
 *****/
#ifndef ICE_BOARD_BOSS_H
#define ICE_BOARD_BOSS_H

//-----
// インクルード
//-----
#include "boss/boss.h"

//=====
// 氷盤の支配者 (ボス1)
//=====
/**
 * @class IceBoardBoss
 * @brief 駒を置くとび盤上の全駒が滑るギミックを持つボス
 * @detail Boss を継承する最初のボス。難易度★1。
 *         ボス側ギミックは BossIceSlideAbility、報酬は IceTileAbility。
 */
class IceBoardBoss : public Boss
{
public:
    //=====
    // コンストラクタ
    //=====
    /** @brief 名前・説明・難易度を設定する */
    IceBoardBoss();

    //=====
    // Boss インターフェース実装
    //=====
    /** @brief ボス戦中に登録するアビリティ群を返す */
    std::vector<std::shared_ptr<Ability>> GetBossAbilities() override;
    /** @brief 撃破報酬として渡すアビリティを返す */
    std::shared_ptr<Ability> GetRewardAbility() override;
};

#endif // ICE_BOARD_BOSS_H

```

game/boss/ice_board_boss.cpp

```

/*****
 * @file   ice_board_boss.cpp
 * @brief  ボス1: 氷盤の支配者 実装
 * @author Natsume Shidara
 * @date   2026/05/15
 *****/
#include "ice_board_boss.h"
#include "ability/abilities/boss_ice_slide.h"
#include "ability/abilities/ice_tile.h"

//=====
// 構築
//=====
IceBoardBoss::IceBoardBoss()
{

```

```

    name          = "氷盤の支配者";
    description    = "駒を置くたび全ての駒が滑る";
    difficulty     = 1;
}

//=====
// Boss インターフェース実装
//=====
std::vector<std::shared_ptr<Ability>> IceBoardBoss::GetBossAbilities()
{
    // ボス側ギミック：設置のたび盤上の全駒を滑らせる
    return { std::make_shared<BossIceSlideAbility>() };
}

std::shared_ptr<Ability> IceBoardBoss::GetRewardAbility()
{
    // 撃破報酬：滑りを1マスに弱化した「氷駒」アビリティ
    return std::make_shared<IceTileAbility>();
}

```

6.3 ボスギミック「氷盤」(boss_ice_slide)

`BossIceSlideAbility` が、氷盤の支配者のギミック本体です。このアビリティは2つのフックを同時に実装します。

実装するフック	役割
<code>IPlacementHandler</code>	駒が置かれるたびに、盤上の全駒を <code>currentDir</code> 方向へスライドさせる（ <code>BoardOps::SlideAll</code> ）。移動は <code>animPhases</code> に記録。
<code>ITurnHandler</code>	着手数自体は変えず（チェイン先へ委譲）、 <code>OnTurnStart</code> でスライド方向 <code>currentDir</code> を毎ターン抽選し直す。

設置イベント（`IPlacementHandler`）は追加式フックなので `AddPlacementHandler` で登録します。ターン処理（`ITurnHandler`）は単一フックなので `SetTurnHandler` でチェインし、旧実装を `chainedTurn` に保持します。`GetPlacementCount` や `OnTurnEnd` は `chainedTurn` へそのまま委譲し、氷盤は「方向の抽選」と「全駒スライド」という自分の関心事だけを担当します。

スライド方向を毎ターン変えることで、プレイヤーは「次にどちらへ滑るか」を読みながら着手する必要性が生まれ、単純な○×に駆け引きが加わります。次にどちらへ滑るかは、ゲームシーンが盤面の縁にシェブロン矢印で可視化します（第10章）。

登録順との関係

ゲームシーンの `LoadBoss` は、プレイヤーのアビリティを先に、ボスのギミックを後に登録します。これにより設置イベントは「プレイヤー効果（氷駒など）→ボス効果（氷盤）」の順で発火します。プレイヤーが置いた駒がまず氷駒で滑り、その後氷盤で盤面全体が滑る、という意図した順序が、登録順だけで実現されます。

game/ability/abilities/boss_ice_slide.h

```

/*****
 * @file   boss_ice_slide.h
 * @brief  氷盤 - ボス1「氷盤の支配者」のボス側アビリティ
 * @author Natsume Shidara
 * @date   2026/05/15

```

```

*
* 仕様：駒が置かれると全ての駒が指定方向の端まで滑る。
* 滑る方向はターン開始時にランダムで決まる。
*****/
#ifndef ABILITY_BOSS_ICE_SLIDE_H
#define ABILITY_BOSS_ICE_SLIDE_H

#include "ability/ability.h"
#include "ability/hooks.h"
#include "direction.h"

#include <memory>

//=====
// 氷盤アビリティ（ボス側）
//=====
/**
 * @class BossIceSlideAbility
 * @brief 全駒スライドを起こすボス側アビリティ
 * @detail IPlacementHandler で設置時に全駒を滑らせ、
 *         ITurnHandler でターン開始時に滑る方向を抽選する。
 */
class BossIceSlideAbility
    : public Ability
    , public IPlacementHandler
    , public ITurnHandler
{
public:
    //-----
    // メンバ変数
    //-----
    Direction currentDir = Direction::Right; // 現在の滑走方向

    // 旧 ITurnHandler（デフォルト等）をチェーンで保持し、委譲する
    std::shared_ptr<ITurnHandler> chainedTurn;

    //=====
    // 構築
    //=====
    BossIceSlideAbility();

    //=====
    // IPlacementHandler
    //=====
    /** @brief 設置時、全駒を currentDir 方向へスライドさせる */
    void OnPlace(GameState& state, Vec2 pos, Piece placedBy) override;

    //=====
    // ITurnHandler
    //=====
    /** @brief 1ターンの着手数（チェーン先へ委譲） */
    int GetPlacementCount(Piece player) override;
    /** @brief ターン開始時、滑走方向をランダム抽選する */
    void OnTurnStart(GameState& state, Piece player) override;
    /** @brief ターン終了時（チェーン先へ委譲） */
    void OnTurnEnd (GameState& state, Piece player) override;

    //=====
    // Ability
    //=====
    /** @brief 設置ハンドラ・ターン処理として registry に登録 */
    void RegisterTo(AbilityRegistry& registry) override;
};

```



```
#endif // ABILITY_BOSS_ICE_SLIDE_H
```

game/ability/abilities/boss_ice_slide.cpp

```

/*****
 * @file   boss_ice_slide.cpp
 * @brief  氷盤（ボス側アビリティ）の実装
 * @author Natsume Shidara
 * @date   2026/05/15
 *****/
#include "boss_ice_slide.h"

#include "ability/ability_registry.h"
#include "board_ops.h"
#include "game_state.h"

#include <random>

//-----
// 内部ヘルパ
//-----
namespace
{
    /** @brief 4方向からランダムに1つ返す */
    Direction RandomDir()
    {
        static std::mt19937 rng(std::random_device{}());
        std::uniform_int_distribution<int> dist(0, 3);
        return static_cast<Direction>(dist(rng));
    }
}

//=====
// 構築
//=====
BossIceSlideAbility::BossIceSlideAbility()
{
    name          = "氷盤";
    description    = "全ての駒が指定方向へ\n端まで滑る";
    rarity         = Rarity::Rare;
    currentDir     = RandomDir(); // 初期方向もランダムに決める
}

//=====
// IPlacementHandler
//=====
void BossIceSlideAbility::OnPlace(GameState& state, Vec2 /*pos*/, Piece /*placedBy*/)
{
    // 全駒スライドを実行し、その移動を1つの効果フェーズとして記録する(アニメ用)
    BoardOps::MoveList moves;
    BoardOps::SlideAll(state.board, currentDir, &moves);
    if (!moves.empty())
    {
        state.animPhases.push_back(std::move(moves));
    }
}

//=====
// ITurnHandler
//=====
int BossIceSlideAbility::GetPlacementCount(Piece player)
{

```

```

    // 着手数自体は変えない。チェイン先(デフォルト等)の値を返す。
    return chainedTurn ? chainedTurn->GetPlacementCount(player) : 1;
}

void BossIceSlideAbility::OnTurnStart(GameState& state, Piece player)
{
    if (chainedTurn) chainedTurn->OnTurnStart(state, player);
    currentDir = RandomDir();    // ターンごとに滑走方向を抽選する
}

void BossIceSlideAbility::OnTurnEnd(GameState& state, Piece player)
{
    if (chainedTurn) chainedTurn->OnTurnEnd(state, player);
}

//=====
// Ability
//=====
void BossIceSlideAbility::RegisterTo(AbilityRegistry& registry)
{
    auto self = shared_from_this();
    // 設置ハンドラとして追加登録
    registry.AddPlacementHandler(std::dynamic_pointer_cast<IPlacementHandler>(self));
    // ターン処理を差し替え、旧実装はチェインで保持する
    auto turnSelf = std::dynamic_pointer_cast<ITurnHandler>(self);
    chainedTurn = registry.SetTurnHandler(turnSelf);
}

```

第7章 AI — 思考ルーチン

ボス（×側）の着手を決めるAIです。プロト版は「勝てるなら勝つ・負けそうなら防ぐ・それ以外はランダム」という、シンプルだが破綻しない思考ルーチンを採用します。

7.1 着手決定のアルゴリズム

公開関数は `AI::ChooseMove_Random` ただ1つです。盤面・自陣営・勝利条件フックを受け取り、置くべきマス（`Vec2`）を返します。空気が無ければ `{-1, -1}`。判断は次の優先順位です。

1. **自分が勝てるマス** — 1手で勝利が確定するマスがあれば、そこへ置く。
2. **相手が勝つマス** — 相手が次に勝てるマスがあれば、そこを塞いでブロックする。
3. **それ以外** — 空きマスからランダムに1つ選ぶ。

1と2は同じ `FindWinningMove` 関数で実現します。これは「各空きマスに試しに駒を置いてみて、勝利条件を満たすか」を総当たりで調べる関数です。引数の陣営を自分にすれば「勝てるマス」、相手にすれば「相手が勝つマス = 塞ぐべきマス」が得られます。試し置きは盤面のコピーに対して行い、調べ終わったら元に戻します。

7.2 勝利条件フックとの連携

重要なのは、AIが勝敗を判定するとき、**固定の3並び判定ではなく、レジストリの `IWinCondition` フックを使う**点です。`ChooseMove_Random` には `g_Registry.GetWinCondition()` が渡され、`FindWinningMove` 内の `CheckWinWith` がそれを呼びます。

これにより、勝利条件を書き換えるアビリティが入っても、AIの「勝てる／防ぐ」判断が自動的に新ルールへ追従します。AIをルール変更のたびに直す必要がありません。これもフック設計の恩恵です。

プロト版AIの限界

このAIは、駒を置いた後の**盤面スライド（氷盤・氷駒）までは先読みしません**。`FindWinningMove` は駒を試し置きするだけで、設置イベントフック（`OnPlace`）は実行しないためです。したがって氷盤ボス戦では、AIはスライド後の結果を計算に入れずに着手します。プロト版としては許容範囲ですが、α版で「スライドを織り込んだ先読み」へ強化する余地がある、と理解しておいてください。

game/ai.h

```
/*
*****
* @file    ai.h
* @brief   AI思考(着手選択)
* @author  Natsume Shidara
* @date    2026/05/15
* @update  2026/05/15 - IWinCondition フック対応 (Day 3)
*/
```

```

*
* プロト版のシンプルAI。「勝てるマスがあれば置く / 相手の勝てる
* マスをブロック / それ以外はランダム」で着手を決める。
* α版で MinimaxAI への差し替えを予定。
*****/
#ifdef AI_H
#define AI_H

#include "piece.h"

class Board;          // 前方宣言
class IWinCondition;  // 前方宣言

//=====
// AI思考 名前空間
//=====
namespace AI
{
    /**
     * @brief AIの着手先を決定する
     * @detail 勝利 → ブロック → ランダム の優先順で1手を選ぶ。
     * @param board 現在の盤面
     * @param self AIが操作する陣営
     * @param win 勝利条件 (AbilityRegistry の IWinCondition*)。
     *           null の場合は3並びを既定として使用する。
     * @return 着手するマス座標。空きマスが無い場合は {-1, -1}。
     */
    Vec2 ChooseMove_Random(const Board& board, Piece self, IWinCondition* win);
}

#endif // AI_H

```

game/ai.cpp

```

/*****
* @file   ai.cpp
* @brief  AI思考の実装
* @author Natsume Shidara
* @date   2026/05/15
*
* 着手選択ロジック(優先順):
* 1. 自分が勝てるマス → 即座に置く
* 2. 相手が次に勝つマス → ブロックする
* 3. それ以外          → 空きマスからランダム
*
* 勝利判定は AbilityRegistry の IWinCondition を経由するため、
* 「ライン本数追加」「拡張勝利」等のアビリティが入っても自動追従する。
*****/
#include "ai.h"
#include "board.h"
#include "win_check.h"
#include "ability/hooks.h"

#include <vector>
#include <random>

//-----
// 内部ヘルパ
//-----
namespace
{
    /** @brief 乱数エンジン (初回呼び出し時に生成し共有する) */
    std::mt19937& Rng()

```

```

{
    static std::mt19937 rng(std::random_device{}());
    return rng;
}

/**
 * @brief 勝利条件で player が勝っているか判定
 * @detail win が指定されていればそれを使い、null なら3並びを既定使用。
 */
bool CheckWinWith(const Board& board, Piece player, IWinCondition* win)
{
    if (win) return win->Check(board, player);
    return WinCheck::HasLine(board, player, 3);
}

/**
 * @brief self が1手で勝てるマスを探す
 * @return 勝てるマス座標。無ければ {-1, -1}。
 */
Vec2 FindWinningMove(const Board& board, Piece self, IWinCondition* win)
{
    Board test = board; // 試し置き用のコピー
    for (int y = 0; y < board.height; ++y)
    {
        for (int x = 0; x < board.width; ++x)
        {
            if (!test.IsEmpty(x, y)) continue;
            test.Set(x, y, self);
            const bool won = CheckWinWith(test, self, win);
            test.Set(x, y, Piece::Empty); // 試し置きを元に戻す
            if (won) return { x, y };
        }
    }
    return { -1, -1 };
}

/**
 * @brief 空きマスから1つをランダムに選ぶ
 * @return ランダムな空きマス座標。空きが無ければ {-1, -1}。
 */
Vec2 PickRandomEmpty(const Board& board)
{
    std::vector<Vec2> empties;
    empties.reserve(static_cast<size_t>(board.width) * board.height);
    for (int y = 0; y < board.height; ++y)
    {
        for (int x = 0; x < board.width; ++x)
        {
            if (board.IsEmpty(x, y)) empties.push_back({ x, y });
        }
    }
    if (empties.empty()) return { -1, -1 };
    std::uniform_int_distribution<size_t> dist(0, empties.size() - 1);
    return empties[dist(Rng())];
}
}

namespace AI
{
    //=====
    // 着手決定
    //=====
    Vec2 ChooseMove_Random(const Board& board, Piece self, IWinCondition* win)
    {

```

```
// 1) 自分が勝てるマスがあれば最優先で取る
Vec2 v = FindWinningMove(board, self, win);
if (v.x >= 0) return v;

// 2) 相手が勝てるマスがあればブロックする
v = FindWinningMove(board, Opponent(self), win);
if (v.x >= 0) return v;

// 3) それ以外は空きマスからランダムに選ぶ
return PickRandomEmpty(board);
}
}
```

第8章 描画とテキスト

本作のビジュアルは「黒背景 + 幾何学図形 + グロー」のミニマル抽象路線です。専用の イラスト素材をほとんど使わず、図形とテキストだけで画面を構成します。本章では、その図形描画・テキスト描画・日本語フォント・フェードの4モジュールを解説します。

8.1 図形描画 (render_primitives)

`Prim` 名前空間は、矩形・直線・円・×を描くヘルパです。専用テクスチャを持たず、**1pxの白画像**（`white.png`）を引き伸ばし・回転させて あらゆる図形を描くのが基本トリックです。

- `DrawRect` — 白画像を指定サイズへ拡大し、頂点カラーで着色。
- `DrawLine` — 始点→終点のベクトルから長さや角度を求め、白画像を「長さ × 太さ」の細い矩形にして、その角度へ回転させて描く。2D描画における「線 = 細い回転矩形」という定番手法です。
- `DrawCircle` — 円周を `segments` 本の弦（直線）で 近似する。内部で `DrawLine` を繰り返し呼びます。
- `DrawCross` — 交差する2本の `DrawLine` で×を描く。

スプライト描画は頂点カラー乗算に対応しているため、白い図形に任意の色（と アルファ）を掛けられます。プレイヤーの○、ボスの×、グリッド線、UIの矩形、グロー効果まで、すべてこの4関数の組み合わせで描けます。利用前に `Prim::Initialize()` で `white.png` をロードしておく必要があります。

game/render_primitives.h

```

/*****
 * @file    render_primitives.h
 * @brief   2D図形描画ヘルパ (矩形 / 直線 / 円 / ×)
 * @author  Natsume Shidara
 * @date    2026/05/15
 *
 * white.png を Sprite_Draw に通すことで、専用テクスチャ無しで
 * ミニマル抽象ビジュアル(線・図形)を描画する。
 * 利用前に Prim::Initialize() を必ず呼ぶこと。
 *****/
#ifndef RENDER_PRIMITIVES_H
#define RENDER_PRIMITIVES_H

#include <DirectXMath.h>

//=====
// 2D図形描画 名前空間
//=====
namespace Prim
{
    /** @brief 初期化 - 内部で white.png をロードする */
    void Initialize();

    /**
     * @brief 塗りつぶし矩形を描画
     */
}
```

```

    * @param x,y 左上座標
    * @param w,h 幅・高さ
    * @param color 塗り色
    */
void DrawRect(float x, float y, float w, float h,
              const DirectX::XMFLOAT4& color);

/**
 * @brief 直線を描画
 * @param x1,y1 始点
 * @param x2,y2 終点
 * @param thickness 線の太さ
 * @param color 線の色
 */
void DrawLine(float x1, float y1, float x2, float y2,
              float thickness, const DirectX::XMFLOAT4& color);

/**
 * @brief 円のアウトラインを描画
 * @detail segments 本の弦(直線)で円周を近似する。
 * @param cx,cy 中心座標
 * @param radius 半径
 * @param thickness 線の太さ
 * @param color 線の色
 * @param segments 円周の分割数 (既定32)
 */
void DrawCircle(float cx, float cy, float radius,
                float thickness, const DirectX::XMFLOAT4& color,
                int segments = 32);

/**
 * @brief x 記号を描画
 * @detail (cx,cy) を中心とした2本の交差線を引く。
 * @param cx,cy 中心座標
 * @param halfSize 中心から各端点までの距離
 * @param thickness 線の太さ
 * @param color 線の色
 */
void DrawCross(float cx, float cy, float halfSize,
               float thickness, const DirectX::XMFLOAT4& color);
}

#endif // RENDER_PRIMITIVES_H

```

game/render_primitives.cpp

```

/*****
 * @file render_primitives.cpp
 * @brief 2D図形描画ヘルパの実装
 * @author Natsume Shidara
 * @date 2026/05/15
 *****/
#include "render_primitives.h"

#include "sprite.h"
#include "texture.h"

#include <cmath>

using namespace DirectX;

//-----
// 内部状態・定数

```



```

//-----
namespace
{
    int g_TexWhite = -1; // 白1pxテクスチャのID
    constexpr float PI_F = 3.14159265358979323846f; // 円周率
}

namespace Prim
{
    //=====
    // 初期化
    //=====
    void Initialize()
    {
        // 多重初期化を避けるため、未ロード時のみ読み込む
        if (g_TexWhite < 0)
        {
            g_TexWhite = Texture_Load(L"assets/white.png");
        }
    }

    //=====
    // 図形描画
    //=====
    void DrawRect(float x, float y, float w, float h, const XMFLOAT4& color)
    {
        if (g_TexWhite < 0) return;
        Sprite_Draw(g_TexWhite, x, y, w, h, 0.0f, color);
    }

    void DrawLine(float x1, float y1, float x2, float y2,
                  float thickness, const XMFLOAT4& color)
    {
        if (g_TexWhite < 0) return;

        // 始点→終点のベクトルから、線の長さと角度を求める
        const float dx = x2 - x1;
        const float dy = y2 - y1;
        const float length = std::sqrt(dx * dx + dy * dy);
        if (length < 1e-4f) return; // 長さ0の線は描かない
        const float angle = std::atan2(dy, dx);
        const float midX = (x1 + x2) * 0.5f;
        const float midY = (y1 + y2) * 0.5f;

        // Sprite_Draw は矩形の中心を回転軸として扱うため、
        // 描画原点 (x,y) には「中心 - 半サイズ」を渡す。
        Sprite_Draw(g_TexWhite,
                    midX - length * 0.5f,
                    midY - thickness * 0.5f,
                    length, thickness, angle, color);
    }

    void DrawCircle(float cx, float cy, float radius,
                   float thickness, const XMFLOAT4& color, int segments)
    {
        if (segments < 3) segments = 3; // 最低限の分割数を保証

        // 円周上の点を順に結ぶ弦で、円のアウトラインを近似する
        float prevX = cx + radius;
        float prevY = cy;
        for (int i = 1; i <= segments; ++i)
        {
            const float a = static_cast<float>(i) / segments * 2.0f * PI_F;
            const float nx = cx + std::cos(a) * radius;

```

```

        const float ny = cy + std::sin(a) * radius;
        DrawLine(prevX, prevY, nx, ny, thickness, color);
        prevX = nx;
        prevY = ny;
    }
}

void DrawCross(float cx, float cy, float halfSize,
               float thickness, const XMFLOAT4& color)
{
    // 左上→右下、左下→右上 の2本で × を構成する
    DrawLine(cx - halfSize, cy - halfSize, cx + halfSize, cy + halfSize,
              thickness, color);
    DrawLine(cx - halfSize, cy + halfSize, cx + halfSize, cy - halfSize,
              thickness, color);
}
}

```

8.2 テキスト描画 (text_draw)

Text 名前空間は、UTF-8文字列を画面に描画します。

- **Draw(x,y,text,charSize,color)** — 文字列を等幅で描画。 `\n` で改行。日本語を含む任意のUnicodeに対応。
- **MeasureWidth(text,charSize)** — 表示幅を計測（中央寄せ等に使う）。

UTF-8デコードと等幅描画

内部の `DecodeUtf8` は、UTF-8のバイト列から1文字（コードポイント）を取り出し、ポインタを次の文字へ進めます。先頭バイトのビットパターンから「2バイト文字／3バイト文字／4バイト文字」を判別し、継続バイトを連結します。

描画は**等幅（各文字を `charSize` 四方のセル）**で行います。1文字ぶん描いてはカーソルを `charSize` だけ右へ送る、を繰り返すだけです。日本語の文字はもともとほぼ正方形なので、等幅でも見た目が破綻しません。実際のグリフ画像は次節の `FontAtlas` から取得します。

game/text_draw.h

```

/*****
 * @file   text_draw.h
 * @brief  ゲーム内テキスト描画 (DirectWrite動的グリフアトラス)
 * @author Natsume Shidara
 * @date   2026/05/15
 * @update 2026/05/22 - 日本語対応 (DirectWriteグリフアトラス)
 *
 * UTF-8文字列を受け取り、FontAtlas が実行時にラスタライズした
 * グリフをスプライト描画する。日本語を含む任意のUnicodeに対応。
 *****/

#ifndef TEXT_DRAW_H
#define TEXT_DRAW_H

#include <DirectXMath.h>

//=====
// テキスト描画 名前空間
//=====
namespace Text
{

```

```

/** @brief フォントアトラスの初期化（多重呼び出しは無視）*/
void Initialize();

/** @brief 終了処理（Direct3D_Finalize より前に呼ぶこと）*/
void Finalize();

/**
 * @brief 文字列を、スクリーン座標（x, y）を左上として描画
 * @param x      左上X座標
 * @param y      左上Y座標
 * @param text    UTF-8文字列（\n で改行）。日本語可。
 * @param charSize 1文字の表示サイズ(正方形・等幅)
 * @param color   頂点カラー
 */
void Draw(float x, float y, const char* text,
          float charSize, const DirectX::XMFLLOAT4& color = {1, 1, 1, 1});

/**
 * @brief 文字列の表示幅を計測
 * @param text    対象の文字列
 * @param charSize 1文字の表示サイズ
 * @return 最長行の幅（charSize × その行のコードポイント数）
 */
float MeasureWidth(const char* text, float charSize);
}

#endif // TEXT_DRAW_H

```

game/text_draw.cpp

```

/*****
 * @file    text_draw.cpp
 * @brief   ゲーム内テキスト描画の実装
 * @author  Natsume Shidara
 * @date    2026/05/15
 * @update  2026/05/22 - 日本語対応（UTF-8デコード + FontAtlas）
 *
 * 等幅描画：各コードポイントを charSize 四方のセルとして送る。
 * 日本語は元々ほぼ正方形のため、等幅でも破綻しない。
 *****/
#include "text_draw.h"

#include "font_atlas.h"
#include "sprite.h"

#include <cstdint>

//-----
// 内部ヘルパ
//-----
namespace
{
    /**
     * @brief UTF-8文字列から1文字をデコードし、ポインタを次の文字へ進める
     * @param p [in,out] 文字列ポインタ。デコード後、次の文字位置へ進む。
     * @return デコードしたUnicodeコードポイント。不正バイトは U+FFFD。
     */
    uint32_t DecodeUtf8(const char*& p)
    {
        const unsigned char c0 = static_cast<unsigned char>(*p);

        // 1バイト (ASCII): そのまま返す
        if (c0 < 0x80)

```

```

{
    ++p;
    return c0;
}

// 先頭バイトのビットパターンから、続く継続バイト数を判定する
uint32_t cp = 0;
int extra = 0;
if ((c0 & 0xE0) == 0xC0) { cp = c0 & 0x1F; extra = 1; } // 2バイト文字
else if ((c0 & 0xF0) == 0xE0) { cp = c0 & 0x0F; extra = 2; } // 3バイト文字
else if ((c0 & 0xF8) == 0xF0) { cp = c0 & 0x07; extra = 3; } // 4バイト文字
else { ++p; return 0xFFFD; } // 不正な先頭バイト

// 継続バイトを連結してコードポイントを組み立てる
++p;
for (int i = 0; i < extra; ++i)
{
    const unsigned char cc = static_cast<unsigned char>(*p);
    if ((cc & 0xC0) != 0x80) return 0xFFFD; // 継続バイトでない(不正)
    cp = (cp << 6) | (cc & 0x3F);
    ++p;
}
return cp;
}
}

```

```

namespace Text
{
    //=====
    // 初期化・終了
    //=====
    void Initialize()
    {
        FontAtlas::Initialize();
    }

    void Finalize()
    {
        FontAtlas::Finalize();
    }

    //=====
    // 描画・計測
    //=====
    void Draw(float x, float y, const char* text,
              float charSize, const DirectX::XMFLLOAT4& color)
    {
        if (!text) return;
        ID3D11ShaderResourceView* atlas = FontAtlas::GetAtlasSRV();
        if (!atlas) return;

        float cursorX = x;
        float cursorY = y;
        for (const char* p = text; *p; )
        {
            // 改行: カーソルを次の行頭へ送る
            if (*p == '\n')
            {
                cursorX = x;
                cursorY += charSize;
                ++p;
                continue;
            }

```

```

        const uint32_t cp = DecodeUtf8(p);
        if (cp == ' ') // 空白はグリフ無し、送りのみ進める
        {
            cursorX += charSize;
            continue;
        }

        // グリフのアトラスUVを取得し、1文字分のスプライトを描画する
        FontAtlas::GlyphUV uv;
        if (FontAtlas::GetGlyph(cp, uv))
        {
            Sprite_Draw(atlas, cursorX, cursorY, charSize, charSize,
                        uv.u0, uv.v0, uv.u1, uv.v1, 0.0f, color);
        }
        cursorX += charSize;
    }
}

float MeasureWidth(const char* text, float charSize)
{
    if (!text) return 0.0f;

    // 各行のコードポイント数を数え、最長行を採用する
    int maxLen = 0;
    int cur = 0;
    for (const char* p = text; *p; )
    {
        if (*p == '\n')
        {
            if (cur > maxLen) maxLen = cur;
            cur = 0;
            ++p;
            continue;
        }
        DecodeUtf8(p);
        ++cur;
    }
    if (cur > maxLen) maxLen = cur;
    return static_cast<float>(maxLen) * charSize;
}

```

8.3 動的グリフアトラス (font_atlas)

FontAtlas は、本作のテキスト描画を支える最も技術的なモジュールです。当初はASCII画像フォントを使っていましたが、UIテキストを日本語化するにあたり、**必要になった文字をその場でラスターライズしてキャッシュする動的アトラス方式**へ切り替えました。

なぜ動的生成か

日本語は文字数が膨大です。全文字をあらかじめ1枚の画像に焼き込むのは非現実的です。そこで、**ゲーム中に実際に使われた文字だけを**、初出時にDirectWriteでラスターライズし、D3D11のアトラステクスチャ（2048×2048、64px×64pxセル）へ書き込んでキャッシュします。2回目以降は同じグリフを再利用します。

1グリフのラスタライズ手順

1. Direct2D で、64px四方のオフスクリーン（WICビットマップ）に1文字を DrawText
2. WICビットマップをロックし、画素（プリマルチプライ済みBGRA）を読む
3. 「白RGB + カバレッジ α 」のストレートRGBA へ変換
4. D3D11アトラステクスチャの空きセルへ UpdateSubresource で転送
5. そのセルの正規化UVを算出してキャッシュ

手順3が肝です。Direct2Dは白い文字をプリマルチプライアルファで描くため、画素のアルファ値がそのまま「文字の被覆率」になります。これを「RGBは白固定・ α は被覆率」というストレートアルファのRGBAへ変換しておくと、2Dシェーダの「テクスチャ色 × 頂点色」だけで、グリフに任意の色を掛けて描画できます。専用のテキストシェーダが要りません。

DirectWrite（フォントとテキストレイアウト）、Direct2D（描画）、WIC（ビットマップ）の3つのCOMコンポーネントを組み合わせる、Windowsのテキスト描画の典型構成です。COMリソースは `Finalize` で `Direct3D_Finalize` より前に解放します。

game/font_atlas.h

```

/*****
 * @file   font_atlas.h
 * @brief  DirectWrite動的グリファトラス
 * @author Natsume Shidara
 * @date   2026/05/22
 *
 * DirectWrite + Direct2D + WIC を使い、必要になった文字を実行時に
 * ラスタライズして D3D11 のアトラステクスチャにキャッシュする。
 * 日本語を含む任意のUnicode文字に対応する。
 *
 * グリフは「白RGB + カバレッジ $\alpha$ 」のストレートアルファで格納するため、
 * 2Dシェーダ (tex * vertexColor) でそのまま色付け描画できる。
 *****/
#ifndef FONT_ATLAS_H
#define FONT_ATLAS_H

//-----
// インクルード
//-----
#include <d3d11.h>
#include <cstdint>

//=====
// グリファトラス 名前空間
//=====
namespace FontAtlas
{
    /**
     * @struct GlyphUV
     * @brief アトラステクスチャ内のグリフ位置（正規化UV）
     */
    struct GlyphUV
    {
        float u0, v0, u1, v1; // 左上(u0,v0)・右下(u1,v1) の正規化UV
    };

    /**
     * @brief グリファトラスの初期化
     * @detail Direct3D初期化後・COM初期化後に呼ぶこと。多重呼び出しは無視する。
     * @return 初期化に成功したら true
     */
}
```

```

bool Initialize();

/** @brief 終了処理 (Direct3D_Finalize より前に呼ぶこと) */
void Finalize();

/**
 * @brief コードポイントのグリフUVを取得 (未キャッシュなら生成)
 * @param codepoint 取得したい文字のUnicodeコードポイント
 * @param outUV [out] グリフのアトラスUV
 * @return 生成・取得に成功したら true
 */
bool GetGlyph(uint32_t codepoint, GlyphUV& outUV);

/** @brief アトラステクスチャのSRVを取得 (Sprite描画で使用) */
ID3D11ShaderResourceView* GetAtlasSRV();
}

#endif // FONT_ATLAS_H

```

game/font_atlas.cpp

```

/*****
 * @file font_atlas.cpp
 * @brief DirectWrite動的グリフアトラスの実装
 * @author Natsume Shidara
 * @date 2026/05/22
 *
 * 処理の流れ (1グリフあたり):
 * 1. Direct2D の WICビットマップRT に1文字を DrawText
 * 2. WICビットマップをロックし、プリマルチプライ済みBGRAを読む
 * 3. 「白RGB + カバレッジα」のストレートRGBAへ変換
 * 4. D3D11アトラステクスチャの空きセルへ UpdateSubresource
 * 5. セルの正規化UVをキャッシュ
 *****/
#include "font_atlas.h"
#include "direct3d.h"

#include <d2d1.h>
#include <d2d1helper.h>
#include <dwrite.h>
#include <wincodec.h>
#include <wrl/client.h>

#include <unordered_map>
#include <vector>

// 必要ライブラリのリンク指定
#pragma comment(lib, "d2d1.lib")
#pragma comment(lib, "dwrite.lib")

using Microsoft::WRL::ComPtr;

//-----
// 内部定数・状態
//-----
namespace
{
    //--- アトラス構成 ---
    constexpr UINT ATLAS_SIZE = 2048; // アトラス1辺(px)
    constexpr UINT CELL_SIZE = 64; // 1セル1辺(px)
    constexpr float GLYPH_FONT_PX = 52.0f; // セル内に描くフォントサイズ
    constexpr UINT CELLS_PER_ROW = ATLAS_SIZE / CELL_SIZE; // 1行あたりのセル数 = 32
    constexpr UINT MAX_CELLS = CELLS_PER_ROW * CELLS_PER_ROW; // 総セル数 = 1024
}

```

```

// 使用する日本語フォント (Windows標準)
const wchar_t* FONT_FAMILY = L"Yu Gothic UI";

//--- COMリソース ---
ComPtr<IDWriteFactory> g_DWrite; // DirectWriteファクトリ
ComPtr<ID2D1Factory> g_D2D; // Direct2Dファクトリ
ComPtr<IWICImagingFactory> g_WIC; // WICファクトリ
ComPtr<IDWriteTextFormat> g_TextFormat; // テキスト描画フォーマット

ComPtr<IWICBitmap> g_CellBitmap; // 1文字描画用オフスクリーン
ComPtr<ID2D1RenderTarget> g_CellRT; // 上記ビットマップへのD2D RT
ComPtr<ID2D1SolidColorBrush> g_Brush; // 文字描画用ブラシ(白)

ComPtr<ID3D11Texture2D> g_AtlasTex; // アトラステクスチャ本体
ComPtr<ID3D11ShaderResourceView> g_AtlasSRV; // アトラスのSRV

//--- キャッシュ状態 ---
UINT g_NextCell = 0; // 次に使う空きセル番号
std::unordered_map<uint32_t, FontAtlas::GlyphUV> g_Cache; // コードポイント→UV
bool g_Ready = false; // 初期化済みフラグ

/**
 * @brief コードポイントをUTF-16へ変換
 * @detail BMP内は1要素、追加面はサロゲートペア(2要素)になる。
 * @param cp 変換元のコードポイント
 * @param out [out] UTF-16の格納先(最大2要素)
 * @return 書き込んだUTF-16要素数 (1 or 2)
 */
int CodepointToUtf16(uint32_t cp, wchar_t out[2])
{
    if (cp <= 0xFFFF)
    {
        out[0] = static_cast<wchar_t>(cp);
        return 1;
    }
    cp -= 0x10000;
    out[0] = static_cast<wchar_t>(0xD800 + (cp >> 10));
    out[1] = static_cast<wchar_t>(0xDC00 + (cp & 0x3FF));
    return 2;
}

/**
 * @brief 1グリフをラスタライズし、アトラスの空きセルへ書き込む
 * @param codepoint ラスタライズする文字
 * @param outUV [out] 書き込んだセルの正規化UV
 * @return 成功したら true (アトラス満杯・描画失敗時は false)
 */
bool RasterizeGlyph(uint32_t codepoint, FontAtlas::GlyphUV& outUV)
{
    if (g_NextCell >= MAX_CELLS) return false; // アトラスに空きが無い

    wchar_t utf16[2];
    const int wlen = CodepointToUtf16(codepoint, utf16);

    /*--- 1. Direct2D で1文字をセルビットマップへ描画 ---*/
    g_CellRT->BeginDraw();
    g_CellRT->Clear(D2D1::ColorF(0.0f, 0.0f, 0.0f, 0.0f));
    const D2D1_RECT_F layout =
        D2D1::RectF(0.0f, 0.0f,
                    static_cast<float>(CELL_SIZE),
                    static_cast<float>(CELL_SIZE));
    g_CellRT->DrawText(utf16, static_cast<UINT32>(wlen),
                      g_TextFormat.Get(), layout, g_Brush.Get());

```



```

if (FAILED(g_CellRT->EndDraw())) return false;

/*--- 2. WICビットマップをロックして画素を読む ---*/
WICRect rc = { 0, 0, static_cast<INT>(CELL_SIZE), static_cast<INT>(CELL_SIZE) };
ComPtr<IWICBitmapLock> lock;
if (FAILED(g_CellBitmap->Lock(&rc, WICBitmapLockRead, &lock))) return false;

UINT stride = 0, bufSize = 0;
BYTE* src = nullptr;
lock->GetStride(&stride);
lock->GetDataPointer(&bufSize, &src);
if (!src) return false;

/*--- 3. プリマルチプライBGRA → 「白RGB + α」ストレートRGBA へ変換 ---*/
// D2Dは白文字をプリマルチプライで描くため、α がそのまま被覆率になる。
std::vector<uint32_t> cell(static_cast<size_t>(CELL_SIZE) * CELL_SIZE);
for (UINT y = 0; y < CELL_SIZE; ++y)
{
    const BYTE* row = src + static_cast<size_t>(y) * stride;
    for (UINT x = 0; x < CELL_SIZE; ++x)
    {
        const BYTE a = row[x * 4 + 3];          // アルファ = 被覆率
        // R8G8B8A8 (little-endian) = 0xAABBGGRR。RGBは白固定 + α。
        cell[static_cast<size_t>(y) * CELL_SIZE + x] =
            (static_cast<uint32_t>(a) << 24) | 0x00FFFFFFu;
    }
}
lock.Reset();

/*--- 4. アトラスの空きセルへ転送 ---*/
const UINT cellX = (g_NextCell % CELLS_PER_ROW) * CELL_SIZE;
const UINT cellY = (g_NextCell / CELLS_PER_ROW) * CELL_SIZE;
D3D11_BOX box = { cellX, cellY, 0, cellX + CELL_SIZE, cellY + CELL_SIZE, 1 };
Direct3D_GetContext()->UpdateSubresource(
    g_AtlasTex.Get(), 0, &box,
    cell.data(), CELL_SIZE * sizeof(uint32_t), 0);

/*--- 5. セルの正規化UVを算出 ---*/
const float inv = 1.0f / static_cast<float>(ATLAS_SIZE);
outUV.u0 = cellX * inv;
outUV.v0 = cellY * inv;
outUV.u1 = (cellX + CELL_SIZE) * inv;
outUV.v1 = (cellY + CELL_SIZE) * inv;

++g_NextCell;
return true;
}
}

namespace FontAtlas
{
//=====
// 初期化・終了
//=====
bool Initialize()
{
    if (g_Ready) return true;    // 多重初期化を無視

    ID3D11Device* device = Direct3D_GetDevice();
    if (!device) return false;

    /*--- 各種ファクトリの生成 ---*/
    // DirectWrite
    if (FAILED(DWriteCreateFactory(DWRITE_FACTORY_TYPE_SHARED,

```

```

        __uuidof(IDWriteFactory),
        reinterpret_cast<IUnknown**>(g_DWrite.GetAddressOf()))))
    return false;

// Direct2D
if (FAILED(D2D1CreateFactory(D2D1_FACTORY_TYPE_SINGLE_THREADED,
    g_D2D.GetAddressOf()))
    return false;

// WIC
if (FAILED(CoCreateInstance(CLSID_WICImagingFactory, nullptr,
    CLSCTX_INPROC_SERVER, IID_PPV_ARGS(g_WIC.GetAddressOf()))))
    return false;

/*--- テキストフォーマット (セル中央寄せ) ---*/
if (FAILED(g_DWrite->CreateTextFormat(
    FONT_FAMILY, nullptr,
    DWRITE_FONT_WEIGHT_NORMAL, DWRITE_FONT_STYLE_NORMAL,
    DWRITE_FONT_STRETCH_NORMAL, GLYPH_FONT_PX, L"ja-jp",
    g_TextFormat.GetAddressOf()))
    return false;
g_TextFormat->SetTextAlignment(DWRITE_TEXT_ALIGNMENT_CENTER);
g_TextFormat->SetParagraphAlignment(DWRITE_PARAGRAPH_ALIGNMENT_CENTER);

/*--- 1文字描画用 WICビットマップ + D2Dレンダーターゲット ---*/
if (FAILED(g_WIC->CreateBitmap(CELL_SIZE, CELL_SIZE,
    GUID_WICPixelFormat32bppPBGRA, WICBitmapCacheOnLoad,
    g_CellBitmap.GetAddressOf()))
    return false;

const D2D1_RENDER_TARGET_PROPERTIES rtProps = D2D1::RenderTargetProperties(
    D2D1_RENDER_TARGET_TYPE_DEFAULT,
    D2D1::PixelFormat(DXGI_FORMAT_B8G8R8A8_UNORM,
        D2D1_ALPHA_MODE_PREMULTIPLIED));
if (FAILED(g_D2D->CreateWicBitmapRenderTarget(
    g_CellBitmap.Get(), rtProps, g_CellRT.GetAddressOf()))
    return false;
g_CellRT->SetTextAntialiasMode(D2D1_TEXT_ANTIALIAS_MODE_GRAYSCALE);
if (FAILED(g_CellRT->CreateSolidColorBrush(
    D2D1::ColorF(D2D1::ColorF::White), g_Brush.GetAddressOf()))
    return false;

/*--- D3D11アトラステクスチャ ---*/
D3D11_TEXTURE2D_DESC td = {};
td.Width          = ATLAS_SIZE;
td.Height          = ATLAS_SIZE;
td.MipLevels       = 1;
td.ArraySize       = 1;
td.Format           = DXGI_FORMAT_R8G8B8A8_UNORM;
td.SampleDesc.Count = 1;
td.Usage            = D3D11_USAGE_DEFAULT;
td.BindFlags        = D3D11_BIND_SHADER_RESOURCE;
if (FAILED(device->CreateTexture2D(&td, nullptr, g_AtlasTex.GetAddressOf()))
    return false;
if (FAILED(device->CreateShaderResourceView(
    g_AtlasTex.Get(), nullptr, g_AtlasSRV.GetAddressOf()))
    return false;

g_NextCell = 0;
g_Cache.clear();
g_Ready = true;
return true;
}

```

```

void Finalize()
{
    // キャッシュとCOMリソースを解放 (Direct3D_Finalize より前に呼ぶこと)
    g_Cache.clear();
    g_AtlasSRV.Reset();
    g_AtlasTex.Reset();
    g_Brush.Reset();
    g_CellRT.Reset();
    g_CellBitmap.Reset();
    g_TextFormat.Reset();
    g_WIC.Reset();
    g_D2D.Reset();
    g_DWrite.Reset();
    g_NextCell = 0;
    g_Ready = false;
}

//=====
// グリフ取得
//=====
bool GetGlyph(uint32_t codepoint, GlyphUV& outUV)
{
    if (!g_Ready) return false;

    // キャッシュ済みならそれを返す
    auto it = g_Cache.find(codepoint);
    if (it != g_Cache.end())
    {
        outUV = it->second;
        return true;
    }

    // 未キャッシュ: ラスタライズして登録する
    GlyphUV uv;
    if (!RasterizeGlyph(codepoint, uv)) return false;
    g_Cache.emplace(codepoint, uv);
    outUV = uv;
    return true;
}

ID3D11ShaderResourceView* GetAtlasSRV()
{
    return g_AtlasSRV.Get();
}
}

```

8.4 フェード (fade)

`fade` は、画面全体を単色の矩形で覆い、そのアルファ値を時間で変化させることでフェードイン／フェードアウトを行います。`Fade_Start(duration, isFadeOut, color)` で開始し、`Fade_Update` が進捗0→1を計算してアルファへ反映、`Fade_Draw` が画面最前面に矩形を重ねます。

状態は `FADE_STATE` 列挙型 (なし／イン中／アウト中／イン完了／アウト完了) で管理します。`Fade_GetState` で状態を問い合わせれば、「フェードアウト完了を待ってシーン遷移」といった演出のつながりが書けます。プロト版では控えめな利用ですが、シーン遷移の質を上げる部品として用意されています。

```

/*****
 * @file   fade.h
 * @brief  画面のフェードイン・アウト制御
 * @author Natsume Shidara
 * @date   2025/07/10
 *
 * 画面全体を単色矩形で覆い、アルファ値を時間で変化させることで
 * フェード演出を行う。シーン遷移の繋ぎなどに利用する。
 *****/

#ifndef FADE_H
#define FADE_H

#include "color.h"

//=====
// フェード制御関数
//=====
/** @brief フェードモジュールの初期化 */
void Fade_Initialize(void);

/** @brief フェードモジュールの終了処理 */
void Fade_Finalize(void);

/**
 * @brief フェードの更新
 * @param elapsed_time 前フレームからの経過時間(秒)
 */
void Fade_Update(double elapsed_time);

/** @brief フェードの描画 (画面最前面に重ねる) */
void Fade_Draw(void);

/**
 * @brief フェードを開始する
 * @param duration フェードにかかる秒数
 * @param isFadeOut true:フェードアウト / false:フェードイン
 * @param color フェードに使う色 (既定は黒)
 */
void Fade_Start(double duration, bool isFadeOut, Color::COLOR color = Color::BLACK);

//=====
// フェード状態
//=====
/**
 * @enum FADE_STATE
 * @brief フェードの進行状態
 */
enum class FADE_STATE
{
    NONE,           // フェードなし
    FADE_IN,        // フェードイン中(画面が明けていく)
    FADE_OUT,       // フェードアウト中(画面が暗転していく)
    FINISHED_IN,    // フェードイン完了
    FINISHED_OUT    // フェードアウト完了
};

/** @brief 現在のフェード状態を取得 */
FADE_STATE Fade_GetState(void);

#endif

```

```

/*****
 * @file   fade.cpp
 * @brief  画面のフェードイン・アウト制御の実装
 * @author Natsume Shidara
 * @date   2025/07/10
 *****/

#include "fade.h"

#include <algorithm>

#include "debug_ostream.h"
#include "direct3d.h"
#include "sprite.h"
#include "texture.h"
#ifdef USE_ASSET_DLL
#include "asset_dll.h"
#include "resource.h"
#endif

//-----
// 内部状態
//-----
static double g_FadeTime      = 0.0;          // フェードにかける総秒数
static double g_FadeStartTime = 0.0;          // フェード開始時の累積時間
static double g_AccumulatedTime = 0.0;        // 起動からの累積時間
static auto   g_State         = FADE_STATE::NONE; // 現在のフェード状態
static int    g_FadeTexId     = -1;           // フェード用1pxテクスチャID

static float   g_Alpha = 0.0f;                // 現在の不透明度(0:透明~1:不透明)
static Color::COLOR g_color = Color::BLACK;    // フェード色

//=====
// 初期化・終了
//=====
void Fade_Initialize()
{
    g_FadeTime      = 1.0;
    g_FadeStartTime = 0.0;
    g_AccumulatedTime = 0.0;
    g_color          = Color::BLACK;
    g_Alpha           = 0.0f;
    g_State           = FADE_STATE::NONE;

    /*--- フェード用テクスチャ(白1px)の読み込み ---*/
#ifdef USE_ASSET_DLL
    {
        // x64構成: AssetDLLに埋め込まれたリソースから読み込む
        const void* pData = nullptr;
        size_t dataSize = 0;
        if (AssetDLL_GetData(IDR_TEX_WHITE, &pData, &dataSize))
            g_FadeTexId = Texture_LoadFromMemory(pData, dataSize, L"fade_white");
    }
#else
    // 非DLL構成: ファイルから直接読み込む
    g_FadeTexId = Texture_Load(L"assets/white.png");
#endif
}

void Fade_Finalize()
{
}

//=====
// 更新・描画

```

```
//=====
void Fade_Update(double elapsed_time)
{
    // 完了状態・無状態のときは更新不要
    if (g_State == FADE_STATE::FINISHED_IN || g_State == FADE_STATE::FINISHED_OUT)
        return;
    if (g_State == FADE_STATE::NONE)
        return;

    g_AccumulatedTime += elapsed_time;

    // 進捗を 0.0~1.0 に正規化
    double progress = min((g_AccumulatedTime - g_FadeStartTime) / g_FadeTime, 1.0);

    // 進捗が1に達したら完了状態へ遷移
    if (progress >= 1)
    {
        g_State = g_State == FADE_STATE::FADE_IN ? FADE_STATE::FINISHED_IN : FADE_STATE::FINISHED_OUT;
    }

    // フェードイン時は不透明→透明、アウト時は透明→不透明
    g_Alpha = static_cast<float>(g_State == FADE_STATE::FADE_IN ? 1.0 - progress : progress);
}

void Fade_Draw()
{
    // 何も覆わない状態では描画不要
    if (g_State == FADE_STATE::NONE)
        return;
    if (g_State == FADE_STATE::FINISHED_IN)
        return;

    // 現在のアルファをフェード色へ適用
    Color::COLOR color = g_color;
    color.w = g_Alpha;

    // 画面全体を覆う矩形を描画
    float width = static_cast<float>(Direct3D_GetBackBufferWidth());
    float height = static_cast<float>(Direct3D_GetBackBufferHeight());
    Sprite_Draw(g_FadeTexId, 0.0f, 0.0f, width, height, 0.0f, color);
}

//=====
// フェード開始・状態取得
//=====
void Fade_Start(double duration, bool isFadeOut, Color::COLOR color)
{
    g_FadeStartTime = g_AccumulatedTime;
    g_FadeTime = duration;
    g_State = isFadeOut ? FADE_STATE::FADE_OUT : FADE_STATE::FADE_IN;
    g_color = color;
    g_Alpha = isFadeOut ? 1.0f : 0.0f;
}

FADE_STATE Fade_GetState()
{
    return g_State;
}

```

第9章 ラン進行とシーン

1回の挑戦（ラン）を通じてシーンをまたいで共有される状態と、タイトル・報酬の2シーンを解説します。中核のゲームシーンは規模が大きいため、独立した第10章で扱います。

9.1 ラン状態 (run_state)

`RunState` 名前空間は、「タイトルから始まり複数のボス戦を続ける1回の挑戦」の進行状態を保持します。シーンは切り替わるたびに `Finalize` / `Initialize` され、ローカル変数は失われます。そこで、シーンをまたいで残すべき情報をここに集約します。

状態	意味
<code>PlayerAbilities()</code>	プレイヤーの所持アビリティ一覧。ランを通じて累積。
<code>RewardChoices()</code>	報酬画面で提示中の3択アビリティ。
<code>CurrentBossIndex()</code>	現在挑戦中のボス番号（0 = ボス1）。

関数は3つです。

- `ResetRun()` — 所持アビリティ・報酬候補・ボス進行をすべて初期化。タイトルから新しいランを始めるときに呼びます。
- `GenerateRewardChoices()` — `AbilityPool::PickRandom(3)` で報酬の3択を抽選します。
- `IncrementBoss()` — ボス番号を1つ進めます（ボス撃破時）。

プロト版ではシンプルな名前空間APIですが、α版ではこれらを `RunManager` クラスへ統合する想定です。「シーン間で共有する状態は1か所にまとめる」という方針を示すモジュールです。

game/run_state.h

```
/*
 * @file run_state.h
 * @brief 1ラン(5戦)の進行状態をシーン間で共有
 * @author Natsume Shidara
 * @date 2026/05/15
 *
 * Game/Reward シーンはローカルにラン状態を持たず、本モジュールを参照する。
 * α版で RunManager クラスへ統合する前提の、シンプルな名前空間API。
 */

#ifndef RUN_STATE_H
#define RUN_STATE_H

//-----
// インクルード
//-----
#include "ability/ability.h"

#include <memory>
```

```

#include <vector>

//=====
// ラン進行状態 名前空間
//=====
namespace RunState
{
    /** @brief プレイヤーの所持アビリティ一覧(ランを通じて累積)を取得 */
    std::vector<std::shared_ptr<Ability>>& PlayerAbilities();

    /** @brief 現在の報酬画面で提示している3択アビリティを取得 */
    std::vector<std::shared_ptr<Ability>>& RewardChoices();

    /** @brief 現在挑戦中のボスindex (0=ボス1, ..., 4=ラスボス) を取得 */
    int CurrentBossIndex();
    /** @brief ボスindexを1つ進める (ボス撃破時に呼ぶ) */
    void IncrementBoss();

    /** @brief ラン状態を完全リセット (タイトルから新ラン開始時に呼ぶ) */
    void ResetRun();

    /** @brief 報酬の3択をプールからランダム抽選して生成する */
    void GenerateRewardChoices();
}

#endif // RUN_STATE_H

```

game/run_state.cpp

```

/*****
 * @file    run_state.cpp
 * @brief   ラン進行状態の実装
 * @author  Natsume Shidara
 * @date    2026/05/15
 *****/
#include "run_state.h"
#include "ability/ability_pool.h"

//-----
// 内部状態
//-----
namespace
{
    std::vector<std::shared_ptr<Ability>> g_PlayerAbilities; // 所持アビリティ(累積)
    std::vector<std::shared_ptr<Ability>> g_RewardChoices;    // 報酬画面の3択
    int g_BossIndex = 0;                                     // 現在挑戦中のボスindex
}

namespace RunState
{
    //=====
    // アクセサ
    //=====
    std::vector<std::shared_ptr<Ability>>& PlayerAbilities() { return g_PlayerAbilities; }
    std::vector<std::shared_ptr<Ability>>& RewardChoices()   { return g_RewardChoices;   }

    int CurrentBossIndex() { return g_BossIndex; }
    void IncrementBoss()   { ++g_BossIndex; }

    //=====
    // ラン制御
    //=====
    void ResetRun()

```



```

{
    // 所持アビリティ・報酬候補・ボス進行をすべて初期化
    g_PlayerAbilities.clear();
    g_RewardChoices.clear();
    g_BossIndex = 0;
}

void GenerateRewardChoices()
{
    // アビリティプールから3つをランダム抽選して報酬候補とする
    g_RewardChoices = AbilityPool::PickRandom(3);
}
}

```

9.2 タイトルシーン (title)

タイトルシーンは、ロゴと点滅プロンプトを表示し、入力で本編へ遷移するだけの 軽量なシーンです。Title_Update の要点は次の通りです。

- 点滅プロンプト用に経過時間を積算する。
- スペース／Enter／左クリックのいずれかで、RunState::ResetRun() を呼んでラン状態を初期化し、Scene_Change(Scene::GAME) で本編へ遷移を予約する。

Title_Draw は、背景・タイトルロゴ・サブタイトルを描き、開始プロンプトを一定周期で点滅させます。図形とテキストだけで構成され、第8章の描画モジュールの 最初の利用例になります。

game/scene/title.h

```

/*****
 * @file   title.h
 * @brief  タイトルシーン
 * @author Natsume Shidara
 * @date   2026/05/15
 *
 * タイトルロゴを表示し、入力でゲーム本編へ遷移する。
 *****/

#ifndef TITLE_H
#define TITLE_H

//=====
// タイトルシーン API
//=====
/** @brief タイトルシーンの初期化 */
void Title_Initialize();

/** @brief タイトルシーンの終了処理 */
void Title_Finalize();

/**
 * @brief タイトルシーンの更新
 * @param elapsed_time 前フレームからの経過秒数
 */
void Title_Update(double elapsed_time);

/** @brief タイトルシーンの描画 */
void Title_Draw();

```

```
#endif // TITLE_H
```

game/scene/title.cpp

```
/*
 * @file title.cpp
 * @brief タイトルシーン (Day 1-2)
 * @author Natsume Shidara
 * @date 2026/05/15
 */

#include "title.h"
#include "scene.h"
#include "run_state.h"
#include "render_primitives.h"
#include "text_draw.h"

#include "direct3d.h"
#include "input_manager.h"
#include "keyboard.h"

//-----
// 定数・内部状態 (無名名前空間)
//-----
namespace
{
    // 文字サイズ
    constexpr float TEXT_TITLE_SIZE = 72.0f; // メインロゴ
    constexpr float TEXT_SUBTITLE = 22.0f; // サブタイトル
    constexpr float TEXT_HINT_SIZE = 22.0f; // 開始プロンプト

    // 配色 (RGBA)
    const DirectX::XMFLLOAT4 COLOR_BG { 0.04f, 0.04f, 0.08f, 1.0f }; // 背景
    const DirectX::XMFLLOAT4 COLOR_TITLE { 0.90f, 0.95f, 1.00f, 1.0f }; // ロゴ文字
    const DirectX::XMFLLOAT4 COLOR_SUB { 0.55f, 0.85f, 1.00f, 1.0f }; // サブタイトル
    const DirectX::XMFLLOAT4 COLOR_HINT { 0.60f, 0.65f, 0.75f, 1.0f }; // プロンプト

    double g_BlinkTimer = 0.0; // 開始プロンプト点滅用の経過時間
}

//=====
// 初期化／終了
//=====
void Title_Initialize()
{
    // 描画モジュールを初期化し、点滅タイマーをリセット
    Prim::Initialize();
    Text::Initialize();
    g_BlinkTimer = 0.0;
}

void Title_Finalize()
{
}

//=====
// 更新
//=====
void Title_Update(double elapsed_time)
{
    // 点滅プロンプト用に経過時間を積算
    g_BlinkTimer += elapsed_time;
}
```

```

// Space / Enter / 左クリックでスタート → 新規ラン開始
if (Keyboard_IsKeyDown(KK_SPACE)
    || Keyboard_IsKeyDown(KK_ENTER)
    || InputManager_IsMouseLeftTrigger())
{
    RunState::ResetRun();          // ラン状態を初期化
    Scene_Change(Scene::GAME);     // ゲーム本編へ遷移を予約
}
}

//=====================================================
// 描画
//=====================================================
void Title_Draw()
{
    const float screenW = static_cast<float>(Direct3D_GetBackBufferWidth());
    const float screenH = static_cast<float>(Direct3D_GetBackBufferHeight());

    // 背景を塗りつぶす
    Prim::DrawRect(0, 0, screenW, screenH, COLOR_BG);

    /*--- タイトルロゴ + サブタイトルを中央に描画 ---*/
    const char* title    = "MARUBATSU";
    const char* subtitle = "〇×ローグライト";
    const float titleW   = Text::MeasureWidth(title, TEXT_TITLE_SIZE);
    const float subW     = Text::MeasureWidth(subtitle, TEXT_SUBTITLE);

    const float titleY   = screenH * 0.35f;
    Text::Draw((screenW - titleW) * 0.5f, titleY, title, TEXT_TITLE_SIZE, COLOR_TITLE);
    Text::Draw((screenW - subW)   * 0.5f, titleY + TEXT_TITLE_SIZE + 8.0f,
               subtitle, TEXT_SUBTITLE, COLOR_SUB);

    /*--- 開始プロンプト (一定周期で点滅) ---*/
    const bool show = (static_cast<int>(g_BlinkTimer * 1.5) % 2) == 0;
    if (show)
    {
        const char* hint = "スペース または クリックで開始";
        const float hintW = Text::MeasureWidth(hint, TEXT_HINT_SIZE);
        Text::Draw((screenW - hintW) * 0.5f, screenH * 0.72f,
                   hint, TEXT_HINT_SIZE, COLOR_HINT);
    }
}
}

```

9.3 報酬シーン (reward)

報酬シーンは、ボス撃破後に3枚のカードからアビリティを1つ選ばせる画面です。ゲーム体験のローグライト性を担う重要なシーンです。

更新 — ホバーと選択

`Reward_Update` は、マウス座標が3枚のカードのどれに乗っているかを判定して `g_HoverIndex` に記録し、左クリックで選択を確定します。確定すると、選んだアビリティを `RunState::PlayerAbilities()` へ追加し、報酬候補をクリアして `Scene::GAME` へ戻ります。次のボス戦が始まります。

描画 — レアリティ別のカード

`Reward_Draw` は、カードを横並びに中央寄せで描きます。各カードは、レアリティ色の縁取り・本体・レアリティ表記・アビリティ名・説明文で構成されます。ホバー中のカードは本体色が明るくなります。カードのレイアウト計算（`CardOriginX` / `CardOriginY`）は更新と描画で共有され、ホバー判定と描画位置が必ず一致するようにしています。

このカードUIの作り（横並び・レアリティ色・ホバー）は、第10章のアビリティ履歴 ポップアップでも同じ発想で再利用されます。

game/scene/reward.h

```
/*
 * @file reward.h
 * @brief 報酬シーン（3択UI）
 * @author Natsume Shidara
 * @date 2026/05/15
 *
 * ボス撃破後に3枚のカードからアビリティを1つ選ばせる画面。
 */

#ifndef REWARD_H
#define REWARD_H

// =====
// 報酬シーン API
// =====
/** @brief 報酬シーンの初期化（報酬候補を生成） */
void Reward_Initialize();

/** @brief 報酬シーンの終了処理 */
void Reward_Finalize();

/**
 * @brief 報酬シーンの更新（ホバー判定・選択確定）
 * @param elapsed_time 前フレームからの経過秒数
 */
void Reward_Update(double elapsed_time);

/** @brief 報酬シーンの描画（カード3枚） */
void Reward_Draw();

#endif // REWARD_H
```

game/scene/reward.cpp

```
/*
 * @file reward.cpp
 * @brief 報酬シーン実装
 * @author Natsume Shidara
 * @date 2026/05/15
 *
 * ボス撃破後に呼ばれ、3枚のカードからアビリティを1つ選ばせる。
 * 選択完了で Scene::GAME へ遷移し、次のボス戦開始。
 */

#include "reward.h"
#include "scene.h"

#include "run_state.h"
#include "render_primitives.h"
#include "text_draw.h"
#include "ability/ability.h"
```

```

#include "direct3d.h"
#include "input_manager.h"
#include "keyboard.h"

#include <DirectXMath.h>

//-----
// 定数・内部状態（無名名前空間）
//-----
namespace
{
    // カードの見た目に関する寸法・文字サイズ
    constexpr float CARD_W      = 280.0f;    // カード幅
    constexpr float CARD_H      = 360.0f;    // カード高さ
    constexpr float CARD_GAP    = 40.0f;     // カード間の余白
    constexpr float CARD_BORDER = 4.0f;      // 縁取りの太さ
    constexpr float TEXT_TITLE  = 28.0f;     // アビリティ名
    constexpr float TEXT_RARITY = 16.0f;     // レアリティ表記
    constexpr float TEXT_DESC   = 16.0f;     // 説明文
    constexpr float TEXT_HEADER = 36.0f;     // 見出し
    constexpr float TEXT_HINT   = 18.0f;     // 操作ヒント

    // 配色 (RGBA)
    const DirectX::XMFLOAT4 COLOR_BG      { 0.04f, 0.04f, 0.08f, 1.0f }; // 背景
    const DirectX::XMFLOAT4 COLOR_CARD_BG { 0.10f, 0.10f, 0.15f, 1.0f }; // カード本体
    const DirectX::XMFLOAT4 COLOR_CARD_HOVER { 0.18f, 0.18f, 0.26f, 1.0f }; // ホバー時の本体
    const DirectX::XMFLOAT4 COLOR_TEXT    { 0.90f, 0.92f, 0.98f, 1.0f }; // 標準文字
    const DirectX::XMFLOAT4 COLOR_HINT    { 0.55f, 0.60f, 0.70f, 1.0f }; // 補助文字
    const DirectX::XMFLOAT4 COLOR_HEADER  { 0.60f, 1.00f, 0.60f, 1.0f }; // 見出し文字

    /**
     * @brief レアリティに対応する表示色を返す
     * @param r レアリティ
     * @return 縁取り・レアリティ表記に使う色
     */
    DirectX::XMFLOAT4 RarityColor(Rarity r)
    {
        switch (r)
        {
            case Rarity::Common:    return { 0.70f, 0.75f, 0.80f, 1.0f };
            case Rarity::Rare:      return { 0.40f, 0.70f, 1.00f, 1.0f };
            case Rarity::Epic:      return { 0.80f, 0.50f, 1.00f, 1.0f };
            case Rarity::Legendary: return { 1.00f, 0.80f, 0.30f, 1.0f };
            case Rarity::Curse:    return { 1.00f, 0.40f, 0.40f, 1.0f };
        }
        return { 1, 1, 1, 1 };
    }

    /**
     * @brief レアリティに対応する日本語名を返す
     * @param r レアリティ
     * @return レアリティ名文字列
     */
    const char* RarityName(Rarity r)
    {
        switch (r)
        {
            case Rarity::Common:    return "コモン";
            case Rarity::Rare:      return "レア";
            case Rarity::Epic:      return "エピック";
            case Rarity::Legendary: return "レジェンダリー";
            case Rarity::Curse:    return "呪い";
        }
        return "";
    }
}

```

```

}

/**
 * @brief index 番目のカードの左端X座標を算出する
 * @param index カード番号 (0始まり)
 * @param screenW 画面幅
 * @param total カード総数
 * @return カード左端のX座標 (カード群は画面中央に配置)
 */
float CardOriginX(int index, float screenW, int total)
{
    const float totalW = total * CARD_W + (total - 1) * CARD_GAP;
    const float startX = (screenW - totalW) * 0.5f;
    return startX + index * (CARD_W + CARD_GAP);
}

/**
 * @brief カードの上端Y座標を算出する
 * @param screenH 画面高さ
 * @return カード上端のY座標 (画面中央やや下寄り)
 */
float CardOriginY(float screenH)
{
    return (screenH - CARD_H) * 0.5f + 20.0f;
}

int g_HoverIndex = -1; // マウスが乗っているカード番号 (-1 = なし)
}

//=====
// 初期化／終了
//=====
void Reward_Initialize()
{
    Prim::Initialize();
    Text::Initialize();

    // 報酬候補が未生成なら生成する
    if (RunState::RewardChoices().empty())
    {
        RunState::GenerateRewardChoices();
    }
    g_HoverIndex = -1;
}

void Reward_Finalize()
{
}

//=====
// 更新
//=====
void Reward_Update(double /*elapsed_time*/)
{
    const auto& choices = RunState::RewardChoices();
    if (choices.empty()) return;

    const float screenW = static_cast<float>(Direct3D_GetBackBufferWidth());
    const float screenH = static_cast<float>(Direct3D_GetBackBufferHeight());
    const int total = static_cast<int>(choices.size());
    const auto& mouse = InputManager_GetMouseState();
    const float oy = CardOriginY(screenH);

    /*--- マウスが乗っているカードを判定 ---*/

```

```

g_HoverIndex = -1;
for (int i = 0; i < total; ++i)
{
    const float ox = CardOriginX(i, screenW, total);
    if (mouse.x >= ox && mouse.x <= ox + CARD_W
        && mouse.y >= oy && mouse.y <= oy + CARD_H)
    {
        g_HoverIndex = i;
        break;
    }
}

/*--- 左クリックで選択を確定 → プレイヤー所持に追加して GAME へ ---*/
if (InputManager_IsMouseLeftTrigger() && g_HoverIndex >= 0)
{
    RunState::PlayerAbilities().push_back(choices[g_HoverIndex]);
    RunState::RewardChoices().clear();
    Scene_Change(Scene::GAME);
    return;
}

/*--- ESC でタイトルへ戻る (ラン中断扱い) ---*/
if (Keyboard_IsKeyDown(KK_ESCAPE))
{
    RunState::RewardChoices().clear();
    Scene_Change(Scene::TITLE);
}
}

//=====
// 描画
//=====
void Reward_Draw()
{
    const float screenW = static_cast<float>(Direct3D_GetBackBufferWidth());
    const float screenH = static_cast<float>(Direct3D_GetBackBufferHeight());

    // 背景を塗りつぶす
    Prim::DrawRect(0, 0, screenW, screenH, COLOR_BG);

    /*--- 見出しを上部中央に描画 ---*/
    const char* header = "報酬を選択";
    const float hw = Text::MeasureWidth(header, TEXT_HEADER);
    Text::Draw((screenW - hw) * 0.5f, 60.0f, header, TEXT_HEADER, COLOR_HEADER);

    const auto& choices = RunState::RewardChoices();
    const int total = static_cast<int>(choices.size());
    const float oy = CardOriginY(screenH);

    /*--- 報酬カードを1枚ずつ描画 ---*/
    for (int i = 0; i < total; ++i)
    {
        const auto& ability = choices[i];
        const float ox = CardOriginX(i, screenW, total);
        const auto rarityColor = RarityColor(ability->rarity);

        // 縁取り (レアリティ色)
        Prim::DrawRect(ox - CARD_BORDER, oy - CARD_BORDER,
                      CARD_W + CARD_BORDER * 2, CARD_H + CARD_BORDER * 2,
                      rarityColor);
        // 本体 (ホバー中は明るい色)
        const auto& bodyColor = (i == g_HoverIndex) ? COLOR_CARD_HOVER : COLOR_CARD_BG;
        Prim::DrawRect(ox, oy, CARD_W, CARD_H, bodyColor);
    }
}

```

```
// レアリティ表記 (カード上部中央)
const char* rarText = RarityName(ability->rarity);
const float rw = Text::MeasureWidth(rarText, TEXT_RARITY);
Text::Draw(ox + (CARD_W - rw) * 0.5f, oy + 18.0f,
           rarText, TEXT_RARITY, rarityColor);

// アビリティ名 (中央寄せ)
const float nw = Text::MeasureWidth(ability->name.c_str(), TEXT_TITLE);
Text::Draw(ox + (CARD_W - nw) * 0.5f, oy + 60.0f,
           ability->name.c_str(), TEXT_TITLE, COLOR_TEXT);

// 説明文 (改行は description 内の \n で明示済み)
Text::Draw(ox + 16.0f, oy + 130.0f, ability->description.c_str(),
           TEXT_DESC, COLOR_HINT);
}

/*--- 操作ヒントを下部中央に描画 ---*/
const char* hint = "カードをクリックで選択    [ESC] ラン中断";
const float hwh = Text::MeasureWidth(hint, TEXT_HINT);
Text::Draw((screenW - hwh) * 0.5f, screenH - 40.0f,
           hint, TEXT_HINT, COLOR_HINT);
}
```


第10章 ゲームシーンの詳解

ゲームシーン（`game.cpp`）は本作の中核であり、最大のファイルです。盤面プレイ、フックの統合、AI、効果アニメーション、HUD、アビリティUIのすべてがここに集まります。本章ではコードの流れを6つの観点に分けて解説し、最後に全ソースを掲載します。

10.1 状態と定数

`game.cpp` は、無名名前空間にレイアウト定数・配色・永続状態をまとめています。ゲームシーンが保持する主な状態は次の通りです。

状態	役割
<code>g_State</code>	試合状態（ <code>GameState</code> ）。盤面・手番・残り時間など。
<code>g_Registry</code>	フック実装の集約（ <code>AbilityRegistry</code> ）。
<code>g_Boss</code> / <code>g_BossAbilities</code>	現在のボスと、そのギミック。
<code>g_AiTimer</code> / <code>g_AiPending</code>	AI着手の遅延制御。
<code>g_AnimPhases</code> ほか	効果アニメーションの再生状態（10.4節）。
<code>g_AbilityPopupOpen</code> ほか	アビリティ履歴ポップアップの状態（10.6節）。

盤面は画面中央に固定サイズ（`BOARD_SIZE`）で描かれます。1マスの大きさ `CellSize()` は、盤面のマス数で `BOARD_SIZE` を割って動的に求めます。盤面拡張で3×3が4×4になっても、この1か所で吸収できる仕組みです。

10.2 試合の初期化フロー

1試合の準備は `ResetMatch()` が行います。流れは次の通りです。

```
ResetMatch()
① g_State を初期化（盤面リセット、結果=Playing、ターン1）
② g_Registry.ResetToDefaults()   フックを標準ルールへ戻す
③ 演出・アニメ・ポップアップ状態を初期化
④ LoadBoss()
    ・ボスを生成（IceBoardBoss）
    ・プレイヤーのアビリティを先に RegisterTo   この順序が
    ・ボスのギミックを後に RegisterTo           ↓ 発火順を決める
⑤ 盤面サイズフックを適用
    int w=3,h=3; g_Registry.GetBoardSize(w,h); board.Resize(w,h);
⑥ BeginTurn(Player)   先手プレイヤーのターン開始
```

ステップ④の**登録順**が重要です。プレイヤーのアビリティを先に登録するため、チェーン（単一フック）ではボス側が外側のラップになり、設置イベント（複数フック）ではプレイヤー効果が先に呼ばれます。第4章・第6章で触れた「氷駒で滑らせてから氷盤で滑らせる」順序が、ここで確定します。

BeginTurn(p) は、その陣営の着手数と思考時間を**フック経由**で取得します（GetPlacementCount / GetTurnTime）。つまり二手打ちや熟考の効果は、ターン開始時に自動で反映されます。敵のターンならAI着手を予約します。

10.3 着手と評価のフロー

駒を1個置く処理は、プレイヤー入力（`HandlePlayerInput`）とAI（`HandleAiTurn`）の両方から、共通の `PlaceAndEvaluate(pos, placedBy)` へ集約されます。

PlaceAndEvaluate(pos, placedBy)

- ① 空きマスでなければ中断
- ② 盤面に駒を置く、placementsRemaining を1減らす
- ③ 設置グロウ演出の位置を記録
- ④ animPhases をクリア、設置直後の盤面を boardStart として控える
- ⑤ g_Registry.OnPlace(...) 設置イベントフック発火
(氷駒・氷盤がここで animPhases に移動を積む)
- ⑥ StartEffectAnimation(boardStart) 積まれたフェーズを再生用に展開
- ⑦ EvaluateAfterPlace(placedBy) 勝敗・ターン進行を評価

`EvaluateAfterPlace` は、勝利 → 引き分け → 追加着手 → ターン終了 の順で判定します。勝利判定で押さえるべき点が1つあります。

両陣営の勝敗を確認する

水盤ギミックは全駒を動かすため、「着手していない側」の駒が並ぶこともあります（ボスが置いた→スライド→プレイヤーの○が斜めに揃う、など）。そのため勝利判定は、着手側だけでなく**プレイヤー・ボスの両方**について `g_Registry.CheckWin` を呼びます。双方が同時に揃った場合はプレイヤーを優先します（ボス側のランダムなギミックによる理不尽な敗北を避けるため）。

10.4 効果アニメーション

氷盤や氷駒は盤面を一瞬で書き換えますが、それをそのまま表示すると駒が「パツと」動いて分かりにくくなります。そこで本作は**フェーズ方式の効果アニメーション**を採用します。

フェーズという単位

盤面を動かす効果フックは、自分の移動を `GameState::animPhases` へ 1つの `MoveList` (=1フェーズ) として積みます。1フェーズ内の移動は同時、フェーズ間は順番に再生されます。たとえば氷駒2個 + 氷盤なら、「氷駒の1マス移動」「氷駒の1マス移動」「氷盤の全駒移動」の3フェーズが積まれ、順に流れます。

再生の仕組み

`StartEffectAnimation` は、各フェーズ開始時点の盤面を `g_PhaseStartBoards` へ前計算します。`Game_Update` はアニメ再生中、入力・AI・タイマーを止めてフェーズを順に進めます。`DrawBoardAndPieces` は、再生中フェーズについて、動かない駒は静止描画し、動く駒は from→to の補間（イーズアウト）で描きます。これにより、駒が滑っていく様子が滑らかに見えます。

設計の指針

「効果が発動したら必ずアニメーションで視覚化する」のが本作の方針です。フェーズ方式は 特定の効果に依存しないため、新しい「駒を動かす効果」を足しても、その効果が `animPhases` へ移動を積みさえすれば、再生側は無改修で対応できます。

10.5 更新ループと入力

`Game_Update` は、状態に応じて3つのモードに分岐します。

```
Game_Update(dt)
・ 演出クロック更新、マウスホイール変化量を算出
・ R キーでリスタート（ESCはウィンドウ側の終了確認に割当て済みのため不使用）

┌ 効果アニメ再生中
│   入力/AI/タイマーを止め、フェーズを順に進めるだけ
├ プレイ中 (result == Playing)
│   HandleTurnTimer（思考時間を消費）
│   ポップアップ表示中?
│       Yes → HandleAbilityPopup（盤面入力は止めるが時間は進行）
│       No  → HandleAbilityClick → 消費しなければ HandlePlayerInput
│   HandleAiTurn
└ ゲーム終了後
    一定時間待つ、結果画面のボタンをマウスで処理
```

入力は「アビリティUIのクリックを先に判定し、消費しなければ盤面クリックへ回す」という優先順で処理します。これにより、アビリティボタンの上をクリックしても誤って駒が置かれません。ESCキーはウィンドウ側で終了確認ダイアログに割り当て済みのため、リスタートやタイトルへの遷移はすべてマウス操作で完結するよう設計されています。

10.6 HUDとアビリティ履歴UI

`DrawHud` は、残り時間・手番・ボス名・所持アビリティを描画します。所持アビリティ表示には、やや込み入った仕様があります。

- **グルーピング** — 同名アビリティは「氷駒 ×3」のようにまとめて表示。`BuildAbilityGroups` が所持リストを同名で集約し、消費系（任意発動）を先頭・続いてレアリティ降順に並べます。
- **5件まで表示** — HUDにはグループを5件まで。超過時は6行目に「.....」ボタンを出します。
- **履歴ポップアップ** — 「.....」クリックで、全所持アビリティを横並びカードで一覧するポップアップを開きます。マウスホイールで左右スクロールでき、消費系カードはクリックで発動できます。

ポップアップ表示中でも時間は進む

履歴ポップアップは表示中でもゲーム時間が進行します（盤面入力だけを止める）。そのため、すぐ使いたい消費系アビリティはHUDの5枠に優先表示し、ポップアップを開かなくても切れるようにしています。「UIを開いている間も時間が進む」という緊張感のあるルールと、それに合わせたUI優先順位の設計です。

以下に `game.cpp` の全ソースを掲載します。本書で解説してきた全システム——フック・アビリティ・ボス・AI・描画・アニメーション——が、ここで1つのシーンとして統合される様子を読み取ってください。

game/scene/game.h

```
/* *****  
 * @file   game.h  
 * @brief  ゲームシーン  
 * @author Natsume Shidara  
 * @date   2025/07/01  
 * @update 2026/05/15 - O×ローグライト用スタブへ書き換え  
 *  
 * 盤面プレイ本編。ボス戦の進行・勝敗判定・描画を担う。  
 * *****/  
#ifndef GAME_H  
#define GAME_H  
  
//=====  
// ゲームシーン API  
//=====  
/** @brief ゲームシーンの初期化 */  
void Game_Initialize();  
  
/** @brief ゲームシーンの終了処理 */  
void Game_Finalize();  
  
/**  
 * @brief ゲームシーンの更新  
 * @param elapsed_time 前フレームからの経過秒数  
 */  
void Game_Update(double elapsed_time);  
  
/** @brief ゲームシーンの描画 */  
void Game_Draw();  
  
#endif // GAME_H
```

game/scene/game.cpp

```
/* *****  
 * @file   game.cpp  
 * @brief  ゲームシーン (Day 3 - フック設計対応)  
 * @author Natsume Shidara  
 * @date   2025/07/01  
 * @update 2026/05/15 - フック設計リファクタ (Day 3)  
 *  
 * 設計のキモ:  
 *   GameState (盤面+ターン状態) + AbilityRegistry (フック実装群) の組合せで、  
 *   勝敗判定/設置時イベント/ターン時間/敗北処理 を全て差し替え可能にする。  
 *  
 * Day 3時点では全フックがデフォルト実装で、Day 2と同等の挙動を再現する。  
 * *****/  
#include "game.h"  
  
using namespace std;  
  
// 盤面プレイ本編の初期化  
void Game_Initialize() {  
    // 初期化処理  
}  
// 盤面プレイ本編の終了処理  
void Game_Finalize() {  
    // 終了処理  
}  
// 盤面プレイ本編の更新  
void Game_Update(double elapsed_time) {  
    // 更新処理  
}  
// 盤面プレイ本編の描画  
void Game_Draw() {  
    // 描画処理  
}
```

```

#include "game.h"

#include "game_state.h"
#include "ai.h"
#include "board_ops.h"
#include "render_primitives.h"
#include "text_draw.h"

#include "ability/ability_registry.h"
#include "ability/abilities/boss_ice_slide.h"
#include "boss/boss.h"
#include "boss/ice_board_boss.h"

#include "run_state.h"

#include "scene.h"
#include "direct3d.h"
#include "input_manager.h"
#include "keyboard.h"

#include <DirectXMath.h>
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <memory>
#include <string>
#include <vector>

//-----
// 定数・内部状態（無名名前空間）
//-----
namespace
{
    //-----
    // 盤面の見た目
    //-----
    constexpr float BOARD_SIZE      = 540.0f;          // 盤面全体の一辺
    constexpr float LINE_THICKNESS  = 4.0f;            // 罫線の太さ
    // 1マスの一辺は盤面サイズ(3×3/4×4)で変わるため CellSize() で動的算出する
    constexpr float PIECE_RADIUS_RATIO = 0.32f;        // 駒半径 / セルサイズ
    constexpr float PIECE_THICKNESS  = 6.0f;          // 駒の線の太さ

    //-----
    // テキスト
    //-----
    constexpr float TEXT_SIZE_HUD    = 24.0f;          // HUD文字
    constexpr float TEXT_SIZE_BIG    = 56.0f;          // 勝敗メッセージ
    constexpr float TEXT_SIZE_HINT   = 18.0f;          // 補助文字

    //-----
    // AI遅延（即時行動だと視認性が悪いため）
    //-----
    constexpr double AI_DELAY = 0.35;

    //-----
    // 効果アニメーション
    //-----
    constexpr double PHASE_DURATION = 0.18;           // 1効果フェーズの再生時間(秒)

    //-----
    // 方向インジケータ（シェブロン矢印）
    //-----
    constexpr int   CHEVRON_COUNT    = 3;             // 滑る先の辺に沿って並べる数
    constexpr float CHEVRON_SIZE     = 18.0f;          // シェブロンの大きさ

```

```

constexpr float CHEVRON_THICKNESS = 6.0f; // シェブロン線の太さ
constexpr float CHEVRON_MARGIN    = 24.0f; // 盤面端からの距離

//-----
// ボタンUI
//-----
constexpr float BUTTON_W          = 240.0f; // ボタン幅
constexpr float BUTTON_H          = 60.0f;  // ボタン高さ
constexpr float BUTTON_GAP        = 32.0f;  // ボタン間の余白
constexpr float TEXT_SIZE_BUTTON = 22.0f;   // ボタン文字サイズ

//-----
// 所持アビリティー一覧
//-----
constexpr float ABILITY_LIST_X = 24.0f; // 一覧の左端X
constexpr float ABILITY_ROW_H  = 36.0f; // 1行の高さ
constexpr float ABILITY_BTN_W  = 220.0f; // 任意発動ボタンの幅
constexpr float ABILITY_BTN_H  = 30.0f;  // 任意発動ボタンの高さ

//-----
// アビリティ履歴ポップアップ
//-----
constexpr int   ABILITY_VISIBLE_MAX = 5;      // HUDに表示する最大グループ数
constexpr float POPUP_CARD_W        = 200.0f; // カード幅
constexpr float POPUP_CARD_H        = 280.0f; // カード高さ
constexpr float POPUP_CARD_GAP      = 24.0f;  // カード間の余白
constexpr float POPUP_SIDE_MARGIN   = 90.0f;  // カード帯の左右マージン
constexpr float POPUP_SCROLL_SPEED  = 1.5f;   // ホイール変化量→スクロール量の係数

//-----
// タイマー加算フィードバック
//-----
constexpr double TIMER_FLASH_DURATION = 0.8; // フラッシュ演出の長さ(秒)

//-----
// カラーテーマ (RGBA)
//-----
const DirectX::XMFLOAT4 COLOR_BG          { 0.04f, 0.04f, 0.08f, 1.0f }; // 背景
const DirectX::XMFLOAT4 COLOR_GRID        { 0.70f, 0.70f, 0.80f, 1.0f }; // 罫線
const DirectX::XMFLOAT4 COLOR_PIECE_O     { 0.50f, 0.85f, 1.00f, 1.0f }; // ○ (プレイヤー)
const DirectX::XMFLOAT4 COLOR_PIECE_X     { 1.00f, 0.45f, 0.85f, 1.0f }; // × (敵)
const DirectX::XMFLOAT4 COLOR_TEXT        { 0.85f, 0.90f, 0.95f, 1.0f }; // 標準文字
const DirectX::XMFLOAT4 COLOR_TEXT_HINT   { 0.55f, 0.60f, 0.70f, 1.0f }; // 補助文字
const DirectX::XMFLOAT4 COLOR_TEXT_WIN    { 0.60f, 1.00f, 0.60f, 1.0f }; // 勝利
const DirectX::XMFLOAT4 COLOR_TEXT_LOSE   { 1.00f, 0.50f, 0.50f, 1.0f }; // 敗北
const DirectX::XMFLOAT4 COLOR_TEXT_DRAW   { 0.80f, 0.80f, 0.90f, 1.0f }; // 引き分け
const DirectX::XMFLOAT4 COLOR_TEXT_URGENT { 1.00f, 0.85f, 0.40f, 1.0f }; // 警告 (残時間少)
const DirectX::XMFLOAT4 COLOR_SLIDE_ARROW { 0.55f, 0.85f, 1.00f, 1.0f }; // 滑り方向矢印
const DirectX::XMFLOAT4 COLOR_BTN_BG      { 0.12f, 0.13f, 0.20f, 0.96f }; // ボタン本体
const DirectX::XMFLOAT4 COLOR_BTN_HOVER   { 0.24f, 0.28f, 0.40f, 0.98f }; // ボタン (ホバー)

//-----
// 永続状態
//-----
GameState      g_State; // 盤面 + ターン状態
AbilityRegistry g_Registry; // フック実装群
double         g_AiTimer = 0.0; // AI着手までの残り遅延
bool           g_AiPending = false; // AI着手待ちフラグ

std::unique_ptr<Boss> g_Boss; // 現在のボス
std::vector<std::shared_ptr<Ability>> g_BossAbilities; // ボス側アビリティ
std::shared_ptr<BossIceSlideAbility> g_IceSlideRef; // HUD表示用 (氷盤参照)

//-----

```

```

// 直近着手の演出
//-----
Vec2  g_LastPlacedPos { -1, -1 };          // 直近に駒を置いたマス
Piece g_LastPlacedPiece = Piece::Empty;    // 直近に置いた駒の陣営
double g_LastPlacedTimer = 0.0;            // グロー演出の残り時間
constexpr double GLOW_DURATION = 0.55;    // グロー演出の長さ(秒)

//-----
// GameOver後の表示 → 報酬遷移の待機
//-----
double g_GameOverDwell = 0.0;              // ゲーム終了後の経過時間
constexpr double GAMEOVER_MIN_DWELL = 0.6; // ボタン表示までの最低待機(秒)

//-----
// 効果アニメーション再生状態
// 駒を動かす効果(氷盤/氷駒/連鎖など)を「フェーズ」単位で順次再生する。
// 1フェーズ内の移動は同時、フェーズ間は順番に再生される。
//-----
std::vector<BoardOps::MoveList> g_AnimPhases;          // 各フェーズの移動
std::vector<Board> g_PhaseStartBoards;               // 各フェーズ開始時の盤面
int g_AnimPhaseIndex = 0;                            // 再生中フェーズ番号 (size以上で再生終了)
double g_AnimPhaseTimer = 0.0;                       // 現フェーズの残り時間

double g_AnimClock = 0.0; // 演出用クロック (シェブロンの脈動)
double g_TimerFlash = 0.0; // 思考時間加算フラッシュ演出の残り時間

//-----
// アビリティ履歴ポップアップの状態
//-----
bool g_AbilityPopupOpen = false; // ポップアップ表示中か
double g_PopupScroll = 0.0;      // カード帯の横スクロール量(px)
int g_WheelPrev = 0;             // 前フレームのホイール累積値
int g_WheelDelta = 0;            // 当該フレームのホイール変化量

//=====
// 補助関数 (状態問い合わせ・座標変換)
//=====
/**
 * @brief 効果アニメーションが再生中かを返す
 * @return 再生中なら true
 */
bool IsEffectAnimating()
{
    return g_AnimPhaseIndex < static_cast<int>(g_AnimPhases.size());
}

/** @brief 盤面左端のX座標 (画面中央寄せ) */
float BoardOriginX() { return (Direct3D_GetBackBufferWidth() - BOARD_SIZE) * 0.5f; }
/** @brief 盤面上端のY座標 (画面中央寄せ) */
float BoardOriginY() { return (Direct3D_GetBackBufferHeight() - BOARD_SIZE) * 0.5f; }

/**
 * @brief 現在の盤面サイズに応じた1マスの一辺を返す
 * @return BOARD_SIZE を盤面の最大マス数で割った値 (正方マス)
 * @detail 盤面拡張アビリティで 3x3→4x4 等に変わるため動的に算出する。
 */
float CellSize()
{
    const int n = (g_State.board.width > g_State.board.height)
        ? g_State.board.width : g_State.board.height;
    return BOARD_SIZE / static_cast<float>(n > 0 ? n : 1);
}

/**

```

```

* @brief スクリーン座標を盤面のセル座標へ変換する
* @param sx スクリーンX座標
* @param sy スクリーンY座標
* @return 命中したセル座標。盤外なら {-1,-1}
*/
Vec2 ScreenToCell(int sx, int sy)
{
    const float rx = sx - BoardOriginX();
    const float ry = sy - BoardOriginY();
    if (rx < 0 || ry < 0 || rx >= BOARD_SIZE || ry >= BOARD_SIZE)
    {
        return { -1, -1 };
    }
    const float CELL_SIZE = CellSize();
    return { static_cast<int>(rx / CELL_SIZE),
            static_cast<int>(ry / CELL_SIZE) };
}

//=====
// 所持アビリティー一覧のレイアウト
// 画面左下にヘッダ + count行を配置する。Draw/クリック判定で共通使用。
//=====
/**
* @brief アビリティー一覧（ヘッダ含む）の上端Y座標
* @param count 一覧の行数
* @return ヘッダ行の上端Y座標
*/
float AbilityListTop(int count)
{
    return static_cast<float>(Direct3D_GetBackBufferHeight())
        - 24.0f - (TEXT_SIZE_HUD + 6.0f + count * ABILITY_ROW_H);
}

/**
* @brief index 番目のアビリティー行の上端Y座標
* @param index 行番号 (0始まり)
* @param count 一覧の行数
* @return 該当行の上端Y座標
*/
float AbilityRowY(int index, int count)
{
    return AbilityListTop(count) + TEXT_SIZE_HUD + 6.0f
        + index * ABILITY_ROW_H;
}

//=====
// 所持アビリティーのグルーピング
//=====
/**
* @struct AbilityGroup
* @brief 同名アビリティーをまとめた表示単位
* @detail 同じアビリティーを複数取得した場合、1グループ + 取得数で表す
*         (例: 氷駒を3個 → "氷駒 ×3")。
*/
struct AbilityGroup
{
    Ability*      rep    = nullptr; // 代表インスタンス (名前/説明/レアリティ表示用)
    int           count = 0;         // 取得回数
    std::vector<Ability*> instances; // 同名アビリティーの実体一覧
};

/**
* @brief プレイヤー所持アビリティーを同名でまとめ、表示順にソートして返す
* @return グループ一覧。消費系(任意発動)を先頭に、続いてレアリティ降順。

```



```

* @detail HUDの5枠表示・履歴ポップアップの双方で共通利用する。
*/
std::vector<AbilityGroup> BuildAbilityGroups()
{
    std::vector<AbilityGroup> groups;
    for (const auto& a : RunState::PlayerAbilities())
    {
        // 同名グループがあれば実体を追加、無ければ新規グループを作る
        AbilityGroup* found = nullptr;
        for (auto& g : groups)
        {
            if (g.rep->name == a->name) { found = &g; break; }
        }
        if (found)
        {
            found->instances.push_back(a.get());
            ++found->count;
        }
        else
        {
            AbilityGroup g;
            g.rep = a.get();
            g.count = 1;
            g.instances.push_back(a.get());
            groups.push_back(std::move(g));
        }
    }
    // 消費系(任意発動)を優先し、その後はレアリティの高い順に並べる
    std::stable_sort(groups.begin(), groups.end(),
        [](const AbilityGroup& x, const AbilityGroup& y)
        {
            if (x.rep->activatable != y.rep->activatable)
                return x.rep->activatable; // 任意発動を先頭へ
            return static_cast<int>(x.rep->rarity)
                > static_cast<int>(y.rep->rarity);
        });
    return groups;
}

/**
* @brief グループの残りチャージ合計を返す
* @param g 対象グループ
* @return チャージ合計。無制限(回数表示なし)を含む場合は -1。
*/
int GroupCharges(const AbilityGroup& g)
{
    int total = 0;
    for (Ability* a : g.instances)
    {
        const int c = a->ChargesLeft();
        if (c < 0) return -1; // 無制限が混在 → 回数表示なし
        total += c;
    }
    return total;
}

/**
* @brief グループ内に発動可能なインスタンスがあるか判定
* @param g 対象グループ
* @return 1つでも発動可能なら true
*/
bool GroupCanActivate(const AbilityGroup& g)
{
    for (Ability* a : g.instances)

```

```

    {
        if (a->CanActivate(g_State)) return true;
    }
    return false;
}

/**
 * @brief グループ内の発動可能なインスタンスを1つ発動する
 * @param g 対象グループ
 * @detail 取得順で最初に発動できたものを使用し、加算フラッシュを出す。
 */
void GroupActivate(const AbilityGroup& g)
{
    for (Ability* a : g.instances)
    {
        if (a->CanActivate(g_State))
        {
            a->Activate(g_State);
            g_TimerFlash = TIMER_FLASH_DURATION;    // 加算フィードバック
            return;
        }
    }
}

/**
 * @brief レアリティに対応する表示色を返す
 * @param r レアリティ
 */
DirectX::XMFLLOAT4 RarityColor(Rarity r)
{
    switch (r)
    {
    {
        case Rarity::Common:    return { 0.70f, 0.75f, 0.80f, 1.0f };
        case Rarity::Rare:     return { 0.40f, 0.70f, 1.00f, 1.0f };
        case Rarity::Epic:     return { 0.80f, 0.50f, 1.00f, 1.0f };
        case Rarity::Legendary: return { 1.00f, 0.80f, 0.30f, 1.0f };
        case Rarity::Curse:    return { 1.00f, 0.40f, 0.40f, 1.0f };
    }
    return { 1, 1, 1, 1 };
}

/**
 * @brief レアリティに対応する日本語名を返す
 * @param r レアリティ
 */
const char* RarityName(Rarity r)
{
    switch (r)
    {
    {
        case Rarity::Common:    return "コモン";
        case Rarity::Rare:     return "レア";
        case Rarity::Epic:     return "エピック";
        case Rarity::Legendary: return "レジェンダリー";
        case Rarity::Curse:    return "呪い";
    }
    return "";
}

//=====
// ターン進行
//=====
/**
 * @brief 指定陣営のターンを開始する
 * @param p 手番を握る陣営

```

```

* @detail 着手数・思考時間をフックから取得し、敵番ならAI着手を予約する。
*/
void BeginTurn(Piece p)
{
    g_State.currentPlayer      = p;
    g_State.placementsRemaining = g_Registry.GetPlacementCount(p);
    g_State.remainingTime      = g_Registry.GetTurnTime(p);
    g_Registry.OnTurnStart(g_State, p);

    if (p == Piece::Enemy)
    {
        g_AiPending = true;
        g_AiTimer    = AI_DELAY;
    }
    else
    {
        g_AiPending = false;
    }
}

/**
* @brief ボスを生成し、アビリティをレジストリへ登録する
* @detail プロト版は固定でボス1。プレイヤー側を先に登録することで
*         OnPlace がプレイヤー効果 → ボス効果の順に発火する。
*/
void LoadBoss()
{
    // プロト版：固定でボス1を起動。
    // α版で RunManager が boss index に応じて切り替える。
    g_Boss = std::make_unique<IceBoardBoss>();
    g_BossAbilities = g_Boss->GetBossAbilities();
    g_IceSlideRef.reset();

    // プレイヤー側を先に登録 → OnPlace はプレイヤー効果が先に発火する。
    // (氷駒が「置いた直後の駒」を滑らせてから、氷盤が全体を滑らせる)
    for (auto& a : RunState::PlayerAbilities())
    {
        a->RegisterTo(g_Registry);
    }
    // ボス側を後に登録 (ITurnHandler等のチェーンはボス側が最終ラッパ)
    for (auto& a : g_BossAbilities)
    {
        a->RegisterTo(g_Registry);
        if (auto ice = std::dynamic_pointer_cast<BossIceSlideAbility>(a))
        {
            g_IceSlideRef = ice;
        }
    }
}

/**
* @brief 対戦を初期状態へリセットする
* @detail 盤面・演出・レジストリを初期化し、ボスを再ロードして
*         プレイヤーの先手ターンを開始する。
*/
void ResetMatch()
{
    /*--- 盤面・対戦状態の初期化 ---*/
    g_State = {};
    g_State.board.Reset();
    g_State.result = MatchResult::Playing;
    g_State.winner = Piece::Empty;
    g_State.turnCount = 1;
}

```

```

/*--- フック・演出状態の初期化 ---*/
g_Registry.ResetToDefaults();
g_LastPlacedPos    = { -1, -1 };
g_LastPlacedPiece  = Piece::Empty;
g_LastPlacedTimer  = 0.0;
g_GameOverDwell    = 0.0;
g_AnimPhases.clear();
g_PhaseStartBoards.clear();
g_AnimPhaseIndex   = 0;
g_AnimPhaseTimer   = 0.0;
g_AbilityPopupOpen = false;
g_PopupScroll      = 0.0;

/*--- ボス再ロード → 盤面サイズ確定 → 先手ターン開始 ---*/
LoadBoss();

// 盤面サイズフックを適用（盤面拡張アビリティ等を反映）
int boardW = 3, boardH = 3;
g_Registry.GetBoardSize(boardW, boardH);
g_State.board.Resize(boardW, boardH);

BeginTurn(Piece::Player);
}

//=====
// 着手後の判定
//=====
/**
 * @brief 駒を1個置いた直後の勝敗・ターン進行を評価する
 * @param justPlaced 直前に駒を置いた陣営
 * @detail 勝利→引き分け→追加着手→ターン終了の順で判定する。
 *         OnPlace は PlaceAndEvaluate 側で先に呼ぶため、ここでは扱わない。
 *         勝利判定はボスギミック（氷盤スライド等）で盤面が動いた後の
 *         最終盤面に対して行い、着手側だけでなく両陣営を確認する。
 */
void EvaluateAfterPlace(Piece justPlaced)
{
    // 1) 設置時イベント（アビリティ用フック）
    // ※ EvaluateAfterPlace の前にOnPlaceを呼んでいるので、ここでは省略。
    // OnPlaceは PlaceAndEvaluate 側で先に呼ぶ。

    // 2) 勝利判定
    // ボスギミックで全駒が動くと、着手していない側の駒が揃うこともある。
    // そのため着手側だけでなく両陣営の勝利条件を確認する。
    const bool playerWon = g_Registry.CheckWin(g_State.board, Piece::Player);
    const bool enemyWon  = g_Registry.CheckWin(g_State.board, Piece::Enemy);
    if (playerWon || enemyWon)
    {
        // 双方同時成立時はプレイヤーを優先（理不尽な敗北を避ける）
        const Piece winner = playerWon ? Piece::Player : Piece::Enemy;
        g_State.winner = winner;
        g_State.result = (winner == Piece::Player)
            ? MatchResult::Win : MatchResult::Lose;
        return;
    }

    // 3) 引き分け判定
    if (g_State.board.IsFull())
    {
        g_State.result = MatchResult::Draw;
        return;
    }

    // 4) 同一プレイヤーの追加着手が残っているか

```

```

    if (g_State.placementsRemaining > 0) return;

    // 5) ターン終了
    g_Registry.OnTurnEnd(g_State, justPlaced);
    ++g_State.turnCount;
    BeginTurn(Opponent(justPlaced));
}

/**
 * @brief 1フェーズ分の移動を盤面へ適用する
 * @param board 対象の盤面 (更新される)
 * @param phase 適用する移動リスト
 * @detail 全 source を空にしてから全 dest へ配置する (同時移動を表現)。
 */
void ApplyPhase(Board& board, const BoardOps::MoveList& phase)
{
    for (const auto& m : phase) board.Set(m.fromX, m.fromY, Piece::Empty);
    for (const auto& m : phase) board.Set(m.toX, m.toY, m.piece);
}

/**
 * @brief OnPlace で積まれた効果フェーズ列をアニメ再生用にセットアップ
 * @param boardStart 効果適用前 (新駒設置済み) の盤面
 * @detail 各フェーズ開始時の盤面を前計算し、描画時の中間状態復元に使う。
 */
void StartEffectAnimation(const Board& boardStart)
{
    g_AnimPhases = g_State.animPhases;
    g_PhaseStartBoards.clear();
    g_AnimPhaseIndex = 0;
    g_AnimPhaseTimer = 0.0;
    if (g_AnimPhases.empty()) return;

    // 各フェーズ開始時の盤面を前計算 (描画時の中間状態復元に使う)
    g_PhaseStartBoards.reserve(g_AnimPhases.size());
    Board cur = boardStart;
    for (const auto& phase : g_AnimPhases)
    {
        g_PhaseStartBoards.push_back(cur);
        ApplyPhase(cur, phase);
    }
    g_AnimPhaseTimer = PHASE_DURATION;
}

/**
 * @brief 駒を1個置き、フック発火と勝敗評価まで行う
 * @param pos 設置先のセル座標
 * @param placedBy 置く陣営
 * @detail 空きマスでなければ何もしない。設置 → 演出記録 →
 *         OnPlace フック → 効果アニメ準備 → 勝敗評価 の順に処理する。
 */
void PlaceAndEvaluate(Vec2 pos, Piece placedBy)
{
    if (!g_State.board.IsEmpty(pos.x, pos.y)) return;
    g_State.board.Set(pos.x, pos.y, placedBy);
    --g_State.placementsRemaining;

    // 設置時演出 (フック発火前に位置を記録)
    g_LastPlacedPos = pos;
    g_LastPlacedPiece = placedBy;
    g_LastPlacedTimer = GLOW_DURATION;

    // 効果フックの出力をクリアし、OnPlace前 (新駒設置済み) の盤面を控える
    g_State.animPhases.clear();

```

```

const Board boardStart = g_State.board;

// 設置時フック (氷駒/氷盤/連鎖/爆弾等が animPhases に効果を積む)
g_Registry.OnPlace(g_State, pos, placedBy);

// 積まれた効果フェーズを順次アニメ再生
StartEffectAnimation(boardStart);

EvaluateAfterPlace(placedBy);
}

//=====
// 入力 / AI
//=====
/**
 * @brief プレイヤーのマウス入力処理し、駒を設置する
 * @detail プレイヤーの手番かつ左クリック、命中マスが空のときのみ設置。
 */
void HandlePlayerInput()
{
    if (g_State.currentPlayer != Piece::Player) return;
    if (!InputManager_IsMouseLeftTrigger()) return;
    const auto& mouse = InputManager_GetMouseState();
    const Vec2 cell = ScreenToCell(mouse.x, mouse.y);
    if (!g_State.board.IsEmpty(cell.x, cell.y)) return;
    PlaceAndEvaluate(cell, Piece::Player);
}

/**
 * @brief 所持アビリティ一覧(HUD)のクリック処理する
 * @return クリックを消費したら true (= 駒設置へ回さない)
 * @detail 任意発動ボタンの命中で発動、「.....」行で履歴ポップアップを開く。
 *         発動不可でもボタン命中ならクリックを消費する。
 */
bool HandleAbilityClick()
{
    if (!InputManager_IsMouseLeftTrigger()) return false;

    const auto groups = BuildAbilityGroups();
    if (groups.empty()) return false;

    const int total = static_cast<int>(groups.size());
    const int visible = (total < ABILITY_VISIBLE_MAX)
        ? total : ABILITY_VISIBLE_MAX;
    const bool showMore = total > ABILITY_VISIBLE_MAX;
    const int rowCount = visible + (showMore ? 1 : 0);
    const auto& mouse = InputManager_GetMouseState();

    // 「.....」行: 履歴ポップアップを開く (手番を問わず操作可)
    if (showMore)
    {
        const float ry = AbilityRowY(visible, rowCount);
        if (mouse.x >= ABILITY_LIST_X && mouse.x <= ABILITY_LIST_X + ABILITY_BTN_W
            && mouse.y >= ry && mouse.y <= ry + ABILITY_BTN_H)
        {
            g_AbilityPopupOpen = true;
            g_PopupScroll = 0.0;
            return true;
        }
    }

    // 任意発動ボタン: プレイヤー手番のみ発動できる
    if (g_State.currentPlayer != Piece::Player) return false;
    for (int i = 0; i < visible; ++i)

```

```

{
    if (!groups[i].rep->activatable) continue;
    const float ry = AbilityRowY(i, rowCount);
    if (mouse.x < ABILITY_LIST_X || mouse.x > ABILITY_LIST_X + ABILITY_BTN_W) continue;
    if (mouse.y < ry || mouse.y > ry + ABILITY_BTN_H) continue;

    // ボタンに命中 - 発動可能なら発動 (不可でもクリックは消費する)
    if (GroupCanActivate(groups[i])) GroupActivate(groups[i]);
    return true;
}
return false;
}

/**
 * @brief AIの手番を処理する
 * @param dt 前フレームからの経過秒数
 * @detail AI_DELAY 経過後に着手。追加着手が残れば再度AIを予約する。
 */
void HandleAiTurn(double dt)
{
    if (g_State.currentPlayer != Piece::Enemy || !g_AiPending) return;
    g_AiTimer -= dt;
    if (g_AiTimer > 0.0) return;

    const Vec2 v = AI::ChooseMove_Random(
        g_State.board, Piece::Enemy, g_Registry.GetWinCondition());
    g_AiPending = false;
    if (v.x < 0)
    {
        g_State.result = MatchResult::Draw;
        return;
    }
    PlaceAndEvaluate(v, Piece::Enemy);
    // 追加着手が残っていれば再びAI実行
    if (g_State.placementsRemaining > 0 && g_State.result == MatchResult::Playing)
    {
        g_AiPending = true;
        g_AiTimer = AI_DELAY;
    }
}

/**
 * @brief プレイヤー手番の思考時間を消費させる
 * @param dt 前フレームからの経過秒数
 * @detail 残時間が尽きたら敗北フックを試行し、打ち消されなければ時間切れ。
 */
void HandleTurnTimer(double dt)
{
    if (g_State.result != MatchResult::Playing) return;
    if (g_State.currentPlayer != Piece::Player) return;
    g_State.remainingTime -= dt;
    if (g_State.remainingTime > 0.0) return;

    g_State.remainingTime = 0.0;
    // 敗北フック (不死等で打ち消し可能)
    if (g_Registry.HandleDefeat(g_State, Piece::Player))
    {
        // 敗北回避: 残時間を新規取得
        g_State.remainingTime = g_Registry.GetTurnTime(Piece::Player);
        return;
    }
    g_State.result = MatchResult::Timeout;
}

```

```

//=====
// 描画
//=====
/**
 * @brief 1マス分の駒を描画する
 * @param p 駒の陣営 (Empty なら何も描かない)
 * @param cx セル中心のX座標
 * @param cy セル中心のY座標
 * @param radius 駒の半径
 */
void DrawPiece(Piece p, float cx, float cy, float radius)
{
    switch (p)
    {
        case Piece::Player:
            Prim::DrawCircle(cx, cy, radius, PIECE_THICKNESS, COLOR_PIECE_0);
            break;
        case Piece::Enemy:
            Prim::DrawCross(cx, cy, radius, PIECE_THICKNESS, COLOR_PIECE_X);
            break;
        default:
            break;
    }
}

/**
 * @brief 指定方向を指すシェブロン矢印 ( > < ^ v ) を描画する
 * @param cx 矢印中心のX座標
 * @param cy 矢印中心のY座標
 * @param d 指し示す方向
 * @param size 矢印の大きさ
 * @param thickness 線の太さ
 * @param color 描画色
 */
void DrawChevron(float cx, float cy, Direction d,
                 float size, float thickness, const DirectX::XMFLOAT4& color)
{
    // 方向ごとに頂点(apex)と両翼端(a1,a2)の座標を決定する
    float apexX, apexY, a1x, a1y, a2x, a2y;
    switch (d)
    {
        case Direction::Right:
            apexX = cx + size; apexY = cy;
            a1x = cx - size; a1y = cy - size;
            a2x = cx - size; a2y = cy + size;
            break;
        case Direction::Left:
            apexX = cx - size; apexY = cy;
            a1x = cx + size; a1y = cy - size;
            a2x = cx + size; a2y = cy + size;
            break;
        case Direction::Down:
            apexX = cx; apexY = cy + size;
            a1x = cx - size; a1y = cy - size;
            a2x = cx + size; a2y = cy - size;
            break;
        case Direction::Up:
            apexX = cx; apexY = cy - size;
            a1x = cx - size; a1y = cy + size;
            a2x = cx + size; a2y = cy + size;
            break;
        default:
            apexX = cx; apexY = cy - size;
            a1x = cx - size; a1y = cy + size;
            a2x = cx + size; a2y = cy + size;
            break;
    }
    // 翼端→頂点→翼端 の2本の線で「く」の字を描く

```



```

    Prim::DrawLine(a1x, a1y, apexX, apexY, thickness, color);
    Prim::DrawLine(apexX, apexY, a2x, a2y, thickness, color);
}

/**
 * @brief 盤面の罫線と駒を描画する
 * @detail 効果アニメ中はフェーズ中間状態を補間描画し、
 *          非アニメ時は現在の盤面をそのまま描画する。
 */
void DrawBoardAndPieces()
{
    const float ox = BoardOriginX();
    const float oy = BoardOriginY();

    // 当該フレームの1マスサイズ (盤面拡張で変動するため動的取得)
    const float CELL_SIZE = CellSize();

    /*--- 盤面背景と罫線 ---*/
    Prim::DrawRect(ox - 8, oy - 8, BOARD_SIZE + 16, BOARD_SIZE + 16, COLOR_BG);

    // 内側の罫線を盤面サイズに応じて引く (3x3なら2本、4x4なら3本)
    const int cols = g_State.board.width;
    const int rows = g_State.board.height;
    for (int i = 1; i < cols; ++i)
    {
        const float gx = ox + CELL_SIZE * i;
        Prim::DrawLine(gx, oy, gx, oy + CELL_SIZE * rows, LINE_THICKNESS, COLOR_GRID);
    }
    for (int i = 1; i < rows; ++i)
    {
        const float gy = oy + CELL_SIZE * i;
        Prim::DrawLine(ox, gy, ox + CELL_SIZE * cols, gy, LINE_THICKNESS, COLOR_GRID);
    }

    const float halfCell = CELL_SIZE * 0.5f;
    const float pieceRadius = CELL_SIZE * PIECE_RADIUS_RATIO;

    /*--- 直近設置のグロー (拡大半径・低不透明度の同色形状を後ろに重ねる) ---*/
    if (g_LastPlacedTimer > 0.0f && g_LastPlacedPos.x >= 0)
    {
        const float t = static_cast<float>(g_LastPlacedTimer / GLOW_DURATION); // 1→0
        const float glowRadius = pieceRadius * (1.0f + (1.0f - t) * 0.6f);
        const float alpha = t * 0.5f;
        const float cx = ox + g_LastPlacedPos.x * CELL_SIZE + halfCell;
        const float cy = oy + g_LastPlacedPos.y * CELL_SIZE + halfCell;
        DirectX::XMVECTOR glowColor = (g_LastPlacedPiece == Piece::Player)
            ? DirectX::XMVECTOR(COLOR_PIECE_O.x, COLOR_PIECE_O.y, COLOR_PIECE_O.z, alpha)
            : DirectX::XMVECTOR(COLOR_PIECE_X.x, COLOR_PIECE_X.y, COLOR_PIECE_X.z, alpha);
        Prim::DrawCircle(cx, cy, glowRadius, PIECE_THICKNESS * 2.0f, glowColor, 24);
    }

    if (IsEffectAnimating())
    {
        /*--- 効果アニメ中: 再生中フェーズの中間状態を描画 ---*/
        const int phaseIdx = g_AnimPhaseIndex;
        const Board& disp = g_PhaseStartBoards[phaseIdx];
        const BoardOps::MoveList& phase = g_AnimPhases[phaseIdx];

        // 進捗 0→1 (ease-out で滑らかに減速)
        float raw = 1.0f - static_cast<float>(g_AnimPhaseTimer / PHASE_DURATION);
        raw = (raw < 0.0f) ? 0.0f : (raw > 1.0f ? 1.0f : raw);
        const float t = 1.0f - (1.0f - raw) * (1.0f - raw);

        // このフェーズで動かない駒を静止描画 (出発マスの駒は補間描画に回す)

```

```

        for (int y = 0; y < disp.height; ++y)
        {
            for (int x = 0; x < disp.width; ++x)
            {
                bool isMover = false;
                for (const auto& m : phase)
                {
                    if (m.fromX == x && m.fromY == y) { isMover = true; break; }
                }
                if (isMover) continue;
                const float cx = ox + x * CELL_SIZE + halfCell;
                const float cy = oy + y * CELL_SIZE + halfCell;
                DrawPiece(disp.Get(x, y), cx, cy, pieceRadius);
            }
        }
        // 移動駒を from->to で補間描画
        for (const auto& m : phase)
        {
            const float fx = m.fromX + (m.toX - m.fromX) * t;
            const float fy = m.fromY + (m.toY - m.fromY) * t;
            const float cx = ox + fx * CELL_SIZE + halfCell;
            const float cy = oy + fy * CELL_SIZE + halfCell;
            DrawPiece(m.piece, cx, cy, pieceRadius);
        }
    }
    else
    {
        /*--- 非アニメ時: 現在の盤面をそのまま描画 ---*/
        for (int y = 0; y < g_State.board.height; ++y)
        {
            for (int x = 0; x < g_State.board.width; ++x)
            {
                const float cx = ox + x * CELL_SIZE + halfCell;
                const float cy = oy + y * CELL_SIZE + halfCell;
                DrawPiece(g_State.board.Get(x, y), cx, cy, pieceRadius);
            }
        }
    }
}

/**
 * @brief 氷盤が次に滑る方向を、辺に沿ったシェブロン矢印で可視化する
 * @detail 氷盤ボスが存在しプレイ中のときのみ描画。矢印は滑る方向へ
 *         流れる脈動アニメーションを伴う。
 */
void DrawDirectionIndicator()
{
    if (!g_IceSlideRef) return;
    if (g_State.result != MatchResult::Playing) return;

    const Direction d = g_IceSlideRef->currentDir;
    const float ox = BoardOriginX();
    const float oy = BoardOriginY();
    const bool horizontalSlide = (d == Direction::Left || d == Direction::Right);

    for (int i = 0; i < CHEVRON_COUNT; ++i)
    {
        // 辺に沿った位置 (端を避けて等間隔に分散)
        const float frac = (i + 1.0f) / (CHEVRON_COUNT + 1.0f);
        const float along = BOARD_SIZE * frac;

        float cx, cy;
        if (horizontalSlide)
        {

```

```

        // 左右スライド → 縦の辺に沿って縦並び
        cy = oy + along;
        cx = (d == Direction::Right) ? (ox + BOARD_SIZE + CHEVRON_MARGIN)
                                     : (ox - CHEVRON_MARGIN);
    }
    else
    {
        // 上下スライド → 横の辺に沿って横並び
        cx = ox + along;
        cy = (d == Direction::Down) ? (oy + BOARD_SIZE + CHEVRON_MARGIN)
                                     : (oy - CHEVRON_MARGIN);
    }

    // 滑る方向へ流れる脈動 (位相を矢印ごとにずらす)
    const float wave = 0.5f + 0.5f * std::sin(
        static_cast<float>(g_AnimClock) * 5.0f - i * 0.9f);
    DirectX::XMFLOAT4 col = COLOR_SLIDE_ARROW;
    col.w = 0.30f + 0.60f * wave;

    DrawChevron(cx, cy, d, CHEVRON_SIZE, CHEVRON_THICKNESS, col);
}

}

/**
 * @brief HUD (残時間・手番・ポス名・所持アビリティ) を描画する
 */
void DrawHud()
{
    const float screenW = static_cast<float>(Direct3D_GetBackBufferWidth());

    /*--- 残り思考時間 (左上) ---*/
    char buf[64];
    const int total = static_cast<int>(g_State.remainingTime + 0.999);
    const int minutes = total / 60;
    const int seconds = total % 60;
    std::snprintf(buf, sizeof(buf), "残り %d:%02d", minutes, seconds);
    DirectX::XMFLOAT4 timeColor =
        (g_State.remainingTime < 30.0
         && g_State.result == MatchResult::Playing
         && g_State.currentPlayer == Piece::Player)
        ? COLOR_TEXT_URGENT : COLOR_TEXT;
    if (g_TimerFlash > 0.0) timeColor = COLOR_TEXT_WIN; // 時間加算フラッシュ
    Text::Draw(24.0f, 24.0f, buf, TEXT_SIZE_HUD, timeColor);

    /*--- 集中などで加算した直後の「+2:00」浮き上がり表示 ---*/
    if (g_TimerFlash > 0.0)
    {
        const float k = static_cast<float>(g_TimerFlash / TIMER_FLASH_DURATION); // 1→0
        DirectX::XMFLOAT4 popColor = COLOR_TEXT_WIN;
        popColor.w = k;
        const float popY = 24.0f - (1.0f - k) * 22.0f;
        Text::Draw(24.0f + Text::MeasureWidth(buf, TEXT_SIZE_HUD) + 14.0f,
                   popY, "+2:00", TEXT_SIZE_HINT, popColor);
    }

    /*--- 現在の手番表示 (右上) ---*/
    if (g_State.result == MatchResult::Playing)
    {
        const char* turn =
            (g_State.currentPlayer == Piece::Player) ? "あなたの番 (○)" : "ボスの番 (×)";
        const float w = Text::MeasureWidth(turn, TEXT_SIZE_HUD);
        Text::Draw(screenW - w - 24.0f, 24.0f, turn, TEXT_SIZE_HUD, COLOR_TEXT);
    }
}

```

```

/*--- ボス情報 (画面中央上) ---*/
if (g_Boss)
{
    char nameBuf[64];
    std::snprintf(nameBuf, sizeof(nameBuf), "ボス : %s", g_Boss->name.c_str());
    const float nameW = Text::MeasureWidth(nameBuf, TEXT_SIZE_HUD);
    Text::Draw((screenW - nameW) * 0.5f, 24.0f, nameBuf,
                TEXT_SIZE_HUD, COLOR_TEXT_URGENT);
}
// 氷盤の滑り方向は DrawDirectionIndicator のシェブロン矢印で可視化

/*--- 所持アビリティ (画面左下) ---*/
// 5グループまで表示し、超過分は「.....」行から履歴ポップアップへ。
const auto groups = BuildAbilityGroups();
if (!groups.empty())
{
    const int groupCount = static_cast<int>(groups.size());
    const int visible = (groupCount < ABILITY_VISIBLE_MAX)
                        ? groupCount : ABILITY_VISIBLE_MAX;
    const bool showMore = groupCount > ABILITY_VISIBLE_MAX;
    const int rowCount = visible + (showMore ? 1 : 0);
    const float listTop = AbilityListTop(rowCount);
    Text::Draw(ABILITY_LIST_X, listTop, "所持アビリティ",
                TEXT_SIZE_HUD, COLOR_TEXT);

    const auto& mouse = InputManager_GetMouseState();
    const bool playerTurn =
        (g_State.result == MatchResult::Playing
         && g_State.currentPlayer == Piece::Player
         && !IsEffectAnimating()
         && !g_AbilityPopupOpen);

    for (int i = 0; i < visible; ++i)
    {
        const AbilityGroup& g = groups[i];
        const float ry = AbilityRowY(i, rowCount);

        if (g.rep->activatable)
        {
            // 任意発動(消費系): クリック可能ボタン
            const bool usable = playerTurn && GroupCanActivate(g);
            const bool hover = usable
                                && mouse.x >= ABILITY_LIST_X
                                && mouse.x <= ABILITY_LIST_X + ABILITY_BTN_W
                                && mouse.y >= ry && mouse.y <= ry + ABILITY_BTN_H;

            // 縁取り (発動可ならアクセント色、不可なら半透明)
            DirectX::XMFLOAT4 accent = usable ? COLOR_SLIDE_ARROW
                                                : COLOR_TEXT_HINT;

            if (!usable) accent.w = 0.40f;
            Prim::DrawRect(ABILITY_LIST_X - 3.0f, ry - 3.0f,
                           ABILITY_BTN_W + 6.0f, ABILITY_BTN_H + 6.0f, accent);
            Prim::DrawRect(ABILITY_LIST_X, ry, ABILITY_BTN_W, ABILITY_BTN_H,
                           hover ? COLOR_BTN_HOVER : COLOR_BTN_BG);

            // ラベル (名前 + 取得数×N + 残りチャージ)
            char countPart[16] = "";
            if (g.count > 1)
                std::snprintf(countPart, sizeof(countPart), " ×%d", g.count);
            char label[128];
            const int charges = GroupCharges(g);
            if (charges >= 0)
                std::snprintf(label, sizeof(label), "%s%s 残%d",
                               g.rep->name.c_str(), countPart, charges);
        }
    }
}

```

```

        else
            std::snprintf(label, sizeof(label), "%s%s",
                           g.rep->name.c_str(), countPart);
        DirectX::XMFLOAT4 txtCol = COLOR_TEXT;
        if (!usable) txtCol.w = 0.5f;
        Text::Draw(ABILITY_LIST_X + 12.0f,
                    ry + (ABILITY_BTN_H - TEXT_SIZE_HINT) * 0.5f,
                    label, TEXT_SIZE_HINT, txtCol);
    }
    else
    {
        // パッシブ: テキスト表示のみ
        char rowBuf[128];
        if (g.count > 1)
            std::snprintf(rowBuf, sizeof(rowBuf), ". %s x%d",
                           g.rep->name.c_str(), g.count);
        else
            std::snprintf(rowBuf, sizeof(rowBuf), ". %s",
                           g.rep->name.c_str());
        Text::Draw(ABILITY_LIST_X,
                    ry + (ABILITY_ROW_H - TEXT_SIZE_HINT) * 0.5f,
                    rowBuf, TEXT_SIZE_HINT, COLOR_TEXT_HINT);
    }
}

/*--- 超過分: 「.....」 行 (クリックで履歴ポップアップを開く) ---*/
if (showMore)
{
    const float ry = AbilityRowY(visible, rowCount);
    const bool hover = !g_AbilityPopupOpen
        && mouse.x >= ABILITY_LIST_X
        && mouse.x <= ABILITY_LIST_X + ABILITY_BTN_W
        && mouse.y >= ry && mouse.y <= ry + ABILITY_BTN_H;
    Prim::DrawRect(ABILITY_LIST_X - 3.0f, ry - 3.0f,
                   ABILITY_BTN_W + 6.0f, ABILITY_BTN_H + 6.0f,
                   COLOR_TEXT_HINT);
    Prim::DrawRect(ABILITY_LIST_X, ry, ABILITY_BTN_W, ABILITY_BTN_H,
                   hover ? COLOR_BTN_HOVER : COLOR_BTN_BG);
    char moreLabel[64];
    std::snprintf(moreLabel, sizeof(moreLabel),
                   "..... 他%d件を見る", groupCount - visible);
    Text::Draw(ABILITY_LIST_X + 12.0f,
                ry + (ABILITY_BTN_H - TEXT_SIZE_HINT) * 0.5f,
                moreLabel, TEXT_SIZE_HINT, COLOR_TEXT);
}
}

}

//=====
// ボタンUI
//=====
/**
 * @struct UiButton
 * @brief 矩形ボタン1個の領域とラベル
 */
struct UiButton
{
    float x, y, w, h;    // ボタンの矩形領域
    const char* label;   // 表示ラベル
};

/**
 * @brief 座標がボタン領域内かを判定する
 * @param b 対象ボタン

```

```

* @param mx 判定するX座標
* @param my 判定するY座標
* @return 領域内なら true
*/
bool ButtonContains(const UIButton& b, int mx, int my)
{
    return mx >= b.x && mx <= b.x + b.w
        && my >= b.y && my <= b.y + b.h;
}

/**
* @brief ボタンを描画する (縁取り + 本体 + 中央ラベル)
* @param b 対象ボタン
* @param hover ホバー中なら true (本体色が変わる)
* @param accent 縁取りのアクセント色
*/
void DrawButton(const UIButton& b, bool hover, const DirectX::XMFLLOAT4& accent)
{
    constexpr float border = 3.0f;
    Prim::DrawRect(b.x - border, b.y - border,
        b.w + border * 2.0f, b.h + border * 2.0f, accent);
    Prim::DrawRect(b.x, b.y, b.w, b.h,
        hover ? COLOR_BTN_HOVER : COLOR_BTN_BG);
    const float tw = Text::MeasureWidth(b.label, TEXT_SIZE_BUTTON);
    Text::Draw(b.x + (b.w - tw) * 0.5f,
        b.y + (b.h - TEXT_SIZE_BUTTON) * 0.5f,
        b.label, TEXT_SIZE_BUTTON, COLOR_TEXT);
}

//-----
// ゲームオーバー画面のボタン定義
//-----
/**
* @enum GameOverAction
* @brief ゲームオーバー画面のボタンが実行する操作
*/
enum class GameOverAction { Reward, Restart, Title };

/**
* @struct GameOverButton
* @brief ゲームオーバー画面のボタン1個 (領域 + 操作 + 色)
*/
struct GameOverButton
{
    UIButton btn; // ボタン領域とラベル
    GameOverAction action; // クリック時の操作
    DirectX::XMFLLOAT4 accent; // 縁取り色
};

/**
* @brief 現在の結果に応じたボタナー覧を生成する
* @return ゲームオーバー画面に表示するボタン群
* @detail 勝利時は「報酬を受け取る」1個、それ以外は
*         「リスタート」「タイトルへ」の2個。Update/Draw で共有する。
*/
std::vector<GameOverButton> BuildGameOverButtons()
{
    std::vector<GameOverButton> list;
    const float screenW = static_cast<float>(Direct3D_GetBackBufferWidth());
    const float screenH = static_cast<float>(Direct3D_GetBackBufferHeight());
    const float by = screenH * 0.5f + 30.0f;

    if (g_State.result == MatchResult::Win)
    {

```

```

        // 勝利：報酬画面へ進むボタン1個
        const float x = (screenW - BUTTON_W) * 0.5f;
        list.push_back({ { x, by, BUTTON_W, BUTTON_H, "報酬を受け取る" },
                        GameOverAction::Reward, COLOR_TEXT_WIN });
    }
    else
    {
        // 敗北/引分/時間切れ：リスタート と タイトルへ の2個
        const float totalW = BUTTON_W * 2.0f + BUTTON_GAP;
        const float x0 = (screenW - totalW) * 0.5f;
        list.push_back({ { x0, by, BUTTON_W, BUTTON_H, "リスタート" },
                        GameOverAction::Restart, COLOR_PIECE_0 });
        list.push_back({ { x0 + BUTTON_W + BUTTON_GAP, by,
                        BUTTON_W, BUTTON_H, "タイトルへ" },
                        GameOverAction::Title, COLOR_TEXT_HINT });
    }
    return list;
}

/**
 * @brief ゲームオーバー時の暗転オーバーレイと結果表示を描画する
 * @detail プレイ中は何も描かない。結果メッセージを表示し、
 *          一定時間 (GAMEOVER_MIN_DWELL) 経過後にボタンを表示する。
 */
void DrawGameOverOverlay()
{
    if (g_State.result == MatchResult::Playing) return;

    const float screenW = static_cast<float>(Direct3D_GetBackBufferWidth());
    const float screenH = static_cast<float>(Direct3D_GetBackBufferHeight());

    // 画面全体を暗転
    Prim::DrawRect(0, 0, screenW, screenH, { 0.0f, 0.0f, 0.05f, 0.55f });

    /*--- 結果メッセージ ---*/
    const char* msg = "";
    DirectX::XMFLOAT4 color = COLOR_TEXT;
    switch (g_State.result)
    {
    case MatchResult::Win:      msg = "勝利！";    color = COLOR_TEXT_WIN; break;
    case MatchResult::Lose:     msg = "敗北";      color = COLOR_TEXT_LOSE; break;
    case MatchResult::Draw:     msg = "引き分け";  color = COLOR_TEXT_DRAW; break;
    case MatchResult::Timeout:  msg = "時間切れ";  color = COLOR_TEXT_LOSE; break;
    default: break;
    }
    const float msgW = Text::MeasureWidth(msg, TEXT_SIZE_BIG);
    Text::Draw((screenW - msgW) * 0.5f,
                screenH * 0.5f - TEXT_SIZE_BIG - 20.0f,
                msg, TEXT_SIZE_BIG, color);

    // 結果を見せるための待機後にボタンを表示
    if (g_GameOverDwell < GAMEOVER_MIN_DWELL) return;

    /*--- 操作ボタン (ホバー判定込み) ---*/
    const auto& mouse = InputManager_GetMouseState();
    for (const auto& go : BuildGameOverButtons())
    {
        const bool hover = ButtonContains(go.btn, mouse.x, mouse.y);
        DrawButton(go.btn, hover, go.accent);
    }
}

//=====
// アビリティ履歴ポップアップ

```

```

//=====
/**
 * @brief ポップアップの「閉じる」ボタン領域を返す
 * @param screenW 画面幅
 */
UIButton PopupCloseButton(float screenW)
{
    return { screenW - 160.0f, 28.0f, 130.0f, 44.0f, "閉じる" };
}

/**
 * @brief ポップアップのカード横スクロール最大量を返す
 * @param total カード総数
 * @param screenW 画面幅
 * @return スクロール可能量(px)。カードが画面に収まる場合は 0。
 */
float PopupMaxScroll(int total, float screenW)
{
    const float stripW = total * POPUP_CARD_W
        + (total - 1) * POPUP_CARD_GAP;
    const float viewport = screenW - POPUP_SIDE_MARGIN * 2.0f;
    return (stripW > viewport) ? (stripW - viewport) : 0.0f;
}

/**
 * @brief index 番目のポップアップカードの左端X座標を返す
 * @param index カード番号 (0始まり)
 * @param total カード総数
 * @param screenW 画面幅
 * @detail カード帯が画面に収まる場合は中央寄せ、溢れる場合は
 * スクロール量 g_PopupScroll を反映する。
 */
float PopupCardX(int index, int total, float screenW)
{
    const float stripW = total * POPUP_CARD_W
        + (total - 1) * POPUP_CARD_GAP;
    const float viewport = screenW - POPUP_SIDE_MARGIN * 2.0f;
    const float baseX = (stripW <= viewport)
        ? (screenW - stripW) * 0.5f
        : (POPUP_SIDE_MARGIN - static_cast<float>(g_PopupScroll));
    return baseX + index * (POPUP_CARD_W + POPUP_CARD_GAP);
}

/**
 * @brief アビリティ履歴ポップアップを描画する
 * @detail 所持アビリティ全件を横並びカードで表示する。
 * 任意発動アビリティはクリックで発動できる。
 */
void DrawAbilityPopup()
{
    if (!g_AbilityPopupOpen) return;
    if (g_State.result != MatchResult::Playing) return;

    const float screenW = static_cast<float>(Direct3D_GetBackBufferWidth());
    const float screenH = static_cast<float>(Direct3D_GetBackBufferHeight());

    // 背景を暗転 (ゲーム画面・HUDより手前)
    Prim::DrawRect(0, 0, screenW, screenH, { 0.0f, 0.0f, 0.05f, 0.85f });

    // 見出し
    const char* title = "所持アビリティー一覧";
    const float titleW = Text::MeasureWidth(title, TEXT_SIZE_HUD);
    Text::Draw((screenW - titleW) * 0.5f, 30.0f,
        title, TEXT_SIZE_HUD, COLOR_TEXT);

```



```

// 閉じるボタン
const auto& mouse = InputManager_GetMouseState();
const UIButton closeBtn = PopupCloseButton(screenW);
DrawButton(closeBtn, ButtonContains(closeBtn, mouse.x, mouse.y),
            COLOR_TEXT_HINT);

const auto groups = BuildAbilityGroups();
const int total = static_cast<int>(groups.size());
const float cardY = (screenH - POPUP_CARD_H) * 0.5f + 10.0f;
const bool playerTurn =
    (g_State.currentPlayer == Piece::Player && !IsEffectAnimating());

/*--- カードを横並びで描画 (画面外は省略) ---*/
for (int i = 0; i < total; ++i)
{
    const AbilityGroup& g = groups[i];
    const float cx = PopupCardX(i, total, screenW);
    if (cx + POPUP_CARD_W < 0.0f || cx > screenW) continue;

    const auto rarityCol = RarityColor(g.rep->rarity);
    const bool activatable = g.rep->activatable;
    const bool usable = activatable && playerTurn && GroupCanActivate(g);
    const bool hover = usable
        && mouse.x >= cx && mouse.x <= cx + POPUP_CARD_W
        && mouse.y >= cardY && mouse.y <= cardY + POPUP_CARD_H;

    // 縁取り (レアリティ色) + 本体
    Prim::DrawRect(cx - 4.0f, cardY - 4.0f,
                  POPUP_CARD_W + 8.0f, POPUP_CARD_H + 8.0f, rarityCol);
    Prim::DrawRect(cx, cardY, POPUP_CARD_W, POPUP_CARD_H,
                  hover ? COLOR_BTN_HOVER : COLOR_BTN_BG);

    // レアリティ表記
    const char* rar = RarityName(g.rep->rarity);
    const float rarW = Text::MeasureWidth(rar, TEXT_SIZE_HINT);
    Text::Draw(cx + (POPUP_CARD_W - rarW) * 0.5f, cardY + 14.0f,
              rar, TEXT_SIZE_HINT, rarityCol);

    // アビリティ名 (+ 取得数)
    char nameBuf[96];
    if (g.count > 1)
        std::snprintf(nameBuf, sizeof(nameBuf), "%s x%d",
                      g.rep->name.c_str(), g.count);
    else
        std::snprintf(nameBuf, sizeof(nameBuf), "%s",
                      g.rep->name.c_str());
    const float nameW = Text::MeasureWidth(nameBuf, TEXT_SIZE_HUD);
    Text::Draw(cx + (POPUP_CARD_W - nameW) * 0.5f, cardY + 48.0f,
              nameBuf, TEXT_SIZE_HUD, COLOR_TEXT);

    // 説明文 (改行は description 内の \n で明示済み)
    Text::Draw(cx + 14.0f, cardY + 96.0f,
              g.rep->description.c_str(), TEXT_SIZE_HINT, COLOR_TEXT_HINT);

    // 任意発動: 残りチャージ / 操作ヒント
    if (activatable)
    {
        const int charges = GroupCharges(g);
        char useBuf[64];
        if (charges >= 0)
            std::snprintf(useBuf, sizeof(useBuf), "残り %d 回", charges);
        else
            std::snprintf(useBuf, sizeof(useBuf), "クリックで発動");
    }
}

```

```

        const DirectX::XMFLOAT4 useCol = usable ? COLOR_SLIDE_ARROW
                                                : COLOR_TEXT_HINT;

        const float useW = Text::MeasureWidth(useBuf, TEXT_SIZE_HINT);
        Text::Draw(cx + (POPUP_CARD_W - useW) * 0.5f,
                   cardY + POPUP_CARD_H - 34.0f,
                   useBuf, TEXT_SIZE_HINT, useCol);
    }
}

/*--- 操作ヒント ---*/
const char* hint = (PopupMaxScroll(total, screenW) > 0.0f)
    ? "ホイールで左右スクロール / 消費アビリティはクリックで発動"
    : "消費アビリティはクリックで発動";
const float hintW = Text::MeasureWidth(hint, TEXT_SIZE_HINT);
Text::Draw((screenW - hintW) * 0.5f, screenH - 36.0f,
           hint, TEXT_SIZE_HINT, COLOR_TEXT_HINT);
}

/**
 * @brief アビリティ履歴ポップアップの入力を処理する
 * @detail ホイールでスクロール、カードクリックで発動、
 *          「閉じる」ボタン/カード外クリックで閉じる。
 *          ポップアップ表示中もゲーム時間は進行する。
 */
void HandleAbilityPopup()
{
    const float screenW = static_cast<float>(Direct3D_GetBackBufferWidth());
    const float screenH = static_cast<float>(Direct3D_GetBackBufferHeight());

    const auto groups = BuildAbilityGroups();
    const int total = static_cast<int>(groups.size());

    /*--- ホイールスクロール (範囲内へクランプ) ---*/
    const float maxScroll = PopupMaxScroll(total, screenW);
    g_PopupScroll -= g_WheelDelta * POPUP_SCROLL_SPEED;
    if (g_PopupScroll < 0.0) g_PopupScroll = 0.0;
    if (g_PopupScroll > maxScroll) g_PopupScroll = maxScroll;

    if (!InputManager_IsMouseLeftTrigger()) return;
    const auto& mouse = InputManager_GetMouseState();

    /*--- 閉じるボタン ---*/
    const UIButton closeBtn = PopupCloseButton(screenW);
    if (ButtonContains(closeBtn, mouse.x, mouse.y))
    {
        g_AbilityPopupOpen = false;
        return;
    }

    /*--- カードクリック ---*/
    const float cardY = (screenH - POPUP_CARD_H) * 0.5f + 10.0f;
    const bool playerTurn =
        (g_State.currentPlayer == Piece::Player && !IsEffectAnimating());
    for (int i = 0; i < total; ++i)
    {
        const float cx = PopupCardX(i, total, screenW);
        if (mouse.x < cx || mouse.x > cx + POPUP_CARD_W) continue;
        if (mouse.y < cardY || mouse.y > cardY + POPUP_CARD_H) continue;

        // カード命中: 任意発動かつ発動可能なら発動 (パッシブは何もしない)
        const AbilityGroup& g = groups[i];
        if (g.rep->activatable && playerTurn && GroupCanActivate(g))
        {
            GroupActivate(g);
        }
    }
}

```

```

    }
    return; // カード上のクリックは消費（ポップアップは閉じない）
}

/*--- カード外クリック：ポップアップを閉じる ---*/
g_AbilityPopupOpen = false;
}
}

//=====================================================
// 初期化／終了
//=====================================================
void Game_Initialize()
{
    Prim::Initialize();
    Text::Initialize();
    ResetMatch();
}

void Game_Finalize()
{
}

//=====================================================
// 更新
//=====================================================
void Game_Update(double elapsed_time)
{
    g_AnimClock += elapsed_time; // シェブロン脈動用クロック
    if (g_TimerFlash > 0.0) g_TimerFlash -= elapsed_time;

    // マウスホイールの変化量を算出（履歴ポップアップのスクロールに使用）
    {
        const int wheel = InputManager_GetMouseState().scrollWheelValue;
        g_WheelDelta = wheel - g_WheelPrev;
        g_WheelPrev = wheel;
    }

    // R はリスタートのショートカット（ゲームオーバー画面のボタンでも可）
    // ※ ESC はウィンドウ側で終了確認に割当て済みのため使用しない
    if (Keyboard_IsKeyDown(KK_R))
    {
        ResetMatch();
        return;
    }

    /*--- 効果アニメ再生中：入力/AI/タイマー/終了処理を止めフェーズを進める ---*/
    if (IsEffectAnimating())
    {
        g_AnimPhaseTimer -= elapsed_time;
        if (g_AnimPhaseTimer <= 0.0)
        {
            ++g_AnimPhaseIndex; // 次フェーズへ
            if (IsEffectAnimating()) g_AnimPhaseTimer = PHASE_DURATION;
        }
        if (g_LastPlacedTimer > 0.0)
        {
            g_LastPlacedTimer -= elapsed_time;
            if (g_LastPlacedTimer < 0.0) g_LastPlacedTimer = 0.0;
        }
        return;
    }

    /*--- プレイ中：タイマー消費 → 入力/アビリティ → AI ---*/

```

```

if (g_State.result == MatchResult::Playing)
{
    if (g_LastPlacedTimer > 0.0)
    {
        g_LastPlacedTimer -= elapsed_time;
        if (g_LastPlacedTimer < 0.0) g_LastPlacedTimer = 0.0;
    }

    HandleTurnTimer(elapsed_time);
    if (g_State.result == MatchResult::Playing)
    {
        if (g_AbilityPopupOpen)
        {
            // 履歴ポップアップ表示中：盤面入力は止めるが時間は進行する
            HandleAbilityPopup();
        }
        else
        {
            // アビリティUIのクリックを優先。消費しなければ駒設置へ。
            if (!HandleAbilityClick()) HandlePlayerInput();
        }
        HandleAiTurn(elapsed_time);
    }
    return;
}

/*--- ゲーム終了後：マウスでボタン操作 ---*/
g_GameOverDwell += elapsed_time;
if (g_GameOverDwell < GAMEOVER_MIN_DWELL) return;
if (!InputManager_IsMouseLeftTrigger()) return;

const auto& mouse = InputManager_GetMouseState();
for (const auto& go : BuildGameOverButtons())
{
    if (!ButtonContains(go.btn, mouse.x, mouse.y)) continue;
    switch (go.action)
    {
    {
        case GameOverAction::Reward:
            // 勝利：ボス番号を進めて報酬画面へ
            RunState::IncrementBoss();
            RunState::GenerateRewardChoices();
            Scene_Change(Scene::REWARD);
            break;
        case GameOverAction::Restart:
            // 同じボスへ再挑戦
            ResetMatch();
            break;
        case GameOverAction::Title:
            // タイトルへ戻る
            Scene_Change(Scene::TITLE);
            break;
    }
    }
    return;
}

}

//=====
// 描画
//=====
void Game_Draw()
{
    DrawBoardAndPieces(); // 盤面と駒
    DrawDirectionIndicator(); // 氷盤の滑り方向矢印
    DrawHud(); // HUD (残時間/手番/アビリティ)
}

```

```
DrawAbilityPopup();    // アビリティ履歴ポップアップ  
DrawGameOverOverlay(); // ゲームオーバー時のオーバーレイ  
}
```

第11章 ゼロからの構築手順

ここまでで各システムの中身を解説してきました。本章は、それらを**どの順番で組み上げるか**の地図です。「動くものを早く作り、後から拡張する」を原則に、9つのステップへ分けて示します。各ステップの詳細は、対応する章を参照してください。

構築の基本方針

いきなり完成形を目指さず、次の原則で進めます。

- **常に動く状態を保つ** — 各ステップの終わりに必ずビルドが通り、何かが動く。
- **素の〇×を先に完成させる** — ローグライト要素より前に、まず普通の〇×ゲームを遊べる状態にする。
- **フックは早めに、しかしデフォルト実装とともに入れる** — フック設計は後付けできない。だが最初は標準ルールのデフォルト実装だけで動かす。
- **1ボス・1アビリティで縦に貫通させる** — 横に機能を並べる前に、「ボス戦→撃破→報酬→次戦」の1周をまず通す。

ステップ1 基盤の確認と土台作り

まずZankuu基盤がビルド・起動し、空ウィンドウが出ることを確認します。3D専用モジュールは削除し、2Dで使うもの（ウィンドウ・Direct3D・スプライト・テクスチャ・入力・サウンド・タイマー・フェード）だけを残します。

- `assets/white.png`（1pxの白画像）を用意する。図形描画の素になる。
- プロジェクト設定に `/utf-8` コンパイルフラグを追加する（付録参照）。ソースを日本語コメント入りBOMなしUTF-8で書くために必須。
- `main.cpp` の初期化・メインループ・終了処理の骨格を組む（第1章）。

→ 第1章「全体アーキテクチャ」

ステップ2 基礎データ構造

ゲームの語彙を定義します。ヘッダだけで完結するので短時間で済みます。

- `piece.h` — `Piece`、`Vec2`、`Opponent`。
- `direction.h` — `Direction` と方向ヘルパ。

`game_state.h` もこの段階で器だけ作っておけますが、メンバは後のステップで増えていくので、まずは盤面と手番くらいから始めて構いません。

→ 第2章「基礎データ構造」

ステップ3 盤面と勝利判定

○×ゲームの心臓部となるデータと判定を作ります。

- `board.h/.cpp` — `Board` クラス。`Get / Set` の盤外安全化と `Resize` を最初から入れておく。
- `win_check.h/.cpp` — `HasLine`。`required` を引数にし、盤面サイズ非依存で書く。

この段階で簡単なテスト（盤面に駒を置いて `HasLine` が正しく `true` を返すか）をしておく、後が楽になります。

→ 第3章「盤面システム」

ステップ4 シーン骨格と描画

画面に絵を出せるようにします。

- `scene.h/.cpp` — シーン切り替え管理。タイトルとゲームの2つから始める。
- `render_primitives` — 図形描画。これで盤面のグリッドが描ける。
- `text_draw` + `font_atlas` — テキスト描画。最初はASCIIだけでもよいが、UIを日本語にするなら早めにアトラス方式にしておく。
- `title` — タイトルシーン（ロゴと開始プロンプト）。

→ 第1章・第8章・第9章

ステップ5 素の○×ゲームを完成させる

ここが最初の山場です。**フックもアビリティもまだ無い**、普通の○×ゲームを遊べる状態にします。ゲームシーン（`game.cpp`）に次を実装します。

- 盤面とグリッド、駒（○×）の描画。
- マウスクリック→マス判定→駒の設置。
- 着手後の勝敗判定（`win_check` を直接呼ぶ）。
- AI（`ai.cpp`） — 勝てる/防ぐ/ランダム。
- 思考時間タイマーと残り時間のHUD表示。
- 勝敗のオーバーレイ表示とリスタート。

この段階のゴール

「人 vs AI の○×が、制限時間つきで、最後まで遊べる」状態です。ローグライト要素はまだ何もなくて構いません。まず1本、遊べるゲームを通すことが重要です。

→ 第7章・第10章

ステップ6 フックの土台を差し込む

ここで設計の核心、フックを導入します。**ゲームの見た目・挙動は何も変えません**。内部配線だけを差し替えます。

- `hooks.h` — 6つのフックインターフェイスを定義。
- `default_hooks` — 標準ルール（3並び・1着手・180秒・3×3）をデフォルト実装として用意。
- `ability_registry` — フックを集約するレジストリ。
- ステップ5で直接書いた勝敗判定・思考時間・着手数・盤面サイズを、すべて `g_Registry` 経由の呼び出しへ置き換える。

置き換え後、デフォルト実装のおかげでゲームはステップ5と**まったく同じ**に動くはず。「動作は不変、しかし内部はフック経由」——この状態が、以降の拡張すべての土台になります。

順序の理由

フックはステップ5の「素の○×」が完成してから入れます。先にフックだけ作っても、通すべき実物のロジックが無く設計が空回りするためです。逆に、アビリティを作り始める 前には必ずフックを入れ終えていなければなりません。フックは後付けが効きません。

→ 第4章「フック設計」

ステップ7 ローグライトの1周を通す

「ボス戦→撃破→報酬→次戦」のループを、最小構成で1周させます。

- `ability.h` — アビリティ基底クラス。
- `boss.h` と1体のボス（`ice_board_boss`、ただしギミックは後回しでも可）。
- `run_state` — 所持アビリティ・ボス進行をシーン間で保持。
- `ability_pool` + `reward` シーン — 3択の報酬画面。
- 最初のアビリティを1つ実装する。パッシブで効果が分かりやすい **熟考**（思考時間+30秒、`ITurnTimeProvider` のチェーン例）が手頃。

これで「ボスを倒す→熟考を選ぶ→次の戦いで時間が増えている」という、ローグライトの コアループ が体験できます。

→ 第5章・第6章・第9章

ステップ8 アビリティとボスギミックを増やす

1周が通ったら、フックの実装例として機能を横に広げます。

- **余裕** — 熟考の派生（パラメータ違いだけ）。
- **二手打ち** — `ITurnHandler` のチェーン例。
- **盤面拡張** — `IBoardModifier` の例。`CellSize()` の 動的化 と野線描画の可変対応もここで行う。

- `board_ops` + 氷盤（ボスギミック） + 氷駒（その撃破報酬） — `IPlacementHandler` の例。
- 効果アニメーション — 氷盤/氷駒で駒が動くので、フェーズ方式のアニメ再生をここで導入する。瞬間的な盤面変化は避ける。

→ 第3章・第5章・第6章・第10章

ステップ9 アクティブアビリティとUIの仕上げ

最後に、任意発動アビリティと、それを操作するUIを整えます。

- 集中 — 任意発動アビリティ（`activatable`、チャージ制）。
- HUDの所持アビリティ表示 — 同名グルーピング（「氷駒 ×3」）、消費系優先の並び、クリック発動ボタン。
- アビリティ履歴ポップアップ — 5件超を「……」へ格納、横スクロール一覧。
- 仕上げ — 滑走方向の可視化、敗北判定の取りこぼし修正、フェード演出など。

→ 第5章・第8章・第10章

全体ロードマップ（早見表）

ステップ	作るもの	到達点	主な章
1	基盤確認・main.cpp骨格	空ウィンドウが出る	1
2	piece / direction	データ語彙が揃う	2
3	board / win_check	盤面と勝敗判定ができる	3
4	scene / 描画 / title	画面に図形と文字が出る	1,8,9
5	素の○×ゲーム + AI	普通の○×が遊べる	7,10
6	フック土台 + デフォルト実装	動作不変・内部はフック経由	4
7	ボス/報酬/熟考	ローグライト1周が回る	5,6,9
8	各アビリティ・氷盤・アニメ	ルールが書き換わる体験	3,5,6,10
9	集中・HUD・履歴UI・仕上げ	プロト版完成	5,8,10

最重要の一点

9ステップの中で唯一「順序を絶対に守るべき」なのは、**ステップ5（素の○×） → ステップ6（フック） → ステップ7以降（アビリティ）**の並びです。フックは後付けが効かないため、アビリティを書き始める前に必ずフック土台を入れ終えてください。それ以外のステップは、状況に応じて多少前後しても構いません。

付録 ビルド設定・拡張のヒント・早見表

A.1 ビルド設定

開発環境は Visual Studio 2022 / C++20 / DirectX 11、構成は `Debug` / `Release` の `x64` です。ゲーム層を組む上で、特に注意すべきプロジェクト設定が1つあります。

`/utf-8` コンパイルフラグ

本プロジェクトのソースは、日本語コメントを含む**BOMなしUTF-8**で書かれています。MSVCはBOMが無いソースを既定でShift-JISと解釈するため、そのままでは日本語コメントや文字列リテラルが文字化け・コンパイルエラーになります。プロジェクトの追加オプションに `/utf-8` を指定することで、ソースをUTF-8として正しく扱えます。これは ゲーム層を書き始める前（構築ステップ1）に必ず設定してください。

ビルドはVisual Studio上のほか、MSBuildでも行えます。新しいソースファイルを追加したときは、`.vcxproj` に `ClCompile` / `ClInclude` エントリを、`.vcxproj.filters` にフィルタ（ソリューションエクスプローラ上の仮想フォルダ）エントリを追加します。

A.2 拡張のヒント

本設計の狙いは「拡張のしやすさ」です。代表的な3つの拡張を、レシピとして示します。

新しいアビリティを追加する

1. どのフックでルールを書き換えるかを定める（例：思考時間なら `ITurnTimeProvider`）。
2. `game/ability/abilities/` に `oo.h/.cpp` を作り、`Ability` と対象フックを多重継承する。
3. コンストラクタで名前・説明・レアリティを設定する。
4. パッシブなら `RegisterTo` でレジストリへ登録（単一フックは `Setoo` でチェイン、複数フックは `Addoo`）。アクティブなら `activatable = true` とし `Activate` 等を実装する。
5. フックのメソッドを実装する。チェイン型なら「自分の関心事だけ変え、残りは `chained` へ委譲」を守る。
6. `ability_pool.cpp` の `CreateAll()` に追加して、報酬抽選の候補に含める。
7. `.vcxproj` / `.vcxproj.filters` にファイルを登録する。

既存のパラメータ違いで済むなら、「余裕」が「熟考」を継承したように、**既存アビリティを継承してコンストラクタだけ書き換える**のが最も簡単です。

新しいボスを追加する

1. ボスのギミックを、アビリティとして実装する（`Ability` + 必要なフック）。

2. `Boss` を継承した派生クラスを作り、`GetBossAbilities()` でそのギミックを、`GetRewardAbility()` で撃破報酬（ギミックの弱体化版）を返す。
3. ゲームシーンの `LoadBoss()` が、ボス番号に応じて生成するボスを切り替えられるようにする（プロト版は固定生成）。

新しいフック（ルールの種類）を追加する

盤面サイズフック `IBoardModifier` を後から追加したときの手順が、そのまま雛形になります。

1. `hooks.h` にインターフェイスを定義する。
2. `default_hooks` に標準実装を追加し、`AbilityRegistry::ResetToDefaults` でセットする。
3. `AbilityRegistry` に保持メンバと `Set/Add`・呼び出しヘルパを足す。
4. ゲーム本体の該当箇所を、レジストリ経由の呼び出しへ置き換える。

A.3 フック早見表

「どのルールを変えたいときに、どのフックを使うか」の対応表です。

変えたいルール	フック	登録	実装例
勝利条件	<code>IWinCondition</code>	単一・チェーン	（α版の拡張勝利など）
駒設置時の効果	<code>IPlacementHandler</code>	複数・追加	氷駒・氷盤
1ターンの着手数	<code>ITurnHandler</code>	単一・チェーン	二手打ち・氷盤
思考時間	<code>ITurnTimeProvider</code>	単一・チェーン	熟考・余裕
敗北の打ち消し	<code>IDefeatHandler</code>	複数・追加	（α版の不死など）
盤面サイズ	<code>IBoardModifier</code>	単一・チェーン	盤面拡張
任意発動の即時効果	（フック不要・ <code>Activate</code> ）	—	集中

A.4 処理フロー早見表

本書で解説した主要なフローを、最後にまとめます。

[アプリ起動]

WinMain → 基盤層初期化 → `Scene_Initialize` → メインループ

[1フレーム]

`Scene_Update` → `Scene_Draw` → `Direct3D_Present` → `Scene_Refresh`

[試合開始]

`ResetMatch` → `LoadBoss`（アビリティ登録）→ `GetBoardSize/Resize` → `BeginTurn`

[1回の着手]

入力 or AI → `PlaceAndEvaluate`

- OnPlace（設置フック発火・animPhasesへ移動を積む）
- StartEffectAnimation（フェーズ展開）
- EvaluateAfterPlace（両陣営の勝敗→引分→追加着手→ターン終了）

[ボス撃破後]

- 結果画面で「報酬を受け取る」→ IncrementBoss → GenerateRewardChoices
- Scene::REWARD（3択）→ 選択 → PlayerAbilitiesへ追加 → Scene::GAME

おわりに

本書は、1つの○×ローグライトを題材に「ルールを差し替え可能にする設計」を解説してきました。フックという1つの考え方が、アビリティ・ボスギミック・盤面構造・UIまでを貫いていることを読み取っていただけたなら幸いです。同じ設計は、カードゲームや他のローグライトにも応用できます。本書が、拡張に強いゲームを設計するための一例として役立つことを願っています。